Resumen Ingeniería de Software

Índice

Índice **1**
Unidad 1: Ingeniería de software **7**
- Software 7
  - Definiciones de software 7
  - Componentes de software 7
  - Características del software 7
  - Tipos de software 7
  - Aplicaciones del software 8
- Ingeniería del Software 8
  - Definición Ingeniería de Software 8
  - Etapas 8
  - Objetivo principal 8
  - Principios de la ingeniería del software 9
  - Capas 9
- Ciclos de vida del desarrollo del software 10
  - Ciclos de vida 10
  - Modelos de ciclos de vida 10
  - ISO/IEC 12207 13
- Metodologías de desarrollo de software 14
  - Definición de metodología 14
  - Ventajas del uso de una metodología 14
  - Metodologías tradicionales y ágiles 15
- Desarrollo iterativo e incremental 16
  - Idea básica 16
  - Debilidades en el modelo 17
  - Rapid Application Development (RAD) 17
  - Rational Unified Process (RUP) 17
    - Módulos 18
    - Fases del ciclo de vida del proyecto 18
  - Desarrollo Ágil 18
- Desarrollo Ágil 19
  - Comparación con otros métodos 19
  - Idoneidad de los métodos ágiles 20
  - Manifiesto Ágil 20
  - Métodos Ágiles 21
    - Gestión de proyectos 21
    - Extreme Programming (XP) 22
    - SCRUM 24
    - Dynamic Systems Development Method (DSDM) 25

Crystal Clear 27
Agile Unified Process (AUP) 27
Prácticas ágiles 28
Test Driven Development (TDD) 28
Integración continua 28
Pair Programming 29
Críticas al desarrollo ágil 30
Unidad 1: Calidad **31**
Definiciones 31
Elementos que influyen en la calidad 31
Metodología 31
Modelo de calidad 32
Ventajas 32
Niveles de gestión de calidad 32
Actividades de pruebas 32
Conceptos 33
Aseguramiento de la calidad y Control de la calidad 33
Tareas en aseguramiento de calidad 34
Entorno de gestión de calidad 34
Métricas 34
Defectos 35
Unidad 2: Proceso de desarrollo de Software **36**
Introducción 36
Calidad y Proceso 36
Procesos 36
Calidad 36
CMMI 36
Representación Contínua 37
Representación por Etapas 37
Área de Proceso 38
Constelaciones 39
Áreas 40
Gestión de configuración (CM) 40
Medición y Análisis (MA) 40
Definición de procesos de la organización (OPD) 40
Enfoque de procesos de la organización (OPF) 40
Formación organizativa (OT) 41
Monitorización y control del proyecto (PMC) 41
Planificación de proyecto (PP) 41
Aseguramiento de la calidad de proceso y de producto (PPQA) 42
Desarrollo de requerimientos (RD) 42
Gestión de requerimientos (REQM) 42
Gestión de riesgos (RSKM) 42
Validación (VAL) 42

Verificación (VER) 43
Estructura de desglose de trabajo (EDT) 43
SPEM 43
Capacidades 44
Características 44
Proceso según SPEM 44
Marco conceptual 44
RUP 45
Elementos del proceso 45
Fases e Iteraciones 47
Fase Inicial 47
Fase Elaboración 48
Fase Construcción 48
Fase Transición 48
Disciplinas 48
Modelado de negocio 48
Requisitos 49
Análisis y Diseño 49
Implementación 49
Prueba 49
Despliegue 49
Gestión de cambios y configuración 49
Gestión de Proyectos 50
Entorno 50
IBM Rational Method Composer (RMC) 50
Unidad 3: Gestión de proyectos **51**
Definiciones 51
Proyecto 51
Gestión de Proyectos 51
Procesos 52
Coordinación del proyecto 52
Alcance 53
Tiempo 53
Costes 53
Calidad 54
Recursos 54
Personal 54
Comunicaciones 54
Riesgos 55
Adquisiciones 55
Modelos de Gestión de Proyectos 56
PMBOK 56
ISO 10006:2003 56
METRICA 56

PRINCE2 56
Unidad 3: Puntos por función **57**
Introducción 57
Metodología 57
Unidad 4: Requisitos **60**
Introducción 60
Definición 60
Tipos de requisitos 60
Roles y responsabilidades 61
Desarrollo 62
Obtención de requisitos 62
Roles 63
Buenas prácticas 64
Técnicas 64
Definición 65
Verificación 66
Especificación 66
Priorización 67
Evolución 67
Gestión 68
Gestión de cambios 68
Evaluar el impacto 69
Aceptación del cambio 69
Implementación del cambio 69
Mejores Prácticas 70
En desarrollo 70
En gestión 70
Unidad 5: El arte de probar el software **72**
Psicología de la prueba 72
Economía de la prueba 72
Principios de prueba 72
Unidad 5: Derivar casos de prueba de casos de uso **74**
Unidad 5: Validación y Verificación **75**
CMMI 75
Metas y prácticas específicas 75
Validación 75
Verificación 75
Validación y verificación de adquisición 76
Validación 76
Verificación 77
Procesos de validación y verificación 77
Validación y verificación en el ciclo de vida 77
Modelo en cascada 77
Modelo en V 77

Modelo incremental y evolutivo 78
Modelo en espiral 79
Modelo de Prototipos 79
Actividades de validación y verificación 79
Pruebas 79
Estrategia de pruebas 81
Niveles de pruebas 83
Buenas prácticas 85
Tipos de pruebas 86
Revisiones 87
Técnicas y herramientas 88
Técnicas de pruebas 88
Técnicas de pruebas dinámicas 88
Técnicas de control estáticas 90
Herramientas de pruebas 92
Beneficios 92
Riesgos 92
Clasificación 92
Unidad 6: Gestión de Configuración 95
Conceptos 95
Roles y responsabilidades 95
Actividades 96
Identificar elementos de Configuración 97
Mecanismo de construcción 98
Mecanismo de liberación 98
Branching and merging 98
Auditoría de gestión de configuración 98
Unidad 7: Proceso de aseguramiento de calidad 100
Introducción 100
Proceso 100
Inicio/planificación del proyecto 100
Ejecución del proyecto 101
Cierre del proyecto 101
Reuniones de SQA 101
Métricas 101
Unidad 8: Métricas 103
Conceptos 103
Métrica y medida 103
GQM 104
Niveles 104
PSM 105
Clasificación de las métricas 106
Métricas de proyecto 106
Métricas a nivel de proceso 106

Métricas a nivel de producto 107
Métricas a nivel de servicio 107
Técnicas de análisis 107
Proceso de Medición y Análisis 108
Unidad 9: Análisis Forense digital 109
Introducción 109
Tipologías de los incidentes 111
Principios de la recolección de evidencias 111
Orden de volatilidad 112
Acciones que deben evitarse 112
Consideraciones de privacidad 112
Consideraciones legales 112
Procedimiento de recolección 112
Reproducible 112
Pasos 113
Procedimiento de almacenamiento 113
Cadena de custodia 113
Dónde y cómo almacenarlo 113
Herramientas necesarias 113
Unidad 9: Protocolo de actuación para pericias informáticas 114
Introducción 114
Objetivos 114
Procedimiento interno para pericias informáticas 114
Secuestro de tecnología informática 116

Unidad 1: Ingeniería de software

Software

Definiciones de software

- IEEE Std. 610: programas, procedimientos y documentación y datos asociados, relacionados con la operación de un sistema informático.
- Webster's New Collegiate Dictionary (1975): software es un conjunto de programas, procedimientos y documentación relacionada asociados con un sistema, especialmente un sistema informático.

Componentes de software

- Programas: conjuntos de instrucciones que proporcionan funcionalidad.
- Datos: componente de soporte para programas, mantenimiento y pruebas, guardan información relevante.
- Documentos: explicación de cómo usar, mantener y modificar el programa, manuales de usuario y de operación.

Características del software

- Es un elemento lógico y se diferencia del hardware, un elemento físico, en sus características.
- Software se desarrolla, no se fabrica.
- Cada producto software es diferente porque se construye para cumplir los requisitos únicos de un cliente y necesita un enfoque de ingeniería. La mayoría se construye a medida.
- Entender requisitos, diseñar el producto para que cumpla los requisitos, implementar el diseño usando un lenguaje de programación y comprobar que el producto cumple con los requisitos.
- El recurso principal son las personas. Añadir personas no siempre acelera el tiempo de desarrollo porque se requiere esfuerzo en equipo siguiendo un objetivo común y manteniendo comunicación efectiva. Un nuevo miembro en el equipo necesita una proceso de iniciación que cuesta tiempo y dinero para poder volverse productivo.
- No se estropea.
- Los componentes reutilizables de software permiten al ingeniero del software crear nuevas aplicaciones a partir de las partes reutilizables a través de librerías de funciones y objetos.

Tipos de software

- Software de aplicaciones: se usan para proveer servicios a clientes y ejecutar negocios de forma más eficiente.
- Software de sistemas: el software de sistemas se usa para operar y mantener un sistema informático. Permite a los usuarios usar los recursos del ordenador directamente y a través de otro software

Aplicaciones del software

- Software de sistemas
- Software de tiempo real
- Software de gestión
- Software de ingeniería y científico
- Software empotrado
- Software de computadoras personales
- Software basado en web
- Software de inteligencia artificial

Ingeniería del Software

Definición Ingeniería de Software

- Ingeniería del Software es el estudio de los principios y metodologías para el desarrollo y mantenimiento de sistemas de software (Zelkovitz, 1978)
- Ingeniería del Software es la aplicación práctica del conocimiento científico al diseño y construcción de programas de computadora y a la documentación asociada requerida para desarrollar, operar (funcionar) y mantenerlos. Se conoce también como desarrollo de software o producción de software (Bohem, 1976)
- Ingeniería del Software trata del establecimiento de los principios y métodos de la ingeniería a fin de obtener software de modo rentable que sea fiable y trabaje en máquinas reales (Bauer, 1972)
- La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación (funcionamiento) y mantenimiento del software; es decir, la aplicación de ingeniería al software. (IEEE, 1993). La definición de IEEE describe la ingeniería del software como un enfoque sistemático cubriendo los aspectos del desarrollo, operación y mantenimiento. Este enfoque es disciplinado y cuantificable.
- Representa un proceso formal que incorpora una serie de métodos bien definidos para el análisis, diseño, implementación y pruebas de software así como para gestión de proyectos.

Etapas

- Análisis de requisitos
- Especificación
- Diseño y arquitectura
- Programación
- Prueba
- Mantenimiento

Objetivo principal

Se centra en el uso de un enfoque sistemático para construir software de alta calidad. Existen un conjunto de prácticas que, al seguirse, aseguran un producto de alta calidad.
Relevancia de la ingeniería del software: las compañías suelen ponderar la velocidad de respuesta por sobre la ingeniería del software, la perciben como algo demasiado formal,

que lleva demasiado tiempo e inflexible. La ingeniería del software es adaptativa y por lo tanto, relevante para cualquiera que construya software. Es una filosofía que puede ser adaptada y aplicada a todas las actividades y dominios de aplicación del desarrollo de software. Presenta una colección de opciones, no existe un enfoque único posible, se debe elegir el correcto adaptado al producto, al equipo y al negocio. No es un dogma o religión y no existen verdades absolutas.

Principios de la ingeniería del software

- Trabajar dirigidos por la calidad. El compromiso del cliente es el factor más crítico en la calidad del software. La calidad es el principal objetivo; la productividad a largo plazo es una consecuencia de una alta calidad. Primero hazlo correctamente, luego hazlo rápido. (Calidad)
- Una buena gestión es más importante que una buena tecnología. Las personas y el tiempo no son intercambiables.
- Seleccionar el modelo de ciclo de vida adecuado.
- Realizar iteraciones en todas las fases del desarrollo excepto en la codificación. Entregar productos al usuario lo más pronto posible. (Feedback rápido)
- Determinar y acotar el problema antes de escribir los requisitos. (Scope)
- Las técnicas son anteriores a las herramientas.
- La entropía del software es creciente.
- Tener procedimientos escritos de desarrollo de software puede ayudar a crear una cultura compartida de buenas prácticas
- Haz que los errores los encuentre un colaborador, no un cliente. La gestión de errores y solicitud de cambios es esencial para controlar la calidad y el mantenimiento.
- La mejora continua de tu proceso de desarrollo de software es posible y esencial. Si mides lo que haces puedes aprender a hacerlo mejor. (Mejora contínua)
- Haz lo que tenga sentido; no recurras a los dogmas. (Pragmático)
- Establecer prioridades según mayores beneficios, no se puede hacer todo a la vez.

Capas

La Ingeniería de Software requiere que la organización esté centrada en conseguir calidad. Es una tecnología multicapa donde reposan las capas de herramientas , métodos y procesos sobre el enfoque de calidad.

- Procesos: un marco de trabajo que ayuda al jefe de proyecto a controlar la gestión del proyecto y las actividades de ingeniería. Es la unión que mantiene juntas las capas de tecnologías y permite un desarrollo racional de la ingeniería del software. Permite planificar la ejecución del proyecto, es una hoja de ruta del trabajo. Guía las personas proporcionando el marco de trabajo. Es el fundamento de la ingeniería del software y da soporte a las capas de métodos y herramientas.
  Los procesos proporcionan claridad acerca de cómo se debe realizar un trabajo, son necesarios en todos los proyectos dónde existe esfuerzo coordinado. Todos los enfoques de construcción de software tienen procesos, pero en muchos casos son ad hoc, invisibles, caóticos.
- Métodos: identifica las tareas de ingeniería que se deben realizar. Contiene los métodos definidos para realizar las actividades de la construcción de software

(análisis, diseño, codificación e implementación) de forma eficiente. Se seleccionan métodos de un amplio rango de métodos disponibles.
- Herramientas: proporciona soporte a las capas de proceso y métodos, centrándose en la automatización de las actividades manuales relacionadas a gestión de proyectos, métodos técnicos, soporte de sistemas genera y marcos de trabajo para otras herramientas. Vuelve el trabajo menos tedioso, reduce los errores y hace más fácil usar buenas prácticas.

Ciclos de vida del desarrollo del software

Ciclos de vida

Conjunto de fases por las que pasa el sistema que se está desarrollando desde que nace la idea inicial hasta que el software es retirado o reemplazado. También se denomina paradigma.
Define:
- Orden de las fases
- Criterios de transición entre fases
- Definir entradas y salidas de cada fase
- Estados por los que pasa el producto
- Esquema base para trabajar

Se compone de fases sucesivas que pueden o no tener retroalimentación, por lo que una misma fase puede ejecutarse varias veces, produciendo resultados intermedios antes de llegar al resultado final.
Fases: conjunto de actividades relacionadas con un objetivo en el desarrollo del proyecto. Se construye agrupando tareas con un tiempo de vida compatible.
Entregables: productos intermedios que generan las fases. Pueden ser materiales o inmateriales. Permiten evaluar el estado del proyecto en base a la adecuación a los requisitos.
Los ciclos de vida se diferencian entre sí por:
- El alcance
- Los características o contenidos de cada fase
- La estructura y sucesión de las etapas, si existe o no retroalimentación e iteraciones

Modelos de ciclos de vida

Modelos que establecen las distintas etapas y estados por los que pasa un producto software. Describen las fases del ciclo de software, el orden en que se ejecutan las fases y los criterios de transición entre las mismas.
Define las fases principales del desarrollo de software y las actividades primarias de cada una, ayuda a administrar el proceso de desarrollo y provee un espacio de trabajo para definir un proceso de desarrollo de software.
- Modelo en Cascada: ordena rigurosamente las etapas del ciclo de vida del software, para comenzar una etapa se debe haber finalizado la anterior. Es un proceso de desarrollo secuencial.
  Es apropiado para proyectos estables con requisitos estáticos dónde se puede predecir totalmente el área del problema el sistema. Proyectos pequeños bien entendidos. Es fácil de gestionar.

Los proyectos reales no tienen una secuencia lineal, es difícil que un cliente pueda establecer desde el principio todos los requerimientos necesarios.Los resultados se ven cuando el producto está terminado, no progresivamente, si aparecen requisitos nuevos durante la implementación, se debe volver a la fase de requerimientos. Es un modelo pobre para proyectos complejos, largos o con incertidumbre.

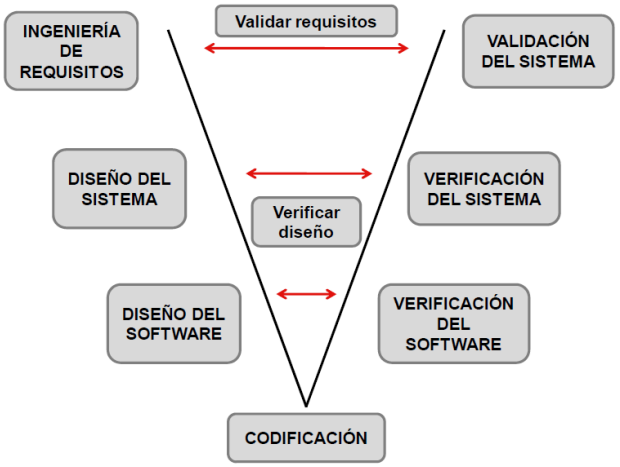
Existen algunas variantes que intentan mejorar algunos de los aspectos más criticados de este tipo de modelo. El modelo Sashimi, conocido también modelo en cascada con fases superpuestas o modelo en cascada con retroalimentación, superpone las fases y permite hacer trabajo de una etapa posterior a la etapa actual.

- Modelo en V: indica que las pruebas deben empezarse lo más pronto posible en el ciclo de vida, integra las actividades de prueba a cada fase del ciclo de vida. Cada uno de los productos de trabajo generados por desarrolladores y analistas de negocio son sujetos de pruebas.

  Las etapas individuales del proceso son similares a las del modelo en cascada, con la diferencia de tener se vuelven a repetir las etapas de validación y verificación luego de la codificación.

  Es un modelo simple y fácil de usar, cada fase tiene entregables específicos, tiene pruebas en etapas tempranas del ciclo de vida. Suele funcionar bien para proyectos pequeños y bien entendidos.

  Es muy rígido, como el modelo de cascada, tiene poca flexibilidad y es difícil ajustar el alcance, no produce prototipos de software, no proporciona caminos claros para problemas encontrados durante la fase de pruebas.



- Modelo iterativo: es un modelo derivado del ciclo de vida en cascada, reduce el riesgo que surge a partir de las diferencias entre las necesidades del usuario y el producto final.

  Se hacen iteraciones de varios ciclos de vida en cascada, al final de cada una se le entrega al cliente una versión mejorada o con mayores funcionalidades del producto. El cliente evalúa el producto, corrige y propone mejoras al final de cada iteración, las iteraciones se repetirán hasta obtener un producto que satisfaga las necesidades del cliente.

  Se crean prototipos para presentarlos y conseguir la conformidad del cliente cuando los requisitos no están tan claros por parte del usuario.

  No hace falta tener los requerimientos completos desde el inicio, se pueden ir refinando a cada iteración. Permite gestionar mejor los riesgos y las entregas.

Pueden ocurrir problemas con la elección de la arquitectura por tener desde el principio claro cuál será el alcance del proyecto.

- Modelo de desarrollo incremental: combina el modelo en cascada con la filosofía de la construcción de prototipos. Consiste en construir incrementando las funcionalidades del programa. Aplica secuencias lineales escalonadas, cada una produce un incremento de software.
  Cada entrega debe ser un subproducto operativo del producto final que se pueda usar y evaluar. El primer incremento suele ser un producto esencial con los requisitos más básicos.
  Genera un software operativo más rápido en etapas más tempranas del ciclo de vida, es flexible a los cambios de alcance y requisitos, las iteraciones pequeñas permiten probar y depurar con facilidad, gestiona mejor los riesgos y cada iteración es un hito fácilmente gestionable.
  Necesita experiencia para definir los incrementos y distribuir las tareas entre ellos. Las fases dentro de cada iteración son rígidas, no se superponen. Pueden ocurrir problemas con la elección de la arquitectura por tener desde el principio claro cuál será el alcance del proyecto.
- Modelo en espiral: las actividades se conforman en un espiral, cada bucle representa un conjunto de actividades que no están fijadas previamente sino que se van eligiendo en función del análisis de riesgos. Tiene en cuenta fuertemente el riesgo que aparece a la hora de desarrollar software, porque lo que requiere habilidades para gestionar riesgos.
  Se comienza mirando las posibles alternativas de desarrollo se va formando el espiral en base a los riesgos más asumibles, si el cliente quiere implementar cambios en el software, se vuelvan a evaluar las alternativas de desarrollo y riesgo y se realiza otra vuelta en el espiral.
  Combina las características del modelo de prototipos y el modelo en cascada, está pensado para proyectos largos, caros y complicados. Se suele interpretar como que dentro de cada ciclo de la espiral se sigue un modelo en cascada, pero no necesariamente ha de ser así.
  Reduce los riesgos del proyecto, incorpora objetivos de calidad, integra el desarrollo con el mantenimiento.
  Permite mejoras y nuevos requerimientos sin romper el modelo, ya que no es rígido ni estático.
  Genera una gran cantidad de trabajo adicional, es un modelo costoso y suele no funcionar bien para proyectos pequeños.
  La distinción más destacada entre el modelo en espiral y otros modelos de software es la tarea explícita de evaluación de riesgos. Para otros modelos la evaluación de riesgos es una subtarea menos importante. Además no hay fases fijadas para la especificación de requisitos, diseño y pruebas en el modelo en espiral. Se puede usar prototipado para encontrar y definir los requisitos.
- Modelo de prototipos: se usa cuando se pueden obtener los requisitos generales pero no se logran identificar los requisitos en detalle, a bajo nivel. Se realizan prototipos rápidos en base a los requisitos generales detectados, el cliente los evalúa y su feedback completa los requisitos en detalle que no estaban definidos antes de la implementación.
  Ofrece visibilidad del producto desde el inicio del ciclo de vida, permite detectar las necesidades reales y definir mejor los requisitos, hay una retroalimentación contínua

del cliente. Si hay un pedido de cambio, se puede introducir en la siguiente iteración.
El prototipo es un documento vivo de buen funcionamiento del producto final, el cliente comprende mucho mejor el estado del proyecto mediante el uso del prototipo que leyendo especificación escrita.
Puede llevar a un desarrollo lento y necesitar inversiones en productos prototipos desechables.
Pueden darse falsas expectativas con el cliente a partir del uso del prototipo.

ISO/IEC 12207

Es una norma que establece un marco de referencia común para los procesos del ciclo de vida del software con una terminología bien definida.
Define un modelo de ciclo de vida como un marco de referencia que contiene los procesos, actividades y tareas involucradas en el desarrollo, operación y mantenimiento de un producto software.
Incluye también un proceso que puede emplearse para definir, controlar y mejorar los procesos del ciclo de vida del software.
Agrupa las actividades que pueden llevarse a cabo durante el ciclo de vida del software en cinco procesos principales, ocho procesos de apoyo y cuatro procesos organizativos. Cada proceso del ciclo de vida está dividido en un conjunto de actividades; cada actividad se subdivide a su vez en un conjunto de tareas.

- Procesos principales del ciclo de vida: dan servicio a las partes principales durante el ciclo de vida del software.
    - Proceso de adquisición
    - Proceso de suministro
    - Proceso de desarrollo
    - Proceso de operación
    - Proceso de mantenimiento
- Procesos de apoyo al ciclo de vida: apoyan a otros procesos como parte esencial de los mismos, con un propósito bien definido, y contribuyen al éxito y calidad del proyecto software.
    - Proceso de documentación
    - Proceso de gestión de la configuración
    - Proceso de verificación
    - Proceso de validación
    - Proceso de revisiones conjuntas
    - Proceso de auditoría Ingeniería del software: Metodologías y ciclos de vida
    - Proceso de solución de problemas
- Procesos organizativos del ciclo de vida: formar una infraestructura construida por procesos y personal asociado al ciclo de vida, y para mejorar continuamente esta estructura y procesos.
    - Proceso de gestión
    - Proceso de infraestructura
    - Proceso de mejora
    - Proceso de formación

Metodologías de desarrollo de software

Definición de metodología

Es un conjunto integrado de técnicas y métodos que permite abordar de forma homogénea y abierta cada una de las actividades del ciclo de vida de un proyecto de desarrollo.
Las metodologías se basan en una combinación de los modelos de proceso genéricos (cascada, incremental...). Definen artefactos, roles y actividades, junto con prácticas y técnicas recomendadas.
Una metodología para el desarrollo de software comprende los procesos a seguir sistemáticamente para idear, implementar y mantener un producto software con altas posibilidades de éxito. Optimiza el proceso y el producto software, define métodos que guían la planificación y el desarrollo del software, define qué hacer, cómo y cuándo durante todo el desarrollo y mantenimiento de un proyecto.
Entre los elementos que forman parte de una metodología se pueden destacar:

- Fases: tareas a realizar en cada fase
- Productos: E/S de cada fase, documentos.
- Procedimientos y herramientas: apoyo a la realización de cada tarea
- Criterios de evaluación: del proceso y del producto. Saber si se han logrado los objetivos

Una metodología de desarrollo de software o metodología de desarrollo de sistemas en ingeniería de software es un marco de trabajo que se usa para estructurar, planificar y controlar el proceso de desarrollo de un sistema de información.
Una metodología de desarrollo de sistemas no tiene que ser necesariamente adecuada para usarla en todos los proyectos, cada una puede ser más adecuada para tipos de proyectos diferentes.
El marco de trabajo de una metodología de desarrollo de software consiste en:

- Una filosofía de desarrollo de software, con el enfoque o enfoques del proceso de desarrollo de software.
- Múltiples herramientas, modelos y métodos para ayudar en el proceso de desarrollo de software.

Estos marcos de trabajo están con frecuencia vinculados a algunos tipos de organizaciones, que se encargan del desarrollo, soporte de uso y promoción de la metodología. La metodología con frecuencia se documenta de alguna manera formal.

Ventajas del uso de una metodología

Desde el punto de vista de gestión:

- Facilitar la tarea de planificación
- Facilitar la tarea del control y seguimiento de un proyecto
- Mejorar la relación coste/beneficio
- Optimizar el uso de recursos disponibles
- Facilitar la evaluación de resultados y cumplimiento de los objetivos
- Facilitar la comunicación efectiva entre usuarios y desarrolladores

Desde el punto de vista de los ingenieros del software:

- Ayudar a la comprensión del problema
- Optimizar el conjunto y cada una de las fases del proceso de desarrollo
- Facilitar el mantenimiento del producto final

- Permitir la reutilización de partes del producto

Desde el punto de vista del cliente o usuario:

- Garantía de un determinado nivel de calidad en el producto final
- Confianza en los plazos de tiempo fijados en la definición del proyecto
- Definir el ciclo de vida que más se adecue a las condiciones y características del desarrollo

Metodologías tradicionales y ágiles

Según la filosofía de desarrollo se pueden clasificar las metodologías en dos grupos.

- Metodologías tradicionales: se basan en una fuerte planificación durante todo el desarrollo, se focalizan en la documentación, planificación y procesos. Centran su atención en llevar una documentación exhaustiva de todo el proyecto y en cumplir con un plan de proyecto, definido todo esto, en la fase inicial del desarrollo del proyecto. Tiene altos costos al implementar un cambio y la poca flexibilidad.
  El principal problema es que nunca se logra planificar bien el esfuerzo requerido para seguir la metodología, aunque si logramos definir métricas que apoyen la estimación de las actividades de desarrollo, muchas prácticas de metodologías tradicionales podrían ser apropiadas.
- Metodologías ágiles: el desarrollo de software es incremental, cooperativo, sencillo y adaptado. Se basa en retrasar las decisiones y la planificación adaptativa. Basan su fundamento en la adaptabilidad de los procesos de desarrollo. La capacidad de respuesta a un cambio es más importante que el seguimiento estricto de un plan.
  Una de las principales ventajas de los métodos ágiles es su peso inicialmente ligero, resultando agradable para personas que no están acostumbradas a trabajar con metodologías. El uso del método ágil no vale para cualquier proyecto.

Tener metodologías diferentes para aplicar de acuerdo con el proyecto que se desarrolle resulta una idea interesante, pueden involucrar prácticas tanto de metodologías ágiles como de metodologías tradicionales. De esta manera podríamos tener una metodología diferente para cada proyecto.

Metodologías ágiles	**Metodologías tradicionales**
Basadas en heurísticas provenientes de prácticas de producción de código	Basadas en normas provenientes de estándares seguidos por el entorno de desarrollo
Impuestas internamente (por el equipo)	Impuestas externamente
Proceso menos controlado, con pocos principios	Proceso mucho más controlado, con numerosas políticas/normas
No existe contrato tradicional o al menos es bastante flexible	Existe un contrato prefijado
El cliente es parte del equipo de desarrollo	El cliente interactúa con el equipo de desarrollo mediante reuniones
Grupos pequeños (<10 integrantes) y	Grupos grandes y posiblemente distribuidos

trabajando en el mismo sitio	
Pocos artefactos	Pocos artefactos Más artefactos
Pocos roles	Más roles
Menos énfasis en la arquitectura del software	La arquitectura del software es esencial y se expresa mediante modelos

Desarrollo iterativo e incremental

El desarrollo iterativo e incremental es un proceso de desarrollo de software cíclico desarrollado en respuesta a la debilidad del modelo en cascada. Es una parte esencial de RUP, de DSDM, XP y generalmente de los marcos de trabajo de desarrollo de software ágil.
Desarrollo incremental: estrategia programada y en etapas, en la que las diferentes partes del sistema se desarrollan en diferentes momentos o a diferentes velocidades, y se integran a medida que se completan.
Desarrollo iterativo: estrategia de programación de reproceso en la que el tiempo se separa para revisar y mejorar partes del sistema, no presupone desarrollo incremental.

Idea básica

La mejora iterativa busca desarrollar un sistema software incrementalmente, permitiendo al desarrollador aprovechar lo producido en las versiones anteriores como conocimiento en las versiones posteriores. Se comienza con un subconjunto simple de requisitos y se mejora iterativamente las versiones hasta que se implementa el sistema entero. En cada iteración, se hacen modificaciones del diseño y se añaden nuevas capacidades. Divide el valor del negocio entregable en iteraciones.
El procedimiento consiste:

- Inicialización: crea una versión base del sistema, busca realizar un producto que los usuarios puedan evaluar y dar feedback. Identifica los aspectos clave del problema y propone una solución simple y fácil de entender.
- Iteración: implica el rediseño y la implementación de una tarea de la lista de control del proyecto y el análisis de la versión actual del sistema. El resultado debe ser simple, sencillo y modular. Cada iteración realiza el análisis en base a la retroalimentación del usuario y las facilidades de análisis del programa disponibles. La lista de control del proyecto se modifica en vista de los resultados del análisis.
- Control de Proyecto: guía el proceso de iteración, crea una lista de control de proyecto que contiene un registro de todas las tareas que necesitan ser realizadas. Contiene, por ejemplo, las nuevas características a ser implementadas y áreas de rediseño de la solución existente. Se revisa constantemente como resultado de la fase de análisis.

El proceso unificado agrupa las iteraciones en fases: inicio, elaboración, construcción y transición. Cada una de las fases puede dividirse en una o más iteraciones, que se agrupan en función de tiempo más que de característica. Los arquitectos y analistas trabajan una iteración por delante de los desarrolladores y testers.

- Inicio: identifica el alcance del proyecto, riesgos, requisitos a un alto nivel en suficiente detalle para que se pueda estimar el trabajo.
- Elaboración: entrega una arquitectura de trabajo que mitiga los riesgos altos y cumple los requisitos no funcionales.
- Construcción: reemplaza incrementalmente la arquitectura con código listo para producción del análisis, diseño, implementación y pruebas de los requisitos funcionales.
- Transición: entrega el sistema al entorno operativo de producción

Debilidades en el modelo

- Puede ser lento debido a la necesidad de retroalimentación de los usuarios finales
- No es ideal para desarrollos que consumen muchos recursos o largos en el tiempo
- Hay un costo extra como consecuencia de tener a los usuarios finales evaluando el producto constantemente

Rapid Application Development (RAD)

Se desarrolló para responder a la necesidad de entregar sistemas muy rápido.
Tiene una lista de tareas y una estructura de desglose de trabajo diseñada para la rapidez. El método comprende el desarrollo iterativo, la construcción de prototipos y el uso de utilidades CASE (Computer Aided Software Engineering). Suele englobar los conceptos de usabilidad, utilidad y rapidez de ejecución.
Está orientada a datos con técnicas de prototipos. Requiere el uso interactivo de técnicas estructuradas y prototipos para definir los requisitos de usuario y diseñar el sistema final. Usando técnicas estructuradas, el desarrollador primero construye modelos de datos y modelos de procesos de negocio preliminares de los requisitos. Los prototipos ayudan entonces al analista y los usuarios a verificar tales requisitos y a refinar formalmente los modelos de datos y procesos.
Los enfoques RAD pueden implicar compromisos en funcionalidad y rendimiento a cambio de permitir el desarrollo más rápido y facilitando el mantenimiento de la aplicación.
Tiene una alta velocidad de desarrollo, aumenta la calidad mediante la inclusión temprana de los usuario en etapas de análisis y diseño, visibilidad temprana por el uso de prototipado, es flexible y tiene ciclos de desarrollo cortos.
Es poco escalable, las características a implementar son reducidas, es difícil evaluar el progreso porque no hay hitos, al conducir su ciclo mediante prototipos puede llevar a una sucesión de los mismos sin llegar a una aplicación de producción satisfactoria.

Rational Unified Process (RUP)

No es un proceso concreto individual, sino un marco de trabajo de proceso adaptable, con la idea de ser adaptado por las organizaciones de desarrollo y los equipos de proyecto de software que seleccionarán los elementos del proceso que sean apropiados para sus necesidades.
RUP resultó de la combinación de varias metodologías y se vio influenciado por métodos previos como el modelo en espiral. Se construyó en base al análisis de proyectos de software fallidos, especialmente los que usaban modelo en cascada. El resultado fue un sistema de mejores prácticas de software al que llamaron RUP.

Módulos

RUP se basa en un conjunto de módulos o elementos de contenido, que describen qué se va a producir, las habilidades necesarias requeridas y la explicación paso a paso describiendo cómo se consiguen los objetivos de desarrollo.
Los módulos principales, o elementos de contenido, son:

- Roles (quién): un rol define un conjunto de habilidades, competencias y responsabilidades relacionadas.
- Productos de trabajo (qué): un producto de trabajo representa algo que resulta de una tarea, incluyendo todos los documentos y modelos producidos mientras que se trabaja en el proceso.
- Tareas (cómo): una tarea describe una unidad de trabajo asignada a un rol que proporciona un resultado significante.

Fases del ciclo de vida del proyecto

El ciclo de vida del proyecto consiste en cuatro fases. Permite presentar un proyecto parecido a un proyecto en cascada aunque luego las iteraciones de desarrollo dentro de todas las fases funcionan de manera incremental. También, cada fase tiene un objetivo clave y un hito al final que denota que el objetivo se ha logrado.

- Fase de iniciación: se define el alcance del proyecto.
- Fase de elaboración: se analizan las necesidades del negocio en mayor detalle y se definen sus principios arquitectónicos.
- Fase de construcción: se crea el diseño de la aplicación y el código fuente.
- Fase de transición: se entrega el sistema a los usuarios.

RUP proporciona un prototipo al final de cada iteración.
Dentro de cada iteración, las tareas se categorizan en nueve disciplinas:
Seis disciplinas de ingeniería

- Modelado de negocio
- Requisitos
- Análisis y diseño
- Implementación
- Pruebas
- Despliegue

Tres disciplinas de soporte

- Gestión de la configuración y del cambio
- Gestión de proyectos
- Entorno

Desarrollo Ágil

Hace referencia a un grupo de metodologías de desarrollo de software que se basan en principios similares. Generalmente promueven:

- Un proceso de gestión de proyectos que fomenta la inspección y adaptación frecuente.
- Una filosofía líder que fomenta trabajo en equipo, organización propia y responsabilidad.
- Un conjunto de mejores prácticas de ingeniería que permite la entrega rápida de software de alta calidad.

- Un enfoque de negocio que alinea el desarrollo con las necesidades de los clientes y los objetivos de la compañía

Desarrollo Ágil

Surge como respuesta a los esquemas tradicionales que tenían un gran trabajo en el control del proceso. Es útil para entornos cambiantes y en donde se exige reducir drásticamente los tiempos de desarrollo pero manteniendo una alta calidad. Llenan el vacío metodológico para aquellos que los métodos tradicionales no les son útiles por las restricciones de tiempos y flexibilidad. Están específicamente diseñadas para proyectos pequeños, asegurando la calidad del producto de una manera más simple que los métodos tradicionales.
Promueven un proceso de gestión de proyectos que basado en el trabajo en equipo, la organización y responsabilidad propia, un conjunto de mejores prácticas de ingeniería que permiten la entrega rápida de software de alta calidad, y un enfoque de negocio que alinea el desarrollo con las necesidades del cliente y los objetivos de la compañía.
Trabaja con incrementos pequeños, típicamente de 1 a 4 semanas, con una planificación mínima. El ciclo de vida de cada iteración es llevado a cabo por completo por un equipo, reduciendo riesgos y permitiendo la adaptación a cambios. Al final de cada iteración se busca tener una nueva versión del producto disponible (puede tener errores mínimos), pueden requerirse varias iteraciones para liberar una nueva versión del producto.
La composición de los equipos es multidisciplinar, no sigue las jerarquías corporativas, las tareas son definidas y repartidas internamente por el equipo.
Los métodos ágiles enfatizan la comunicación cara a cara sobre los documentos escritos. Los equipos son pequeños (de 5 a 9 personas) para facilitar la comunicación y colaboración. Los esfuerzos grandes son repartidos entre varios equipos en simultáneo, pudiendo requerir coordinación de prioridades entre ellos.
Suele haber una comunicación cara a cara, diaria y formal entre los miembros del equipo, incluyendo a representantes del cliente y otras partes interesadas. En esta sesión se comenta que se hizo el día anterior, que se hace hoy y los obstáculos presentes.
El representante del cliente es designado por las personas del negocio para actuar en su nombre, debe estar disponible para responder preguntas. Al final de cada iteración, se junta con las personas involucradas en el negocio para revisar el progreso y evaluar prioridades.
Los métodos ágiles enfatizan software operativo como la medida principal del progreso.

Comparación con otros métodos

Los métodos ágiles son más cercanos a los métodos adaptativos que a los predictivos.
Los métodos adaptativos se centran en la adaptación rápida a los cambios, tienen dificultades para describir lo que pasará en el futuro, en cuanto más lejana es la fecha, menos información tienen. Por ejemplo, pueden conocer las tareas de esta semana, la planificación del mes y los objetivos del trimestre.
Los métodos predictivos se centran en la planificación del futuro en detalle, puede informar exactamente de las características y tareas planificadas para el proceso de desarrollo entero pero tienen dificultades a la hora de cambiar de dirección.
Los métodos ágiles tienen mucho en común con las técnicas de RAD (desarrollo rápido de aplicaciones).

- Modelo iterativo: construyen software liberable en cortos periodos de tiempo, los periodos de tiempo se miden en semanas y el trabajo se realiza de forma colaborativa.
- Modelo en cascada: tienen poco en común con el modelo en cascada, es el más estructurado de los métodos, el progreso se mide generalmente en términos de artefactos entregables mientras que en métodos ágiles se mide en software operativo. Mientras que hacer cambios en cascada es caro y riesgoso, ágiles pone énfasis en entregar la menor funcionalidad factible para agregar valor de negocio pronto de manera continua, por lo que los cambios son bien manejados y esperados.
- Codificación “cowboy”: ausencia de un método definido, hay frecuentes reevaluaciones de planes, se pondera la comunicación cara a cara y hay escaso uso escaso de documentos. Los equipos ágiles, sin embargo, siguen procesos definidos (con frecuencia disciplinados y rigurosos).

Idoneidad de los métodos ágiles

No hay mucho consenso en qué tipo de proyectos de software son los más adecuados para las metodologías ágiles.
Está ampliamente documentado su éxito para equipos pequeños, de menos de 10 personas.
Un proyecto ágil puede ser negativamente afectado por:

- Esfuerzos de desarrollo a gran escala (más de 20 desarrolladores)
- Esfuerzos de desarrollo geográficamente distribuido
- Culturas de compañías de ordenar y controlar
- Forzar un proceso ágil en un equipo de desarrollo

Metodologías ágiles	**Metodologías orientadas a planes**
Baja criticidad	Alta criticidad
Desarrolladores seniors	Desarrolladores juniors
Los requisitos cambian con mucha frecuencia	Los requisitos no cambian con mucha frecuencia
Pequeño número de desarrolladores	Gran número de desarrolladores
Cultura que prospera en el caos	Cultura que demanda orden

Manifiesto Ágil

Las metodologías ágiles son una familia de metodologías, no un enfoque individual de desarrollo de software, unificadas en el manifiesto ágil, considerada la canónica del desarrollo ágil.
El manifiesto ágil esboza los valores y principios para desarrollar software rápidamente y con flexibilidad a los cambios. Resume la filosofía “ágil”.
Valores:

1. La gente es el principal factor de éxito de un proyecto software por sobre procesos y herramientas. Es más importante un buen equipo que un buen entorno.
2. Producir solamente documentos cortos y fundamentales, no producir documentos a menos que sean necesarios de forma inmediata para tomar una decisión importante.
3. La colaboración de clientes sobre la negociación del contrato, se busca una interacción constante entre el cliente y el equipo de desarrollo.
4. La respuesta a cambios es más importante que seguir un plan, es determinante en el éxito de un proyecto. La planificación no debe ser estricta sino flexible y abierta.

Principios:

1. La prioridad más alta es satisfacer al cliente a través de la entrega temprana y continua de software de valor
2. Los cambios son bienvenidos, incluso tarde en el desarrollo. Los procesos ágiles aprovechan los cambios como ventaja competitiva del cliente.
3. Entregar software que funcione con frecuencia, prefiriendo escalas de tiempo cortas.
4. Las personas de negocio y los desarrolladores deben trabajar juntos diariamente durante todo el proyecto.
5. Se necesitan individuos motivados y darles entorno, soporte y confianza.
6. El método de comunicación más efectiva es la conversación cara a cara.
7. Software que funciona es la medida principal del progreso.
8. Se busca un desarrollo sostenible en el tiempo, se debe mantener un ritmo constante indefinido.
9. Excelencia técnica y los buenos diseños aumentan la agilidad.
10. Simplicidad.
11. Los equipos organizados por sí mismos generan los mejores resultados.
12. Cada una determinada cantidad de tiempo, los equipos reflexionan sobre cómo ser más efectivos y ajustan su comportamiento de acuerdo con ello.

Métodos Ágiles

Adaptación de métodos: proceso mediante el cual un equipo determina un enfoque de desarrollo de un sistema para una situación de proyecto específica.
Casi todos los métodos ágiles son adecuados para la adaptación. Los métodos ágiles permiten a los equipos de proyecto adaptar las prácticas de trabajo de acuerdo con las necesidades del proyecto individual.

Gestión de proyectos

Los métodos ágiles suelen cubrir la gestión de proyectos aunque algunos se suplementan con guías de gestión de proyectos.
Existen herramientas especialmente diseñadas para planificar, hacer seguimiento, analizar e integrar trabajo en desarrollo ágil.
Algunas de las características comunes incluyen: integración de control de versiones, seguimiento de progreso, asignación de trabajo, versiones integradas y planificación de iteraciones, foros de discusión e informe y seguimiento de defectos de software.

Extreme Programming (XP)

Pone énfasis en la adaptabilidad que en la previsibilidad, los cambios de requisitos sobre la marcha son un aspecto natural, inevitable e incluso deseable del desarrollo de proyectos. Es especialmente adecuada para proyectos con requisitos imprecisos y muy cambiantes,
Ser capaz de adaptarse a los cambios de requisitos en cualquier punto de la vida del proyecto es una aproximación mejor y más realista que intentar definir todos los requisitos al comienzo del proyecto.
Es la adopción de las mejores metodologías de desarrollo de acuerdo a lo que se pretende llevar a cabo con el proyecto y aplicarlo de manera dinámica durante el ciclo de vida del software.
Son claves las relaciones interpersonales, trabajo en equipo, aprendizaje de los desarrolladores y buen clima de trabajo. Se basa en realimentación continua entre el cliente y el equipo de desarrollo, comunicación fluida entre todos los participantes, simplicidad en las soluciones implementadas y coraje para enfrentar los cambios.
Busca disminuir la curva exponencial del coste del cambio a lo largo del proyecto, lo suficiente para que el diseño evolutivo funcione.
Elementos:

- Historias de usuario: técnica utilizada para especificar los requisitos del software. Describe brevemente las características que el sistema debe poseer, sean requisitos funcionales o no funcionales. Debe ser comprensible y delimitada para que los programadores puedan implementarlas en unas semanas.
- Roles:
    - Programador: escribe el código del sistema y pruebas unitarias
    - Cliente: escribe las historias de usuario y las pruebas funcionales. Define prioridades.
    - Encargado de pruebas (tester): ayuda al cliente a escribir las pruebas funcionales, las ejecuta y difunde sus resultados.
    - Encargado de seguimiento (tracker): proporciona realimentación al equipo, mide los desvíos de las estimaciones. Realiza el seguimiento del progreso de cada iteración.
    - Entrenador (coach): responsable del proceso global, provee guías al equipo de forma que se apliquen las prácticas XP y se siga el proceso correctamente.
    - Consultor: miembro externo con conocimiento específico en algún tema necesario para el proyecto.
    - Gestor (big boss): vínculo entre clientes y programadores, su labor esencial es de coordinación.
- Proceso :
    1. El cliente define el valor de negocio a implementar
    2. El programador estima el esfuerzo necesario para su implementación
    3. El programador construye ese valor
    4. Vuelve al paso 1
- Fases:
    1. Exploración
    2. Planificación de la entrega
    3. Iteraciones
    4. Producción

5. Mantenimiento
6. Muerte del proyecto

- Prácticas:
    1. Planificación: Hay una comunicación frecuente entre el cliente y los programadores, equipo técnico estima esfuerzos, los clientes deciden sobre el ámbito y tiempo de las entregas y de cada iteración.
    2. Entregas pequeñas: versiones del sistema rápidas que sean operativas con funcionalidad parcial del sistema que constituyen un resultado de valor para el negocio.
    3. Metáfora: el sistema se define mediante un conjunto de metáforas compartidas por el cliente y el equipo de desarrollo, son historias compartidas que describen cómo debería funcionar el sistema.
    4. Diseño simple: la solución más simple que pueda funcionar y ser implementada.
    5. Pruebas. La producción de código está dirigida por las pruebas unitarias, son establecidas por el cliente y son ejecutadas constantemente ante cada modificación del sistema.
    6. Refactorización: actividad constante de reestructuración del código para hacerlo más sólido y flexible.
    7. Programación en parejas: la producción de código debe realizarse con trabajo en parejas de programadores.
    8. Propiedad colectiva del código. Cualquier programador puede cambiar cualquier parte del código en cualquier momento.
    9. Integración continua. Cada pieza de código es integrada en el sistema una vez que esté lista, puede haber varias implementaciones en el mismo día.
    10. 40 horas por semana: máximo de 40 horas por semana, no se trabajan horas extras en dos semanas seguidas (síntoma de un problema). El trabajo extra desmotiva al equipo.
    11. Cliente in-situ. El cliente tiene que estar presente y disponible todo el tiempo para el equipo para determinar que aporta mayor valor al negocio y responder de manera inmediata cualquier duda de los programadores. La comunicación oral es más efectiva que la escrita.
    12. Estándares de programación: la comunicación de los programadores es a través del código, se deben seguir estándares de programación para mantener el código legible.
- Principios:
    1. Simplicidad: para agilizar el desarrollo y facilitar el mantenimiento. La refactorización del código mantiene el código simple a medida que crece. El código debe comentarse en su justa medida, buscando que el código esté autodocumentado, es importante elegir buenos nombres de variables, métodos y clases.

2. Comunicación: el código comunica mejor mientras más simple sea, el código autodocumentado es más fiable que los comentarios ya que éstos últimos pronto quedan desfasados, debe comentarse sólo aquello que no va a variar.
   Las pruebas unitarias son otra manera de comunicación ya que muestran ejemplos concretos de cómo consumir el código.
   Los programadores se comunican constantemente gracias a la programación por parejas.
   La comunicación con el cliente es fluida porque el cliente forma parte del equipo de desarrollo, el cliente decide prioridades y siempre debe estar disponible para solucionar dudas.
3. Retroalimentación: al estar integrado al equipo, el cliente brinda su opinión sobre el proyecto en tiempo real. Los ciclos cortos minimizan el tener que rehacer partes que no cumplen con los requisitos y ayuda a los programadores a centrarse en los que es más importante.
   También se recibe feedback a partir de herramientas de desarrollo que informan sobre el estado de salud del código.
4. Valentía: se requiere valentía para aceptar la programación por parejas, simplicidad, no se debe pausar las entregas para desarrollar un marco de trabajo, se deben construir mediante la refactorización del código en sucesivas aproximaciones.

- Actividades:
  1. Codificar: único producto realmente importante del proceso de desarrollo de sistemas es el código. También puede usarse para entender la solución más apropiada, creando varias soluciones alternativas y viendo cual es más eficiente. Puede ayudar también a comunicar pensamientos sobre programación, explicar una solución usando el código para demostrar lo que quería decir y recibir feedback.
  2. Probar: no es una necesidad primaria percibida por el cliente aunque uno no puede estar seguro de que una función funciona si no la prueba. Con pruebas unitarias, se comprueba que el código coincide con lo que quería significar el programador. Con pruebas de aceptación, se comprueba que lo que significó el programador coincide con los requisitos dados por el cliente.
  3. Escuchar: deben escuchar las necesidades del negocio manteniendo conversaciones con los clientes. Deben proveer feedback al cliente para mejorar el propio entendimiento del cliente sobre el problema.
  4. Diseñar: Cuando el sistema crece, se vuelve muy complejo y las dependencias dentro del sistema dejan de estar claras. Buenos diseños evitarán pérdidas de dependencias dentro de un sistema; esto significa que cambiar una parte del sistema no tendrá por qué afectar a otras.

SCRUM

Es un proceso ágil que se puede usar para gestionar y controlar desarrollos complejos de software y productos usando prácticas iterativas e incrementales. Se puede usar para la gestión de proyectos de desarrollo, equipos de mantenimiento o como un enfoque de gestión de programas.
Incluye un conjunto de prácticas y roles predefinidos.
Roles:

- Scrum Master: mantiene los procesos y trabaja junto con el jefe de proyecto.
- Product Owner: representa a las personas implicadas en el negocio.
- Team: desarrolladores.

Las iteraciones se llaman sprints y duran de 2 a 4 semanas, cada una da un incremento de software operativo.
El backlog es un conjunto priorizado de requisitos de trabajo (ítems) de alto nivel.
Durante la reunión de planificación de la iteración, el Product Owner informa al equipo cuáles ítems del backlog desea incluir en el sprint, el equipo determina cuánto de eso puede comprometerse. Durante el sprint, los requisitos están congelados, no pueden ser cambiados. Al finalizar el sprint, el equipo hace una demostración del software implementado.
Entiende que los clientes pueden cambiar de opinión y no se los puede predecir. Por esto, usa un enfoque empírico en el que acepta que no se puede definir el problema completamente, centrándose en maximizar las habilidades del equipo para entregar rápidamente y responder a los requisitos emergentes.
Reuniones:

- Scrum diario: reunión de estado del proyecto.
- Reunión de planificación de iteración (sprint): al principio del ciclo de la iteración.
- Reunión de revisión de iteración: al final del ciclo de la iteración.
- Iteración retrospectiva: al final del ciclo de la iteración.

Prácticas:

- Los clientes se convierten en parte del equipo de desarrollo.
- Entregables intermedios funcional, permite al cliente trabajar con el software desde antes y permite cambiar los requerimientos.
- Monitorización, gestión de riesgos y mitigación de riesgos.
- Transparencia en la planificación y desarrollo, que sean visibles quien tiene cada responsabilidad.
- Frecuentes reuniones con personas del negocio para monitorizar el progreso.
- Mecanismo de advertencias avanzado.
- Reconocer y exponer los problemas e imprevistos.

Dynamic Systems Development Method (DSDM)

Es una metodología de desarrollo de software originalmente basada en la metodología RAD, es un enfoque iterativo e incremental que enfatiza la participación continua del usuario. Su objetivo es entregar sistemas software en tiempo y presupuesto ajustándose a los cambios de requisitos durante el proceso de desarrollo.
Se centra en proyectos con planificaciones y presupuestos estrictos.
Fases:

- Fase pre-proyecto
- Fase de ciclo de vida del proyecto
    - Estudio de viabilidad
    - Estudio de negocio
    - Iteración de modelo funcional
    - Iteración de diseño
    - Construcción e implementación
- Fase post-proyecto

Se pueden integrar prácticas de otras metodologías como complemento tales como RUP, XP y PRINCE. Otro método que tiene alguna similitud en el proceso y concepto de DSDM es Scrum.
En DSDM hay nueve procesos subyacentes que consisten en cuatro fundamentos y cinco puntos de partida:

- La participación del usuario es la clave principal para un proyecto eficiente y efectivo.
- El equipo debe tener poder para tomar decisiones que son importantes para el progreso del proyecto.
- Pone foco en la entrega frecuente de productos, la entrega se puede probar y revisar y el registro de pruebas y el documento de revisión sirven como entrada para la siguiente entrega.
- El criterio de aceptación se define en base a cuanto se satisfacen las necesidades del negocio actuales. Hay que concentrarse en las funcionalidades críticas.
- El desarrollo es iterativo e incremental y guiado por la retroalimentación del usuario.
- Todos los cambios durante el desarrollo son reversibles.
- Se debería hacer una línea base del alcance y los requisitos a alto nivel antes de que el proyecto empiece.
- Las pruebas se llevan a cabo a lo largo de todo el ciclo de vida del proyecto.
- La comunicación y la cooperación son necesarias para ser eficientes y efectivos.

Supuestos:

- Armar las iteraciones según el orden de criticidad, construir primero lo que brinde más beneficios. Esto mitiga el riesgo de salirse de plazos y presupuesto.
- Las entregas se hacen en tiempo, presupuesto y con calidad.
- Cada paso de desarrollo debe completarse antes del siguiente paso para que la siguiente iteración pueda comenzar sin un retraso.
- Se incorporan técnicas de gestión de proyectos y desarrollo.
- Se puede aplicar en proyectos nuevos o para ampliar sistemas actuales.
- La evaluación de riesgos debería centrarse en la funcionalidad del negocio que se va a entregar, no en el proceso de desarrollo o sus artefactos.
- Se premia más la entrega de productos que la finalización de tareas.
- La estimación debería basarse en la funcionalidad del negocio.

Factores críticos de éxito:

- Aceptación de la metodología por parte de gerencia y otros empleados.
- Participación de usuario final.
- El equipo debe estar formado por profesionales habilidosos, tiene que poseer el poder y posibilidad de tomar decisiones dentro del proyecto sin recurrir a la gerencia.

Comparación:

- XP tiene también un enfoque iterativo para el desarrollo con una participación amplia del usuario.
- RUP es también una forma dinámica de desarrollo de sistemas de información con enfoque iterativo .

Características únicas:

- Proporciona un marco de trabajo independiente.
- Permite a los usuarios completar los pasos específicos del proceso con sus propias técnicas.
- Las variables en el desarrollo no son tiempo/recursos, sino los requisitos.
- Fuerte foco en la comunicación y la participación de todas las personas involucradas en el sistema.

Crystal Clear

Es una metodología ágil, está pensada para aplicarse a equipos pequeños de 6 a 8 desarrolladores ubicados en el mismo sitio trabajando en sistemas que no son críticos. Se centra en eficiencia y habitabilidad.
Se centra en las personas, no en los procesos o artefactos.
Propiedades:

- Entrega frecuente de código usable a los usuarios (requerida)
- Mejora reflexiva (requerida)
- Comunicación osmótica preferiblemente estando en la misma ubicación (requerida)
- Seguridad personal
- Fácil acceso a los usuarios expertos
- Pruebas automatizadas, gestión de la configuración e integración frecuente

Agile Unified Process (AUP)

El Agile Unified Process (AUP) es una versión simplificada del Rational Unified Process (RUP) que se enfoca en el desarrollo de software de aplicaciones de negocios usando técnicas y conceptos ágiles.
AUP aplica técnicas ágiles como el desarrollo orientado a pruebas, el modelado ágil, la gestión de cambios ágil y la refactorización de bases de datos para mejorar la productividad. AUP se basa en filosofías de simplicidad, agilidad, independencia de herramientas y enfoque en actividades de alto valor, y se puede adaptar para cumplir con necesidades específicas.
Fases:

- Inicio
- Elaboración
- Construcción
- Transición

Disciplinas:

- Modelado
- Implementación
- Pruebas
- Despliegue
- Gestión de Configuración
- Gestión de Proyecto
- Entorno

Filosofías:

- AUP se basa en la idea de que los empleados saben lo que están haciendo, pero proporciona orientación a alto nivel y enlaces a detalles.
- Simplicidad. Todo está descrito de forma concisa.
- Agilidad, se ajusta a los valores y principios de desarrollo de software ágil
- El foco está en las actividades de alto valor, lo que significa que se priorizan las actividades que realmente hacen la diferencia.
- Se puede usar cualquier herramienta, se recomienda usar las más simples.
- Se puede adaptar a las necesidades propias

Prácticas ágiles

Test Driven Development (TDD)

Es un método de diseño de software, no sólo un método de pruebas.
Usa iteraciones de desarrollo cortas basadas en casos de prueba escritos previamente que definen las mejoras deseadas o nuevas funcionalidades. Las pruebas unitarias automatizadas se escriben antes de que se escriba el código para que éste cumpla con las pruebas.
Una ventaja de esta forma de programación es evitar escribir código innecesario. Se intenta escribir el mínimo código posible.

- Ayuda a asegurar que la aplicación se escribe para poder ser probada
- Asegura que se escriben pruebas para cada característica

Ciclo de desarrollo:

1. Añadir una prueba: cada nueva funcionalidad empieza con la escritura de una prueba, que deberá fallar hasta que se haya implementado la característica. Puede ser también un cambio en una funcionalidad o prueba ya existente.
2. Ejecutar las pruebas y comprobar fallo.
3. Escribir implementación: escribir un código que pase las pruebas, no será perfecto y puede funcionar de forma poco elegante.
4. Ejecutar las pruebas y comprobar su éxito.
5. Refactorizar el código para mejorar su diseño: se limpia el código y se ejecutan de nuevo las pruebas.

Ventajas:

1. Los casos de prueba sirven como documentación del sistema, definen de manera formal los requisitos.
2. Al escribir una prueba unitaria, se piensa en la forma correcta de utilizar un módulo que aún no existe.
3. Hacer modificaciones es más seguro.

Inconvenientes:

1. No puede manejar pruebas funcionales.
2. Necesita soporte de gestión sino puede percibirse como pérdida de tiempo escribiendo pruebas.
3. Las pruebas se han visto históricamente como una posición más baja que los desarrolladores o arquitectos.

Integración continua

Conjunto de prácticas de ingeniería del software que aumentan la velocidad de entrega de software disminuyendo los tiempos de integración (compilación y ejecución de pruebas).
Permite comprobar continuamente que todos los cambios a subir no producen problemas de integración con el código existente y del resto del equipo.
Los entornos de integración contínua construyen el software leyendo el código desde un repositorio y los despliegan en un entorno de integración sobre el que se realizan pruebas, se integra el desarrollo de forma incremental y continua.
Se debe tener precaución con el desfase de las versiones locales de los desarrolladores con la versión de código actual subida y la versión del repositorio, puede requerir integraciones.
Prácticas recomendadas:

- Mantener un repositorio de código: usar un sistema de control de revisiones para el código fuente del proyecto.
- Automatizar la construcción: sistema debería ser construible usando un único comando, incluyendo la integración, documentación, páginas web, estadísticas y distribución.
- Hacer las pruebas propias de la construcción: se deben pasar todas las pruebas para confirmar que se comporta como el desarrollador esperaba que lo hiciera.
- Mantener la construcción rápida
- Probar en un clon del entorno de producción: para evitar fallos en los sistemas probados cuando se despliegan en el entorno de producción.
- Hacer fácil conseguir los últimos entregables: mantener documentación de las construcciones a mano para reducir el esfuerzo de probar de nuevo después de envíar un arreglo.
- Todo el mundo puede ver los resultados de la última construcción.
- Despliegue automático.

Ventajas:

- Reducción del tiempo de integración.
- Ante un error, los desarrolladores pueden revertir el código base a un estado libre de defectos.
- Los problemas de integración se detectan y arreglan continuamente, sin tener que esperar a la fecha de liberación.
- Avisos tempranos de código no operativo o incompatible.
- Avisos tempranos de cambios conflictivos.
- Pruebas unitarias inmediatas de todos los cambios.
- Disponibilidad constante de una construcción actual para propósitos de pruebas, demo o liberación.
- Incentiva trabajar de forma incremental con entregas funcionales cortas porque el impacto de subir código incompleto es inmediato.

Inconvenientes:

- Aumenta el mantenimiento del sistema.
- Necesita un servidor para construir el software.
- La construcción automática desincentiva los commits y, en consecuencia, existe un riesgo de pérdida de trabajo porque no hay una copia de seguridad en un servidor.

Pair Programming

Dos programadores trabajan juntos en el mismo ordenador, uno escribe el código (driver) y el otro revisa línea a línea (observer). Los dos programadores cambian de roles con frecuencia, usualmente cada 30 minutos.

El observer dirige, genera ideas de majetas y detecta problemas a futuro mientras que el driver puede centrar toda su tarea actual.

Ventajas:

- Calidad de diseño: dos programadores consideran más alternativas de diseño que los programadores que trabajan solos y llegan a diseños más simples.
- Coste reducido del desarrollo: debido a la reducción en la tasa de defectos.
- Aprendizaje y formación: el conocimiento pasa fácilmente entre programadores.
- Superar problemas difíciles: trabajando de pares los problemas se vuelven más sencillos.

- Moral mejorada: mayor alegría y confianza.
- Disminución del riesgo de gestión: como el conocimiento está distribuido, es menos costoso que un programador abandone el equipo.
- Incremento de la disciplina y mejor gestión del tiempo.
- Flujo elástico: cuando hay una interrupción a unos de los programadores, el otro puede seguir trabajando.
- Menos interrupciones: la gente es más reacia a interrumpir a una pareja que a una persona que trabaja sola.
- Menos puestos de trabajo requeridos.

Inconvenientes:

- Preferencia de trabajo: muchos ingenieros prefieren trabajar solos.
- Intimidación: desarrolladores con menos experiencia pueden sentirse intimidados por compañeros más experimentados.
- Coste de tutoría: de reduce la producción de un desarrollador experimentado por enseñarle a uno con menos experiencia.
- Egos y conflictos potenciales.
- Hábitos personales molestos.
- Coste: hay dos personas a las que pagar.

Críticas al desarrollo ágil

- Con frecuencia se usa como medio de sacar dinero al cliente a través de la falta de definición de un entregable.
- Falta de estructura y documentación necesaria.
- Sólo funciona con desarrolladores experimentados.
- Incorpora diseño de software insuficiente.
- Requiere encuentros a intervalos frecuentes con un enorme coste para los clientes.
- Requiere demasiado cambio cultural para adaptarlo.
- Puede conducir a negociaciones contractuales más difíciles.
- Puede ser muy ineficiente, si los requisitos de un área de código cambian durante varias iteraciones.
- Es difícil armar un presupuesto al principio del proyecto porque no se conoce el alcance completo.
- Puede aumentar el riesgo de cambios del alcance, debido a la falta de documentación de requisitos detallada.
- El desarrollo ágil está orientado a características, los atributos de calidad no funcionales son difícil de plasmar como historias de usuario.

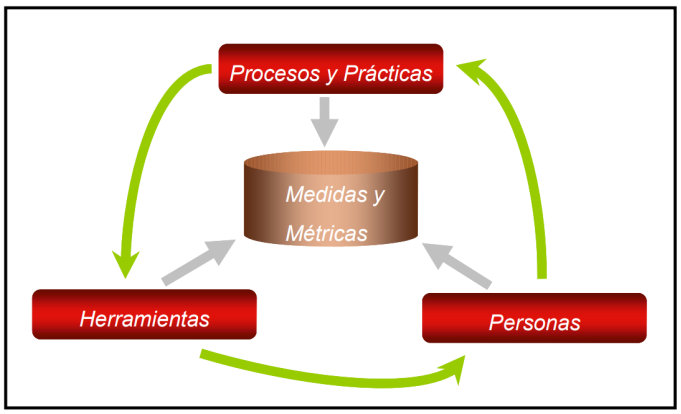
Unidad 1: Calidad

Definiciones

- Calidad es la idoneidad de uso. Es decir, las características del producto que satisfacen las necesidades del cliente y por tanto producen satisfacción de producto. La calidad es la inexistencia de deficiencias.
- “La calidad se define desde el punto de vista del cliente, como cualquier cosa que aumenta su satisfacción”.
- “La calidad es la totalidad de características de un producto o servicio que tienen la capacidad de satisfacer necesidades explícitas o implícitas”.
- “Nivel al que una serie de características inherentes satisfacen los requisitos”.
- “Conjunto de propiedades y características de un producto o servicio que le confieren su aptitud para satisfacer unas necesidades explícitas o implícitas”.
- “La capacidad de un conjunto de características inherentes de un producto, o componente del producto, o proceso, de satisfacer por completo los requisitos del cliente”.

Elementos que influyen en la calidad

- Procesos y buenas prácticas: La calidad aumenta al aplicar una serie de procesos o metodologías y buenas prácticas. Controlan el proceso para poderlo analizar y mejorar.
- Herramientas: Proporcionan apoyo a la gestión de la calidad.
- Personas: Son elementos clave en la calidad como creadores y ejecutores.
- Medidas y métricas: Son los datos los que permiten evaluar el estado actual y ejecutar acciones para mejorar.



Metodología

Una metodología o guía de buenas prácticas proporciona:

- Un punto de partida.
- Los beneficios de la experiencia de otras empresas.
- Un lenguaje y una visión común.

- Las técnicas para crear un modelo.

Modelo de calidad

Al crear un modelo, se crea una visión simplificada de la situación real, la cual es más sencilla de controlar y mejorar. Siguiendo un modelo, se puede predecir mejor el comportamiento y rendimiento de una empresa. Trabajar con un modelo a partir de una metodología probada ayuda a aumentar la calidad.

Ventajas

Al tener más control sobre el proyecto y los procesos implicados:

- Disminuye el número de defectos totales.
- Disminuyen los tiempos de entrega.
- Disminuyen los costes.
- Aumenta la satisfacción del cliente.
- Aumentan los beneficios.

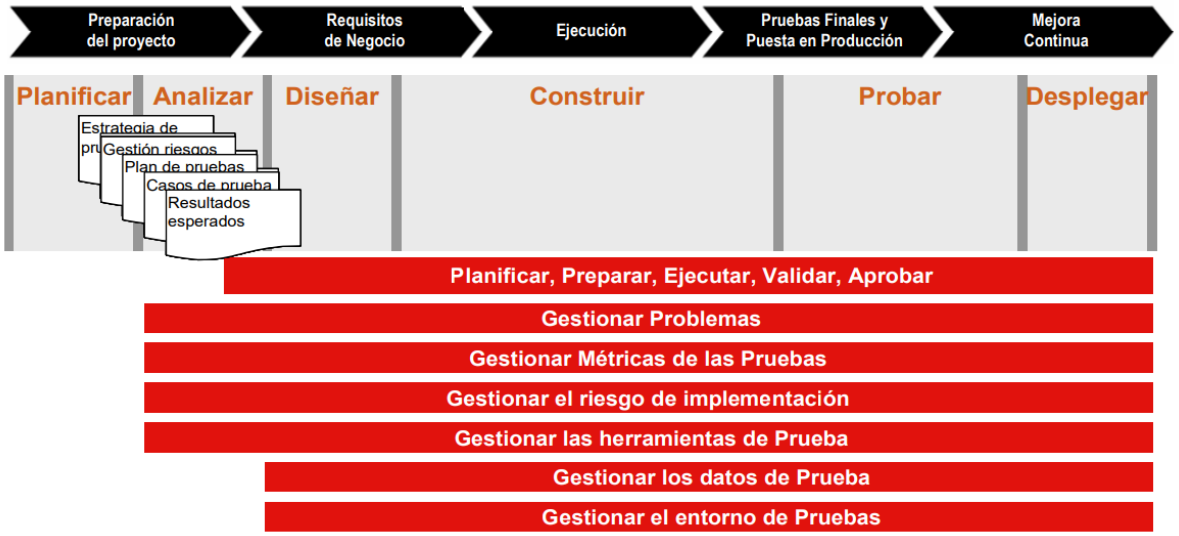
Niveles de gestión de calidad

La calidad de un producto software puede gestionarse a distintos niveles:

- Producto: Cuando nos centramos en el proceso de desarrollo de software y hacemos una serie de pruebas en paralelo con cada etapa, para detectar y corregir los posibles defectos que puedan surgir.
- Proyecto: Cuando nos centramos en controlar todas las fases y áreas de gestión de proyecto, implantando metodologías y mejores prácticas que aseguren la correcta gestión de las mismas.
- Proceso: Cuando nos centramos en gestionar todas las áreas de proceso de una organización, mediante la implantación de una metodología. Así se consigue tener mayor información de los procesos de modo que puedan controlarse y mejorarse, y produzcan así un aumento de la calidad de los productos y servicios relacionados con ellos.

Actividades de pruebas

Las actividades de prueba deben comenzar pronto y continuar a lo largo del ciclo de vida del desarrollo.



Conceptos

Aseguramiento de la calidad y Control de la calidad

QA : Aseguramiento de la calidad	**QC : Control de la calidad**
Verificación	**Validación**
Preventivo y proactivo.	Reactivo.
Orientado a proceso.	Orientado a producto o servicio.
Responsabilidad a nivel de organización.	Verifica si los atributos especificados están presentes en el producto o no.
Evalúa si QC funciona o no.	Responsabilidad a nivel del equipo de control.
Identifica las debilidades de ciertos procesos y las mejoras.	
Ejemplos: Auditorías de proceso, Definiciones de procesos, Selección de herramientas, Formación	Ejemplos: Revisiones, Inspecciones, Ejecución de pruebas
Cuándo: Antes y durante el desarrollo de software	Cuándo: Cuando la mayor parte de desarrollo de software se ha completado
Método: Revisiones, análisis e inspecciones.	Método: Pruebas de caja negra
¿Se está construyendo el producto de manera correcta según el proceso definido?	¿Se está construyendo el producto correcto/adecuado que satisfaga las necesidades del cliente?

Orientado a detectar y corregir errores en cada fase del desarrollo	Orientado a que el producto final no tenga errores, tampoco de concepto.

Tareas en aseguramiento de calidad

- Añadir valor: Definir procesos eficientes que creen productos de calidad y reduzcan
- la repetición de trabajo.
- Trabajo Inteligente: Entender y evaluar objetivamente el conjunto de buenas prácticas
- seleccionado para el proyecto.
- Prevención: Entender y evaluar objetivamente el conjunto de buenas prácticas
- seleccionado para el proyecto.
- Mejorar: Verificar y asegurar la calidad del producto.
- Reutilizar la experiencia: Contribuir al repositorio de información.

Entorno de gestión de calidad

- Procesos de Gestión de la Calidad
    - Objetivos de Calidad
    - Gestión de Requisitos
    - Estrategia de Pruebas
    - Gestión de Activos de Pruebas
    - Desarrollo de Pruebas
    - Ejecución de las Pruebas
    - Gestión de defectos
    - Gestión de entregas
- Informes y Análisis
- Soporte y gestión del entorno
    - Gestión del Entorno
    - Gestión de Herramientas
    - Servicios Comunes

Métricas

Si no se puede medir, no se puede controlar. Si no se puede controlar, no se puede mejorar.

- Medidas: Dan valores de ciertos atributos
- Métricas: Dan información significativa acerca del producto o proceso, y suelen relacionar varias medidas entre sí.

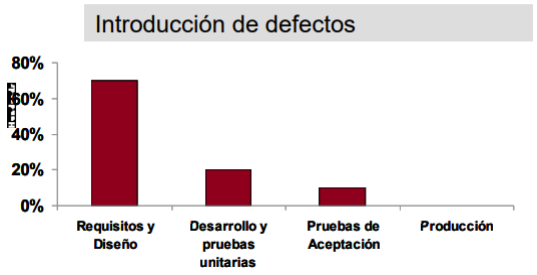
Las métricas:

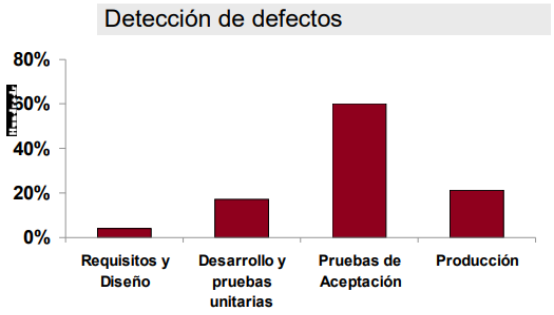
- Se relacionan con los objetivos y metas a conseguir.
- Son concretas y están definidas claramente.
- Son sencillas de entender e implementar.
- Ayudan a entender el pasado, controlar el presente y predecir el futuro.

Las métricas han de analizarse y compararse con los objetivos propuestos. Si existe desviación, se deben tomar acciones preventivas y correctoras.
Key Performance Indicators (KPIs): son medidas que reflejan los factores críticos de éxito de una organización. Son los valores que dan una visión general del estado de la organización. Representan una medida del progreso hacia los objetivos propuestos.

Defectos

La mayoría de los defectos se introducen durante las fases de requisitos y diseño, sin embargo, la mayoría de ellos se detectan durante las pruebas de aceptación y sobre producción.





Unidad 2: Proceso de desarrollo de Software

Introducción

CMMI (Integración de Modelos de Madurez de Capacidades - Capability Maturity Model Integration):

- Es un modelo de madurez de mejora de los procesos para el desarrollo de productos y de servicios.
- Consiste en las mejores prácticas que tratan las actividades de desarrollo y de mantenimiento que cubren el ciclo de vida del producto, desde la concepción a la entrega y el mantenimiento.
- Creado por SEI (Software Engineering Institute)

Las tres dimensiones críticas sobre las cuales típicamente se concentran las organizaciones para mejorar su actividad (SEI):

- Personas: personas con habilidades, formación y motivación
- Métodos y procedimientos: métodos y procedimientos que definen las relaciones entre tareas
- Herramientas y equipamiento

Los procesos sustentan el conjunto.

Calidad y Proceso

Procesos

- Permiten alinear el modo de operar de su organización, evolucionar e incorporar los conocimientos de cómo hacer mejor las cosas.
- Permiten también explotar mejor sus recursos y comprender las tendencias de su actividad.
- Un enfoque centrado en el proceso proporciona la infraestructura necesaria para hacer frente a este mundo en constante evolución, maximizar la productividad de las personas y utilizar la tecnología con el fin de ser más competitivos.

Calidad

La calidad de un sistema o de un producto está muy influenciada por la calidad del proceso empleado para desarrollarlo y para mantenerlo.

CMMI

Es un modelo de mejora de los procesos, surgió para evaluar proveedores de software y luego se extendió y desarrolló a varias áreas distintas.
Es un marco de trabajo que contiene la descripción de elementos claves para lograr la mejora de procesos. Trasforma procesos ad hoc y no maduros en procesos disciplinados y maduros, con calidad y eficacia mejorada utilizando el concepto de buenas prácticas (no mejores prácticas, este concepto no es general, depende de la empresa y cambia en el tiempo.
Las empresas de software pueden certificar internamente CMMi para desarrollo y, para el

momento de vender los servicios, CMMi para servicios.
Estos modelos son una guía para saber qué hacer pero no cómo hacer y es por esto que CMMi está alineado con otras normas y metodologías.
Tiene dos representaciones diferentes, cada una es una forma de certificación: contínua y por etapas.

Representación Contínua

Orientada a la capacidad para mejorar procesos.
Se desarrolló para empresas que ya han certificado otro estándar y no necesitan pasar por el proceso escalonado.
Utiliza niveles de capacidad para caracterizar la mejora concerniente a un área de proceso individual. Cada nivel de CMMi describe un camino evolutivo recomendado para una organización que quiera mejorar los procesos y pueden también ser el resultado de la actividad de calificación de las evaluaciones.
Existen metas específicas para los procesos seleccionados y metas genéricas para las áreas donde pertenecen estos procesos. Es importante conocer si un proceso se ha realizado o está incompleto. Por lo tanto, al punto de partida de la representación continua se le da el nombre “incompleto”.

0. Incompleto: El proceso o bien no se ejecuta, o se ejecuta parcialmente. No se satisfacen ninguna de las metas específicas del área de proceso. No existen metas genéricas ya que el proceso no se institucionaliza.
1. Realizado: Es un proceso que satisface las metas específicas del área de proceso. Soporta y permite el trabajo necesario para producir los productos del trabajo.
2. Gestionado: Existen políticas, infraestructura básica; personal con habilidades; recursos adecuados; involucra a las partes interesadas relevantes; se monitoriza, controla y revisa; se evalúa la adherencia a su descripción de proceso. Asegura que las prácticas se mantienen durante tiempos de estrés.
3. Definido: Se adapta a partir del conjunto de procesos estándar de la organización. Se establece claramente el propósito, entradas, criterios de entrada, actividades, roles, medidas, etapas de verificación, salidas y criterios de salida. Los procesos se gestionan de forma más proactiva

Representación por Etapas

Orientada a la madurez global de la organización. utiliza conjuntos predefinidos de áreas de proceso para definir un camino de mejora para una organización.
Es la opción que normalmente toman las empresas cuando certifican sus procesos con algún estándar compatible con CMMi.
Se caracteriza por niveles de madurez, cada nivel de madurez proporciona un conjunto de áreas de proceso que caracterizan diferentes comportamientos organizativos.
Se interesa por la madurez global de la organización, no es interés primario si los procesos individuales son realizados o están incompletos. Su punto de partida se nombre “inicial”.

1. Inicial: Los procesos son ad-hoc, caóticos y no tienen soporte adecuado. La organización depende de la competencia individual de las personas y no del uso de procesos probados.
    a. Los proyectos no se pueden planear
    b. Los requerimientos no se pueden tener bajo control

    c. No se controla el versionado de los productos
    d. La calidad es percibida como una burocracia
    e. Los procesos son una asunto personal
    f. No se puede verificar ni validar el producto
    g. Existen condiciones de estrés y frustración permanentes.
2. Gestionado: existe un orden a nivel de proyecto, se realizan y gestionan de acuerdo a sus planes y políticas, el personal tiene la habilidad necesaria para llevar adelante los, se monitorean, controlan y revisan y se evalúan en cuanto a su adherencia a sus descripciones de proceso. Las prácticas existentes se mantienen durante tiempos de estrés. El estado de los productos de trabajo y la entrega de los servicios son visibles a la dirección. Se establecen compromisos de entrega y cumplimiento y se revisan si es necesario. Los productos de trabajo y servicios se controlan de forma apropiada, satisfacen sus descripciones de proceso especificadas, estándares y procedimientos.
    a. Los procesos están caracterizados sólo en proyectos
    b. La administración es reactiva
3. Definido: existe un orden a nivel de proceso, son bien caracterizados y comprendidos, se usan estándares, procedimientos, herramientas y métodos consistentes en toda la organización. Cada proceso tiene un propósito, entradas, criterios de entrada, actividades, roles, medidas, etapas de verificación, salidas y criterios de salida. Los procesos están definidos para la organización. La administración es proactiva, previniendo los problemas antes de que ocurran
    a. Los procesos son sólo predecibles cualitativamente.
4. Gestionado cuantitativamente: se establecen objetivos cuantitativos para la calidad y el proceso, se comprenden en términos estadísticos, se utilizan como criterios en la gestión de los procesos. Se basan en las necesidades del cliente, usuarios finales, organización e implementadores del proceso.
   Todas las mediciones se incorporan en el repositorio de medición de la organización para dar soporte a la toma de decisiones basada en hechos. Se analizan las estadísticas y se corrigen las fuentes de las causas especiales para prevenir sus futuras ocurrencias.
   El rendimiento del proceso es predecible (cualitativa y cuantitativamente) y se controla utilizando técnicas estadísticas y otras técnicas cuantitativas.
5. En optimización: se mejoran los procesos continuamente a partir de una comprensión cuantitativa de los mismos con mejoras incrementales e innovadoras de proceso y tecnológicas.
   Los objetivos cuantitativos de mejora de procesos se alinean a objetivos del negocio y se utilizan como criterios para gestionar la mejora de procesos. Los efectos de las mejoras de procesos desplegadas se miden y evalúan frente a los objetivos cuantitativos de mejora de procesos.
    a. Se identifican, evalúan, e implementan mejoras incrementales y de innovación que aumentan las capacidades de los procesos.

Área de Proceso

Es un conjunto de prácticas relacionadas en un área que, cuando se ejecutan colectivamente, satisfacen un conjunto de metas específicas y genéricas. Cada área de proceso define varios componentes, los mismos pueden ser de 3 tipos:

- Requerido: necesario para satisfacer un área de proceso, debe ser visiblemente implementado en los procesos de una organización, son las metas específicas y genéricas.
- Esperado: describen lo que implementará una organización típicamente para lograr un componente requerido, guían a las personas que implementan mejoras.
- Informativo: proveen detalles que ayudan a pensar en la manera en cómo lograr los componentes requeridos y esperados.

Los componentes son:

- Declaración de propósito: es un componente informativo.
- Notas introductorias: conceptos globales del área de proceso, es un componente informativo.
- Áreas de procesos relacionadas: lista de áreas de proceso relacionadas.
- Metas específicas: características necesarias para satisfacer un área de proceso.
- Metas generales: características necesarias comunes a varias áreas de proceso. Describen características que deben estar presentes para poder institucionalizar el proceso que implementa un área de proceso.
- Prácticas específicas: describe actividades esperadas para lograr una meta específica asociada. Es un componente de tipo esperado.
- Productos de Trabajo: listan ejemplos de salidas de una práctica específica. Es un componente de tipo informativo.
- Subprácticas: proveen una descripción detallada que puede llegar a ser útil para entender y llevar a cabo una práctica específica. Son componentes informativos.
- Prácticas genéricas: la misma práctica aparece en múltiples áreas de proceso. Es una descripción de una actividad esperada para satisfacer un objetivo general. Es un componente esperado.

Constelaciones

Tiene 3 constelaciones que cubren las necesidades de empresas dedicadas al desarrollo y mantenimiento de productos y servicios, desarrollo y operación de servicios y adquisición de productos y servicios:

- CMMi para Desarrollo: sirve como guía para medir, monitorear y administrar los procesos usados para desarrollar y mantener productos y servicios.
- CMMi para Servicios: sirve como guía para mejorar los procesos de gestión, operación y entrega de servicios ya sea internamente en una organización o a clientes externos. El foco está del lado del proveedor de servicios.
- CMMi para Adquisiciones: sirve como guía para mejorar el proceso de adquisición de productos y servicios, es decir, para interactuar eficaz y eficientemente con proveedores. El foco está del lado del cliente que adquiere los servicios o productos.

En las tres constelaciones las áreas de procesos se dividen en categorías.
Existen 16 áreas de procesos que están compartidas por las tres constelaciones y conforman el CORE del Model Foundation, las mismas se encuentran en las categorías Gestión de procesos, Gestión de proyectos y Soporte. Esto permite alinear las prácticas tanto del cliente que adquiere los servicios o el desarrollo y de los proveedores.
Cada una de las constelaciones tienen categorías y áreas de proceso particulares.
Cada área de proceso tiene un Nivel de Madurez asociado a partir del cual se espera que se implemente.

Áreas

Las áreas de Gestión de procesos contienen las actividades transversales a los proyectos relacionadas con la definición, planificación, despliegue, implementación, monitorización, control, evaluación, medición y mejora de los procesos.
Áreas de Gestión de procesos básicas
Enfoque en procesos de la organización (OPF)
Definición de procesos de la organización + IPPD (OPD)
Formación organizativa (OT)
Áreas de Gestión de procesos avanzadas
Rendimiento de procesos de la organización (OPP)
Innovación y despliegue en la organización (OID)

Gestión de configuración (CM)

El área CM se ocupa de mantener la integridad de los productos de trabajo estableciendo líneas base y siguiendo los cambios de los distintos elementos de configuración. Contar con una gestión de configuración permite que, por ejemplo, se conozcan las características de un producto en base a su número de versión.

Medición y Análisis (MA)

El área MA se encarga de definir qué información necesita la empresa acerca de su funcionamiento, obtener los datos y procesarlos para obtener la información necesaria. Un ejemplo típico de medición es la productividad de la empresa, la cual se obtiene recopilando los tiempos insumidos en los diferentes procesos de la organización.

Definición de procesos de la organización (OPD)

El área OPD está a cargo de definir los estándares de la organización, ya sean procedimientos, plantillas, guías de uso, estándares de trabajo, entre otros. Todos estos documentos se almacenan en una biblioteca de activos de manera de estar centralizado y accesible por toda la organización.
Aquí también se toman medidas pero con el objetivo específico de medir el desempeño de los procesos de la organización.
Una subpráctica muy común en las distintas prácticas específicas de OPD es realizar revisiones entre pares. Es importante tener esto en cuenta ya que la revisión entre pares utilizada en el área VER aplica a los productos de un proyecto específico, mientras que en OPD aplica a toda la organización. Estos enunciados suelen aclarar que el procedimiento a analizar aún no ha sido publicado.

Enfoque de procesos de la organización (OPF)

El área OPF es la encargada de llevar a cabo las mejoras de los procesos. Para detectar las mejoras que son necesarias, se puede realizar una revisión activa, en la que el área recorre los proyectos de la organización y observa cómo se desempeñan, si siguen estándares de mercado, etc. Otra forma de identificar posibles mejoras es la revisión pasiva, que consiste en detectar oportunidades de mejora a través de las lecciones aprendidas generadas en los proyectos (mediante el área PMC) y de los informes de no conformidad producidos por el área PPQA.

Las áreas OPD, OPF y PPQA se encuentran muy relacionadas. Por ejemplo, dada una determinada plantilla de casos de prueba creada por el área OPD, es PPQ Aquien se ocupa de controlar si dicha plantilla está siendo completada de manera correcta por los diferentes proyectos de la organización. Si existen varios informes de PPQA en los que se indican que los proyectos no completan correctamente la plantilla, entonces el área OPF puede sugerir confeccionar una guía sobre el llenado de la plantilla.
Es importante saber diferenciar las metas específicas 2 y 3de OPF. La SG2 está enfocada a planificar y probar la implementación de las mejoras, por lo que se trabaja con proyectos piloto. Por otro lado, la SG3 se ocupa de desplegar las mejoras en toda la organización, una vez que fueron probadas en proyectos piloto. Por esta razón resulta interesante detectar palabras clave en el enunciado para ver si se habla de un proceso definitivo o de una mejora que aún no fue implementada en la organización.
Hay que tener en cuenta que OPF se encarga de proponer las mejoras, probarlas en proyectos piloto, e informar a todos los proyectos de la organización cuando la mejora fue aceptada, pero es OPD quien se encarga de dejar asentadas en la biblioteca de activos de la organización las mejoras propuestas por OPF.

Formación organizativa (OT)

El área OT se ocupa de todo lo relacionado con la capacitación del personal, ya sea capacitando internamente o tercerizando la formación.
Esta área no se encarga sólo de capacitar al personal, sino que primero debe definir las capacidades con las que debe contar un empleado para poder ejecutar los procesos de la organización.
No se debe confundir con la SP2.4 de PP, ya que esta se enfoca a un proyecto en particular, mientras que está enfocada a la organización en general.

Monitorización y control del proyecto (PMC)

El área PMC está a cargo del seguimiento del proyecto para ver si se cumple con lo planificado y, en caso de no hacerlo, determinar las acciones correctivas que deben realizarse para tratar los problemas surgidos durante la ejecución del proyecto.
Otra función de PMC es registrar las lecciones aprendidas de cada proyecto, las que, como se mencionó anteriormente, servirán como fuente de información al área OPF para detectar posibles mejoras a los procesos.

Planificación de proyecto (PP)

El área PP se relaciona con un proyecto particular,previamente al comienzo del mismo. Entra sus tareas se encuentra todo lo relacionado a estimaciones, compromisos, riesgos, entre otras.
Un error común con esta área es confundir tamaño con tiempo y/o costo. La estimación del tamaño del producto (por ejemplo, mediante puntos función) está contemplada en la SP1.2, mientras que las estimaciones de tiempo y costo (por ejemplo, dividiendo el tamaño en puntos función por la productividad de la empresa) son tareas realizadas en la SP1.4.
Otro dato a tener en cuenta es la SP2.2 Planificar la gestión de los datos. Esta práctica específica se encarga de definir, entre otras cosas, qué datos se registrarán durante el proyecto, por lo que a los enunciados asociados con esta área de proceso se los suele relacionar (incorrectamente) con el área MA. Para evitar esta confusión se debe prestar

atención en el ámbito de aplicación que establece el enunciado; si habla de un proyecto específico probablemente esté refiriéndose a PP.

Aseguramiento de la calidad de proceso y de producto (PPQA)

El área PPQA es la encargada de controlar que los proyectos cumplan con los estándares definidos por la organización. Se limita prácticamente a evaluar los procesos y registrar resultados, ya que las mejoras en base a las falencias detectadas quedan a cargo de OPF.
Otra función de PPQA consiste en escalar las no conformidades cuando corresponda. Escalar una no conformidad consiste en informar al gerente adecuado cuando la misma no puede ser resuelta. Por ejemplo, si un equipo de trabajo utiliza una plantilla de forma inadecuada y, ante reiterados pedidos del área PPQA la situación sigue siendo la misma, entonces se le informa al gerente para que tome las acciones correctivas correspondientes.
No se debe confundir el Sector de SQA de una empresa con el área de proceso PPQA. El sector de SQA es un departamento físico que se encarga de evaluar los procesos y proponer mejoras, es decir que aplica las áreas de proceso PPQA y OPF.

Desarrollo de requerimientos (RD)

El área RD se ocupa de la comprensión de
los requerimientos y su posterior traducción a un lenguaje técnico para que
sean comprensibles por el equipo de trabajo.

Gestión de requerimientos (REQM)

El área REQM se encarga del seguimiento de los requerimientos durante toda la vida del proyecto. Entre sus tareas se encuentran las de gestionar los cambios y el impacto que los mismos tienen sobre el proyecto y el producto, ya sea en la duración del proyecto, impacto en otros productos de trabajo, entre otras.
Hay que tener en cuenta que, en lo referido a cambios, REQM se encarga solamente de un cambio de requerimiento, es decir, aquellos cambios que solicita el cliente. Los cambios generados por el equipo de trabajo no son incumbencia de esta área. Por ejemplo, si el equipo de trabajo solicita modificar una determinada función para hacerla más óptima no es un cambio de requerimiento. En los ejercicios de CMMI, ese tipo de cambios internos suelen relacionarse con el área CM.

Gestión de riesgos (RSKM)

El área RSKM, como su nombre lo indica, está a cargo de la gestión de los riesgos de un proyecto determinado. Sus tareas abarcan la identificación, cuantificación y mitigación de los riesgos, entre otras. Es un área que suele estar muy relacionada con las otras áreas de CMMI por lo que es difícil encontrarla en los ejercicios de CMMI.

Validación (VAL)

El área VAL se ocupa justamente de validar el producto con el cliente, de manera de comprobar que el mismo se ajusta a sus expectativas. Sus actividades abarcan la selección de productos a validar, la definición de los métodos a utilizar y la validación propiamente dicha. Es recomendable leer la Guía de Validación y Verificación para una mejor comprensión de estas áreas.

Verificación (VER)

El área VER es la encargada de los productos de trabajo cumplen con los requerimientos. Sus actividades están muy relacionadas con las de VAL pero se desarrollan a nivel interno. Por esta razón también se incluye la revisión entre pares como un método de verificación ya que las mismas se realizan entre miembros del equipo de trabajo.

Estructura de desglose de trabajo (EDT)

Se lleva a cabo en Planificación de Proyecto (PP), SP 1.1 Estimar El Alcance Del Proyecto.
Se puede extender en SP 2.4 Planificar Los Recursos Del Proyecto agregando la disponibilidad de recursos.
Es una disposición de los elementos de trabajo y de su relación entre ellos y con el producto final.
La EDT evoluciona con el proyecto. Inicialmente, una EDT de alto nivel puede servir para estructurar la estimación inicial. El desarrollo de una EDT divide el proyecto global en un conjunto interconectado de componentes manejables. Normalmente, la EDT es una estructura orientada al producto que proporciona un esquema para identificar y organizar las unidades lógicas de trabajo a ser gestionadas, las cuales se llaman “paquetes de trabajo”.
La EDT proporciona un mecanismo de referencia y de organización para asignar el esfuerzo, el calendario y la responsabilidad; y se usa como el marco básico para planificar, organizar y controlar el trabajo realizado en el proyecto. Algunos proyectos usan el término “EDT de contrato” para referirse a la porción de la EDT cubierta por el contrato (posiblemente toda la EDT). No todos los proyectos tienen una EDT de contrato (p. ej., desarrollo financiado internamente).
La EDT proporciona un esquema para organizar el trabajo del proyecto alrededor del producto y de los componentes del producto que da soporte al trabajo. La EDT debería permitir la identificación de los siguientes elementos:

- Riesgos identificados y sus tareas de mitigación.
- Tareas para los entregables y las actividades de soporte.
- Tareas para la adquisición de habilidades y conocimiento.
- Tareas para el desarrollo de planes de soporte necesarios, tales como los planes de gestión de la configuración, de aseguramiento de la calidad y de verificación.
- Tareas para la integración y la gestión de los elementos que no son de desarrollo.

La EDT de alto nivel está prevista para ayudar a calibrar el esfuerzo del trabajo del proyecto en términos de tareas, roles y responsabilidades organizativas. La cantidad de detalle en la EDT en este nivel más detallado ayuda en el desarrollo de calendarios realistas, con lo que se minimiza la necesidad de contingencia.

SPEM

Software Process Engineering Meta-Model es un estándar OMG (Object Management Group es un consorcio dedicado al establecimiento de diversos estándares, por ejemplo, UML).
Establece los elementos clave para la representación en ingeniería del software de métodos, ciclos de vida, técnicas, roles, actividades, procesos, metodologías, plantillas, etc.

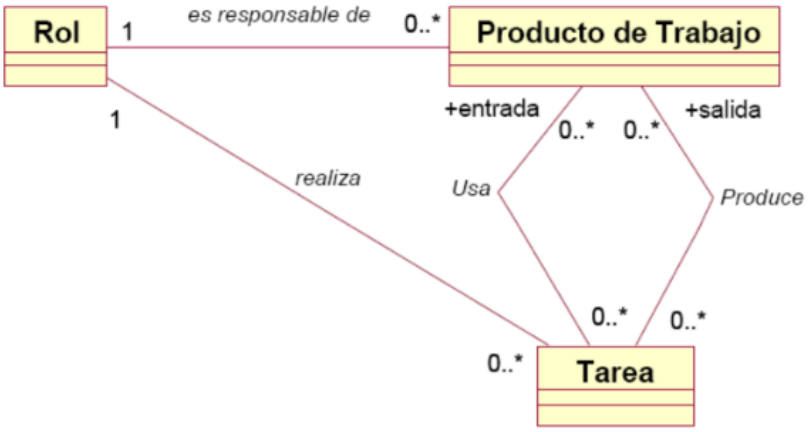
Capacidades

- Facilitar la comprensión y comunicación humana.
- Facilitar la reutilización.
- Dar soporte a la mejora de procesos.
- Dar soporte a la gestión de procesos.
- Guiar la automatización de procesos.
- Dar soporte para la ejecución automática.

Características

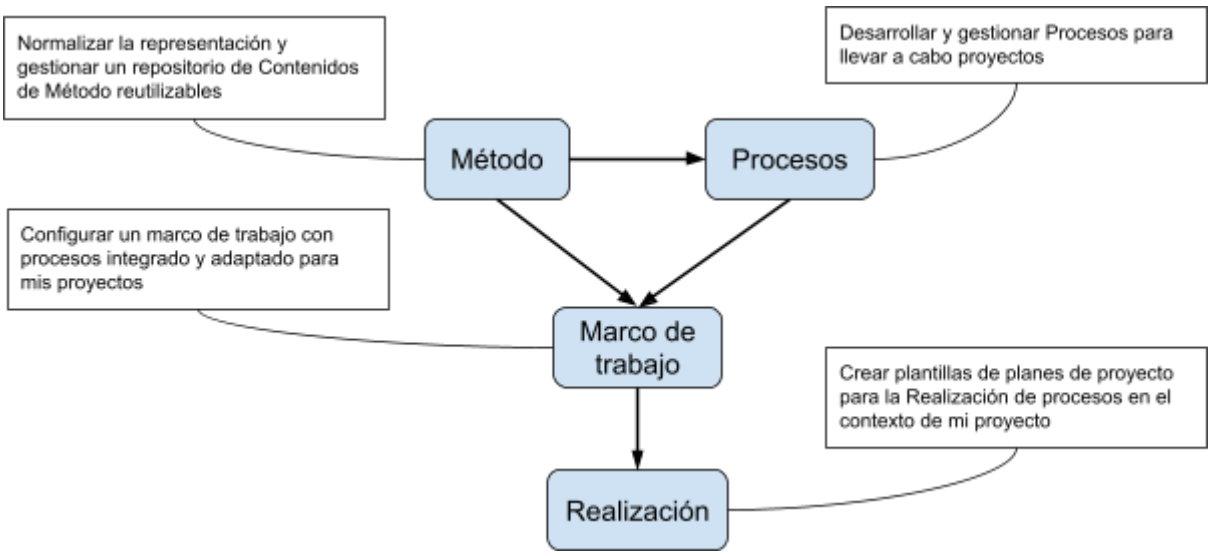
- Sirve para definir procesos de desarrollo de software y sistemas y sus componentes.
- Su alcance se limita a los elementos mínimos necesarios para definir dichos procesos sin añadir características específicas de un dominio o disciplina particular
- Sirve para métodos y procesos de diferentes estilos, culturas, niveles de formalismo, o modelos de ciclos de vida.
- No es un lenguaje de modelado de procesos en general, ya que está orientado a los procesos software.

Proceso según SPEM



- Las tareas representan el esfuerzo a hacer
- Los roles representan quien lo hace
- Los productos de trabajo representan las entradas que se utilizan en las tareas y las salidas que se producen.

Marco conceptual



Un método es la guía detallada que muestra secuencial y ordenadamente como realiza una persona un trabajo. En algunos métodos, los pasos exactos varían. Un proceso es el conjunto de elementos que interactúan para transformar los insumos en bienes o productos terminados.

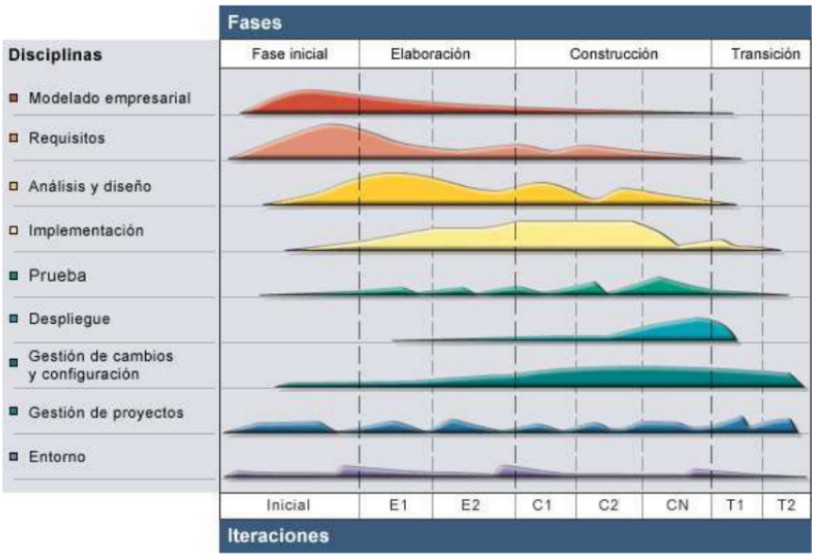
RUP

Elementos del proceso

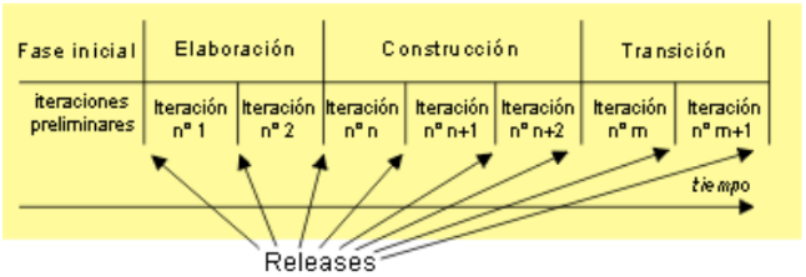
- Fase: El ciclo de vida del software se descompone en fases. Cada fase es esencialmente un periodo de tiempo entre dos objetivos importantes.
    - Concepción o Inicial (Inception)
    - Elaboración (Elaboration)
    - Construcción (Construction)
    - Transición (Transition)
- Disciplina: categorización de tareas basada en las similitudes de las preocupaciones y la cooperación del esfuerzo de trabajo. Una disciplina tiene una o más actividades
    - Modelado de negocio
    - Requisitos
    - Análisis y diseño
    - Implementación
    - Prueba
    - Despliegue
    - Configuración y Gestión de cambios
    - Gestión de proyectos
    - Entorno
- Guías: generaliza todas las formas de contenido cuyo objetivo principal es proporcionar explicaciones acerca de otros elementos.
    - Conceptos
    - Directrices
    - Materiales de soporte
    - Documentación

    - Plantillas
    - Listas de comprobación
    - Ejemplos
    - Informes
    - Guías de herramientas
- Actividad: da soporte al anidado y la agrupación lógica de elementos de proceso relacionados, también denominados elementos de desglose. Pueden contener referencias a tareas, roles y productos de trabajo.
- Tarea: describe una unidad de trabajo. Son llevadas a cabo por roles específicos. Dura entre unas horas y unos días. Afecta a un producto de trabajo o a un número reducido de ellos. Pueden desglosarse en secciones o pasos.
    - Rol
    - Producto de trabajo de entrada
    - Plantilla
    - Lista de comprobación
    - Directriz
    - Producto de trabajo de salida
- Rol: define un conjunto de habilidades, competencias y responsabilidades que están relacionadas.
- Producto de trabajo: resultado significativo de un proceso, los roles utilizan productos de trabajo para realizar tareas y producen productos de trabajo mientras realizan las tareas.
    - Artefacto: proporciona una descripción y una definición para productos de trabajo tangibles, es decir, que no son triviales.
    - Resultado: describe productos de trabajo intangibles que son un resultado o un estado.
    - Entregable: proporciona una descripción y una definición del empaquetado de otros productos de trabajo y puede entregarse a una parte interna o externa.
- Plantilla: tipo de guía especifica la estructura de un producto de trabajo proporcionando una tabla predefinida de contenidos, secciones, paquetes o cabeceras, un formato estandarizado, así como descripciones de cómo deben utilizarse y completarse las secciones.
- Directriz: proporcionan datos adicionales sobre cómo manejar un elemento de contenido concreto. Las directrices suelen aplicarse a las tareas y a los productos de trabajo.
- Lista de comprobación: identifica una serie de elementos que deben completarse o verificarse. Las listas de comprobación suelen utilizarse en las revisiones, como los ensayos o las inspecciones.
- Ejemplo: es un tipo de guía que representa una típica instancia de muestra, parcialmente completa de uno o más elementos de contenido. Los ejemplos suelen proporcionarse para los productos de trabajo
- Concepto: esboza una idea fundamental, referirse a temas más generales que las directrices y abarcan diversos productos de trabajo, tareas o actividades.
- Guías de herramientas: muestran cómo utilizar una herramienta específica para crear parte de un producto de trabajo en el contexto de una tarea o actividad o de forma independiente.

- Documentación: representa los documentos publicados externamente a los que se hace referencia en RUP.
- Informe: plantillas predefinidas de un resultado generado a partir de otros productos de trabajo como salida de alguna forma de automatización de una herramienta.



Fases e Iteraciones



Cada fase puede desglosarse en iteraciones. Una iteración es un bucle de desarrollo completo que tiene como resultado una versión (reléase) de un sistema ejecutable.

Fase Inicial

- Establecer el ámbito de software y las condiciones de los límites del proyecto, incluidas una visión operativa, criterios de aceptación y lo que debe contener el producto y lo que no.
- Identificar los casos de uso más importantes del sistema
- Definir una arquitectura posible
- Estimar el coste global y la planificación de todo el proyecto
- Estimar los riesgos potenciales
- Preparar el entorno de soporte para el proyecto.

Fase Elaboración

- Garantizar que la arquitectura, los requisitos y los planes son lo bastante estables, y que los riesgos están suficientemente mitigados para poder determinar con antelación el coste y la planificación de la finalización del desarrollo.
- Tratar todos los riesgos arquitectónicamente significativos del proyecto.
- Demostrar que la arquitectura dará soporte a los requisitos del sistema a un coste razonable y en un plazo razonable.
- Establecer un entorno de soporte.

Fase Construcción

- Minimizar los costes de desarrollo optimizando los recursos y evitando las reconstrucciones y los fragmentos innecesarios.
- Conseguir la calidad adecuada
- Conseguir versiones útiles (alfa, beta y otros releases de prueba)
- Completar el análisis, diseño, desarrollo y prueba de toda la funcionalidad necesaria.
- Desarrollar de forma iterativa e incremental un producto completo que esté preparado para la transición a su comunidad de usuarios.
- Decidir si el software, los sitios y los usuarios están listos para la aplicación que debe desplegarse.

Fase Transición

- Prueba de versión beta para validar el nuevo sistema contra las expectativas del usuario
- Prueba de versión beta y operaciones paralelas relativas a un sistema heredado al que sustituye
- Convertir bases de datos operativas
- Formación de usuarios
- Realizar el despliegue: como el empaquetado y la producción comercial y el despliegue de ventas.
- Solución de defectos, la mejora del rendimiento y la usabilidad
- Alcanzar la capacidad de soporte propio del usuario

Disciplinas

Modelado de negocio

- Entender los problemas actuales en la organización e identificar posibilidades de mejora.
- Evaluar el impacto del cambio organizacional.
- Garantizar que los clientes, usuarios, desarrolladores, y otras partes tienen una comprensión común de la organización.
- Obtener los requisitos de software del sistema necesarios para apoyar la organización de destino.
- Entender cómo el software a ser implementado encaja en la organización

Requisitos

- Establecer y mantener un acuerdo con los clientes y otros interesados acerca de lo que debe hacer el sistema.
- Proporcionar desarrolladores de sistema con un buen conocimiento de los requisitos del sistema.
- Definir los límites del sistema.
- Proporcionar una base para planificar el contenido técnico de las iteraciones.
- Proporcionar una base para la estimación del coste y del tiempo en que desarrollar el sistema.
- Definir una interfaz de usuario para el sistema, centrándose en las necesidades y los objetivos de los usuarios.

Análisis y Diseño

- Transformar los requisitos en un diseño del sistema en creación.
- Evolucionar una arquitectura para el sistema.
- Adaptar el diseño para que se ajuste al entorno de implementación.

Implementación

- Definir la organización del código, en términos de los subsistemas de implementación organizados en capas
- Implementar los elementos de diseño en términos de los elementos de implementación (archivos de origen, binarios, programas ejecutables y otros)
- Probar y desarrollar componentes como unidades
- Integrar los resultados producidos por los implementadores individuales (o equipos) en un sistema ejecutable

Prueba

- Buscar y documentar los defectos en la calidad del software.
- Validar y demostrar las suposiciones efectuadas en las especificaciones de diseño y requisitos con una demostración concreta.
- Validar que el producto de software funciona según lo diseñado.
- Validar que los requisitos se han implementado de forma adecuada.

Despliegue

- Describe las actividades asociadas al garantizar que el producto de software esté disponible para los usuarios
- Tres modalidades de despliegue
    - instalación personalizada
    - oferta de producto "comercializable"
    - acceso al software a través de Internet

Gestión de cambios y configuración

- Un sistema de Gestión de cambios y configuración es esencial para controlar los numerosos productos de trabajo que producen las muchas personas que trabajan en un proyecto común.

- Ayuda a evitar confusiones costosas y garantiza que los productos de trabajo resultantes sean conflictivos debido a algún problema de los tipos siguientes:
    - Actualización simultánea
    - Notificación limitada
    - Versiones múltiples

Gestión de Proyectos

- Proporcionar una infraestructura para gestionar los proyectos de software intensivo.
- Proporcionar directrices prácticas para la planificación, selección de personal, ejecución y supervisión de los proyectos.
- Proporcionar una infraestructura para gestionar los riesgos.

Entorno

- Proporcionar a la empresa de desarrollo de software un entorno de desarrollo de software (los procesos y las herramientas) que den soporte al equipo de desarrollo.

IBM Rational Method Composer (RMC)

IBM Rational Method Composer es una plataforma de herramientas que permite a los ingenieros y gestores de procesos implementar, desplegar y mantener procesos para organizaciones o proyectos individuales.
Permite la creación, configuración, visualización y publicación de métodos y procesos.

Unidad 3: Gestión de proyectos

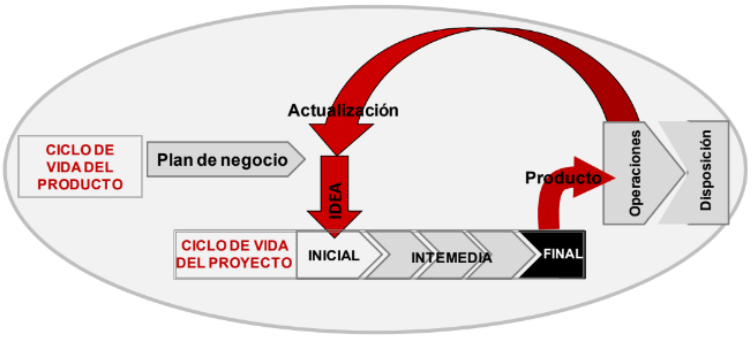
Definiciones

Proyecto

Es un conjunto de actividades coordinadas y controladas, con fechas de inicio y fin definidas, con la finalidad de crear un producto o servicio único conforme a requisitos específicos.

- Pueden ser de larga duración y estar sujetos a influencias externas e internas.
- Frecuentemente tienen restricciones de coste y recursos.
- Conllevan cierto grado de riesgo e incertidumbre.
- Crean productos entregables únicos, entendiendo por productos entregables los
- productos, servicios o resultados generados.
- Se define de forma general al comienzo del proyecto y se hace más detallado a medida que el proyecto avanza y el equipo entiende mejor los objetivos y los productos entregables.
- Tienen una duración limitada, con un comienzo y un final definido. El final se alcanza cuando se han logrado los objetivos del proyecto o cuando se cancela.

Por ciclo de vida se entiende el conjunto de fases que conectan el inicio de un proyecto con su fin. La transición de una fase a otra dentro del ciclo de vida de un proyecto generalmente está definida por alguna forma de transferencia técnica.
Relación entre el ciclo de vida del producto y ciclo de vida del proyecto:



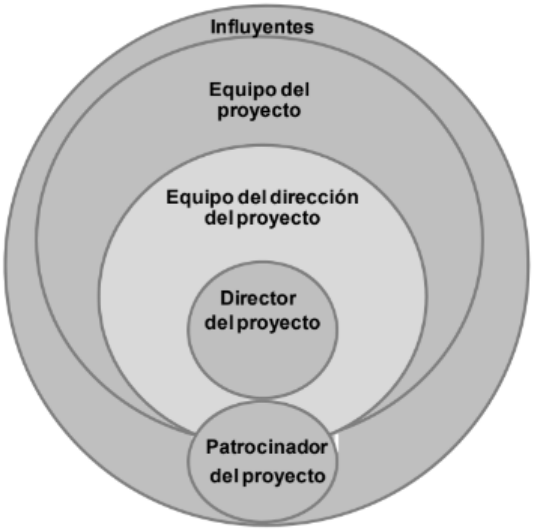
Gestión de Proyectos

Es conjunto de conocimientos, habilidades, herramientas y técnicas a las actividades de un proyecto para satisfacer los requisitos del proyecto.
La gestión de proyectos no es un proceso perfectamente definido. Cada profesional tiene enfoques distintos a la hora de gestionar sus proyectos, unos valoran más el control y el seguimiento mientras que otros se centran en los aspectos de liderazgo y gestión de personas.
Una gestión integral del proyecto es necesaria para abarcar todo el ciclo de vida.
La gestión de proyectos cambia la forma en la que se gestionan los eventos,su valor radica en contar con un proceso estandarizado para lidiar con las posibles contingencias, no hace desaparecer los problemas. Es un elemento imprescindible para lograr la calidad del resultado final, no una pérdida de tiempo, dinero o esfuerzo.

- Ahorros de tiempo y coste.

- Más rapidez en la resolución de problemas.
- Optimización en la resolución de riesgos.
- Mayor efectividad en la comunicación y gestión de expectativas.
- Mayor calidad de productos y servicios.
- Optimización de la gestión financiera.
- Mejora del proceso de toma de decisiones.
- Mejora del ambiente laboral.

Participantes:



Procesos

Coordinación del proyecto

Incluye los procesos y actividades necesarios para identificar, definir, combinar, unificar y coordinar los distintos procesos y actividades del proyecto.

- Iniciar el proyecto: definición, autorización y apertura formal del proyecto. Se documentan
- Desarrollar el plan del proyecto: define cómo se ejecuta, se supervisa y controla, y se cierra el proyecto.
- Gestionar la ejecución del proyecto: dirigir y gestionar la ejecución del proyecto, ejecutar el plan de gestión y lograr los objetivos del proyecto.
- Supervisar el trabajo del proyecto: seguimiento sobre todas las actividades del proyecto durante el inicio, la planificación, la ejecución y el cierre. Se toman mediciones, tendencias, se toman acciones correctivas y preventivas.
- Control integrado de cambios: se ocupa de que el plan de gestión del proyecto, el alcance del proyecto y los productos entregables se mantengan actualizados.
- Cerrar el proyecto: establecer los procedimientos para interactuar y coordinar la aceptación formal de los productos entregables y la transferencia del proyecto completado o cancelado

Alcance

Comprende los procesos necesarios para asegurarse de que el proyecto incluya todo el trabajo requerido, y sólo el trabajo requerido, para completar el proyecto satisfactoriamente. Definir y controlar que está y que no está incluido en el proyecto.

- Definir el alcance: describir en detalle los productos entregables del proyecto y el trabajo requerido. Se crean los requisitos a partir de las necesidades, expectativas, asunciones Produce el enunciado del alcance del proyecto.
- Definir las actividades: identifica y documenta las actividades a realizar dentro del alcance del proyecto para lograr los objetivos.
- Verificar y controlar el alcance: revisar los productos entregables, obtener la aceptación formal, gestionar los cambios y controlar su impacto sobre el alcance del proyecto.

Tiempo

Determina las dependencias y duración de las actividades y tareas. Establece las fechas de entrega y las fechas en las que deben recibirse entradas externas. Busca lograr la conclusión del proyecto a tiempo.

- Establecer la secuencia de las actividades: identificar y documentar las relaciones lógicas e interdependencias entre las actividades del proyecto teniendo en cuenta adelantos y atrasos.
- Estimar la duración de las actividades: determina la cantidad de períodos laborables necesarios para completar cada actividad estimando el esfuerzo y la cantidad prevista de recursos disponibles.
- Desarrollar el cronograma: establece un cronograma que sirve como línea base determinando las fechas de inicio y finalización planificadas para las actividades a través de un proceso iterativo. Exige que se revisen y corrijan las estimaciones de duración y las dependencias de las actividades. Debe identificar el camino crítico, las actividades críticas y los hitos del proyecto (entradas y salidas relevantes).
- Controlar el cronograma: determina el estado actual del cronograma, influye sobre los factores que lo afectan, determina los cambios que necesita y gestiona su implementación. Estudia las causas de las variaciones favorables y desfavorables que provocan cambios en el cronograma.

Costes

Se ocupan de la predicción y gestión de los costes del proyecto. Busca asegurar que el proyecto se completa dentro del presupuesto aprobado y que la información de costes esté disponible para la organización.

- Estimar los costes: desarrollar una aproximación de los costes de los recursos necesarios para completar cada una de las actividades contempladas en el cronograma. Costes de recursos humanos y materiales. Debe tener en cuenta incertidumbres como inflación, tipo de cambio, riesgos y contingencias.
  En caso de estar disponibles y ser de aplicación a las condiciones de proyecto, pueden contrastarse las estimaciones de coste de experiencias pasadas.
- Elaborar los presupuestos: consolida los costes estimados de las distintas actividades del cronograma en una línea base de coste, que servirá para la

supervisión del rendimiento del proyecto. Debe incluir contingencias de gestión para cambios no planificados.
- Controlar los costes: influir sobre los factores que producen cambios en la línea base de coste, registrar y comunicar los cambios en los costes..

Calidad

Determinan las políticas, los objetivos y las responsabilidades relativas a la calidad.

- Planificar la calidad: identifica las normas de calidad útiles para el proyecto y define cómo implementarlas. Debe tener en cuenta costes y beneficios.
- Realizar aseguramiento de la calidad: aplicación de actividades planificadas y sistemáticas relativas a la calidad para garantizar que el proyecto cumple con los requisitos. Revisa políticas, procesos y procedimientos de la organización para aplicar cambios que aumenten la efectividad y eficiencia de la organización.
- Realizar control de calidad: supervisar los resultados del proyecto, para determinar si cumplen con las normas de calidad e identificar modos de cambiar los resultados insatisfactorios.

Recursos

Se ocupan de planificar y controlar los recursos necesarios para alcanzar los objetivos del proyecto.

- Planificar los recursos: determinar los recursos que necesita cada una de las actividades del proyecto indicando cuales son, cantidad y cuanto tiempo se necesitan. Si es necesario, se descompone la actividad para poder asignar recursos a cada parte y luego sumarlos.
- Controlar los recursos: supervisa los recursos del proyecto y toma acciones correctivas cuando es necesario. Permite determinar si se dispone de recursos suficientes para cumplir los objetivos.

Personal

Crea un ambiente en donde las personas pueden contribuir eficaz y eficientemente a que se consigan los objetivos del proyecto.

- Definir el equipo del proyecto: determina los roles del proyecto, las responsabilidades y crea el plan de gestión de personal. Los roles y responsabilidades suelen documentarse con organigramas y descripciones de responsabilidades.
- Gestionar el equipo del proyecto: mejorar las competencias e interacciones de los miembros del equipo, evaluar desempeño, proporcionar retroalimentación, intermediar polémicas y brindar recompensas.. Busca mejorar el desarrollo grupal e individual.

Comunicaciones

Se ocupan de asegurar la generación, recogida, distribución, almacenamiento, recuperación y entrega final de la información del proyecto en tiempo y forma. Busca lograr que los intercambios de información sean exitosos.

- Planificar las comunicaciones: determina las necesidades de información y comunicación de los interesados. Se obtiene el plan de gestión de las comunicaciones que tiene el detalle de quién. cómo y cuándo y de quién requerirá información.
- Gestionar la información y los interesados: implica poner la información necesaria a disposición de los interesados en el proyecto de manera oportuna, satisfaciendo sus necesidades y resolviendo polémicas con ellos.

Riesgos

Busca aumentar la probabilidad y el impacto de los eventos positivos, y disminuir la probabilidad y el impacto de los eventos adversos para el proyecto.

- Planificar la gestión de riesgos: crea el plan de gestión de riesgos dónde se decide cómo abordar y llevar a cabo las actividades afines, incluye la metodología a seguir, los roles y responsabilidades, presupuesto y periodicidad.
- Identificar los riesgos: crea un registro de riesgos a partir de determinar los riesgos que pueden afectar al proyecto y documentar sus características, se compone de lista de riesgos, sus causas y sus posibles respuestas.
- Analizar los riesgos: de cada riesgo identificado, determinar la probabilidad de ocurrencia, el impacto del riesgo, el impacto acumulativo de múltiples riesgos y la prioridad.
- Planificar la respuesta a los riesgos: desarrolla las posibles opciones y determina acciones para mejorar las oportunidades y reducir las amenazas. Produce el plan de respuesta de riesgos con la estrategia, acciones y responsables de cada respuesta a cada riesgo.
- Controlar los riesgos: controla y monitoriza los riesgos teniendo en cuenta riesgos residuales, aparición de nuevos riesgos y los planes de ejecución.

Adquisiciones

Se encargan de comprar o adquirir fuera del equipo del proyecto los productos, servicios o resultados necesarios para realizar el trabajo.

- Planificar las adquisiciones: determina las necesidades del proyecto que pueden satisfacerse con productos o servicios externos. Define los requisitos en un documento que sirve como base para armar un contrato.
- Planificar la contratación: evaluar y seleccionar proveedores y preparar la documentación requerida para ellos.
- Solicitar respuesta a proveedores: solicitar a proveedores ofertas, propuestas o información.
- Seleccionar proveedores: la recepción de ofertas o propuestas y la aplicación de los criterios de evaluación para seleccionar un proveedor.
- Administrar el contrato: negociación y puesta en marcha del contrato, monitorización y control de cambios. Debe asegurarse de que el producto o servicio es entregado cuando es necesario y que está acorde con los requisitos del contrato.
- Cerrar el contrato: verificación del cumplimiento del contrato y obligaciones contractuales junto con las actividades administrativas necesarias para darle cierre. Se relaciona principalmente con la aceptación de los entregables.

Modelos de Gestión de Proyectos

PMBOK

Es un estándar de gestión de proyectos del PMI y acreditado por ANSI. Su objetivo principal es definir un conjunto de buenas prácticas para mejorar las posibilidades de éxito de un proyecto.
La guía se estructura en tres secciones:

- Primera sección: que establece el marco conceptual de la dirección de proyecto
- Segunda sección: dedicada a la norma para la dirección de proyectos
- Tercera sección: que detalla las 9 áreas de conocimiento de la gestión de proyectos.

ISO 10006:2003

La norma ISO 10006 forma parte de la colección de estándares ISO 9000 y tiene como objetivo servir de guía en aspectos relativos a elementos, conceptos y prácticas de sistemas de calidad para mejorar la gestión de proyectos. El estándar describe los procesos clave para gestionar un proyecto en diferentes áreas. La norma se enfoca en definir los procesos para garantizar la calidad de los proyectos, pero no define las técnicas a usar, dejándolo a juicio del equipo del proyecto.

METRICA

METRICA es una metodología del Ministerio de Administraciones Públicas que ofrece herramientas para sistematizar el ciclo de vida del software, con un conjunto de normas, técnicas y documentos para el desarrollo y mantenimiento de sistemas de información. METRICA V3 contempla una estructura principal con tres procesos básicos, y también incluye una serie de interfaces, como la interfaz de gestión de proyectos, que tiene como objetivo planificar, seguir y controlar las actividades y recursos del proyecto.

PRINCE2

PRINCE2® es una metodología de gestión de proyectos orientada a la generación de productos, que cubre aspectos de organización, gestión y control. Se compone de procesos que interactúan con componentes básicos como el caso de negocio, la organización, los planes y la gestión del riesgo, entre otros. Los procesos de gestión de proyectos según esta metodología son los siguientes: puesta en marcha del proyecto, dirección del proyecto, iniciación del proyecto, gestión de los límites de las etapas, control sobre la etapa, gestión de la entrega del producto, cierre del proyecto y planificación.

Unidad 3: Puntos por función

Introducción

El análisis de puntos función (APF) es un método para determinar el alcance y el tamaño del software a partir de una evaluación de los requisitos del usuario sustentado mayormente por la experiencia práctica. Además, el APF se puede aplicar en el mantenimiento y mejoras del software.
Para poder determinar el alcance y tamaño del proyecto de mejora es necesario conocer los puntos función (PF) del sistema actual. Una buena documentación funcional del sistema es fundamental para identificar el alcance de la mejora.
Si no está disponible, habrá que realizar el APF de la parte del sistema actual que se vea afectada. Para ello, es necesario identificar:

- Funciones del sistema
- Tamaño de cada función de datos (fichero lógico interno o fichero de interfaz externa) y los tipos de datos que utilizan.
- Tamaño de cada función de transacciones (entradas, salidas, consultas) y los tipos de datos y tipos de ficheros referenciados.

Se evalúan los cambios propuestos contra el funcionamiento actual del sistema, para detectar posibles cambios en las reglas de negocio, en los tipos de datos, en los tipos de ficheros referenciados o en la interfaz del usuario. El analista necesita considerar:

- El alcance de los cambios en funciones de datos y transacciones individuales.
- Las implicaciones adicionales que puedan tener los cambios especificados en otras funciones de datos y transacciones.

Propuesta de mejora: debe especificar las mejoras con detalle suficiente para poder evaluar los efectos en cada función de datos y transacciones afectadas. No debe dejar dudas ni ser ambigua.
Plan de pruebas: debe especificar las funciones de datos y transacciones que deben ser probadas y definir el alcance de esas pruebas. La unidad más pequeña a probar suele ser un componente del sistema, independientemente del alcance de la mejora. Debe identificar tanto los componentes como las funciones que se van a probar.

Metodología

El APF procede comienza determinando el tamaño de cada función que se va añadir, modificar o eliminar utilizando el método APF. El resultado es el tamaño de puntos función sin ajustar (PFD).
Cada función afectada debe ser analizada para identificar el alcance del impacto ya que las funciones de datos y transacciones añadidas, modificadas o eliminadas pueden tener impactos más allá de aquellos identificados en la propuesta de mejora. El factor de impacto (FI) refleja el grado de cambio de cada una de las funciones identificadas.
El tamaño de la mejora para cada función afectada se calcula multiplicando su tamaño básico (PFD) por el factor de impacto (FI). Por tanto, el tamaño de la mejora se mide en puntos función de la mejora (PFM).
Se lleva adelante en 6 pasos:

- Identificar las funciones dentro del alcance de la mejora: se utilizan como fuentes la propuesta de mejora, la documentación funcional y el APF del sistema actual, este

último resulta esencial porque todas las funciones existentes que se vean afectadas directa o indirectamente por la mejora contribuyen al tamaño de PF de la mejora. La parte afectada por el proyecto de mejora se calcula como la sumatoria:

- Determinar el tamaño de las funciones de datos y transacciones añadidas: especificar las funciones de datos y transacciones que se van a añadir a la aplicación. Se calcula el tamaño de las funciones que se van a añadir usando APF.
- Determinar el tamaño de las funciones de datos y transacciones eliminadas: las funciones a eliminar del sistema actual están descritas en la propuesta de mejora, cada una tiene sus correspondientes puntos por función.
  Para las funciones eliminadas se utiliza un factor de impacto de 0,4.
- Determinar el tamaño de las funciones de datos modificadas: pueden ser ficheros lógico interno (FLI) o ficheros de interfaz externa (FIE). Cada uno de estos tipos de funciones de datos se evalúa para identificar:
  - Funciones de datos que cambian internamente: tipos de datos añadidos, modificados o eliminados.
  - Funciones de datos que cambian pero no internamente (por ejemplo, un FLI que se convierte en un FIE o viceversa).

  El factor de impacto se determina comparando la cantidad de tipos de datos modificados con la cantidad total de tipos de datos existentes:
- Determinar el tamaño de las funciones de transacciones modificadas: Una función de transacción se considera modificada si se altera de alguna forma pero mantiene el mismo nombre y propósito después del cambio. Los motivos de modificación pueden ser:
  - Una transacción afectada por un tipo de datos que ha sido añadido, modificado o eliminado.
  - Una transacción afectada por un FLI o un FIE que ha sido añadido, modificado o eliminado.
  - La interfaz de usuario ha cambiado.
  - La lógica de negocio que hay detrás de la transacción ha cambiado.

  Para calcular los PF de la mejora producidos por un cambio en una transacción:
  - Identificar los tipos de datos y ficheros referenciados utilizados por la transacción.
  - Determinar los porcentajes de tipos de datos y ficheros referenciados que cambian como resultado de la mejora.
  - Determinar el factor de impacto para la transacción
  - Calcular el número de puntos por función de la transacción.
- Calcular el tamaño de la mejora
- Aplicar el factor de ajuste y calcular el número de puntos por función ajustado de la mejora
- Calcular el tamaño del sistema después de la mejora

El tamaño de las funciones a probar se puede medir en puntos función de pruebas (PFP). El tamaño de cada función a probar se mide mediante la metodología estándar de APF. Las funciones que se van a tener en cuenta son aquellas que van a estar en el sistema mejorado, por tanto, en este caso no importa distinguir si una función ha sido añadida, modificada o borrada.

El número de PFP se determina de la siguiente forma:

- Funciones de transacciones:

    - Determinar el tamaño en PF de cada transacción involucrada directamente en una prueba.
    - Calcular el número total de PF para todas las transacciones involucradas en las pruebas.
- Funciones de datos:
    - Determinar el tamaño en PF de cada función de datos involucrada directamente en una prueba.
    - Calcular el número total de PF para todas las funciones de datos involucradas en las pruebas.

El número total de PFP es la suma de las PFP de las funciones de transacciones y las PFP de las funciones de datos.

Una vez obtenido el tamaño de la mejora en puntos función (PFM), se puede establecer una correlación entre este tamaño y el esfuerzo necesario para la mejora. A partir del tamaño de las funciones a probar en puntos función (PFP), también se puede obtener el esfuerzo necesario para las pruebas.

Estos valores de tamaño, tanto de PFM como de PFP, se pueden utilizar para obtener métricas de productividad como, por ejemplo, horas por PFM y horas por PFP.

El esfuerzo total necesario para la mejora, incluidas las pruebas, se puede expresar como:

$$Esfuerzo\ total\ para\ mejora = \left(PFM \times \frac{horas}{PFM}\right) + \left(PFP \times \frac{horas}{PFP}\right)$$

Unidad 4: Requisitos

Introducción

Un producto bueno es la consecuencia de buenos requisitos, se deben entender qué pretende conseguir el producto y cómo hacerlo. Si no se entiende cómo se quiere que sea y funcione el producto, no se podrá construir correctamente.
Lo que hace el producto y las condiciones que debe satisfacer en este contexto son los requisitos del producto, deben ser entendidos por todas las partes.
El primer paso es la búsqueda de requisitos, es una exploración del producto con la intención de descubrir su funcionalidad y comportamiento, produce una descripción escrita de los requisitos que serán usados como entradas del diseño del producto.
Se busca expresar los requerimientos de manera tecnológicamente neutra para no influenciar el diseño ya que son un comunicado de las necesidades independientes de la implementación.
El diseño del producto traduce los requisitos en un plan en el que se detalle cómo será construido el producto final.
Una vez que el producto está en uso, los usuarios demandan más funcionalidad y el producto debe ser capaz de crecer alojando las nuevas demandas, se debe aceptar la evolución del producto.
La mayoría de los errores se originan en la etapa de requisitos, se termina pagando por el producto mucho más que su precio real. Esto se evitaría si el desarrollo y análisis de requisitos se hubiese realizado de la forma correcta desde el primer momento. El coste de una buena recogida de requisitos y análisis del sistema a desarrollar es menor comparado con el coste resultante de tener requisitos pobres.

Definición

Un requisito es algo que el producto debe hacer o una característica que debe tener. Un requisito existe por el tipo de demanda que tiene el producto o porque el cliente quiere que el requisito sea parte del producto entregado.
La tarea de todo analista de requisitos es hablar con la gente, entenderla, escuchar lo que dicen y también lo que no dicen, para entender lo que necesitan. Si bien el esfuerzo de recopilar los requisitos está repartido en todos los agentes que intervienen en el negocio, la traducción de ese conocimiento en requisitos de producto es tarea del analista.
Los pasos para identificar y definir requisitos:

1. Observar y entender el trabajo desde el punto de vista del usuario.
2. Interpretar el trabajo del usuario.
3. Inventar mejores formas de hacer el trabajo.
4. Plasmar estos resultados en forma de especificación de requisitos

Tipos de requisitos

- Requisitos de negocio: descripción a alto nivel de lo que el sistema debe hacer, proporcionan la dirección que ha de seguir el proyecto pero no el detalle. Forman la base de los requisitos de usuario.

- Requisitos de usuario: Son una descripción de las tareas que el sistema ha de ejecutar cuando el usuario opera con él. Estos requisitos describen la funcionalidad necesaria para satisfacer tareas específicas, necesidades operacionales y grupos de usuarios.
- Requisitos de sistema: definen las funcionalidades y características que debe tener el sistema para satisfacer tanto los requisitos de negocio como los de usuario. Este tipo de requisitos van a servir como base para llevar a cabo la arquitectura, diseño y planes de pruebas del sistema.
    - Requisitos funcionales: especifican las acciones que el producto debe llevar a cabo, aquello que el producto debe hacer desde el punto de vista del negocio. Su especificación sirve de contrato a la hora de construir el producto.
    - Requisitos no funcionales: son las cualidades que debe tener el producto, hacen que sea atractivo, útil, rápido, fiable o seguro. No son requeridos pero son deseables. Normalmente, son definidos después de la funcionalidad del producto.
    -
        - Look and Feel
        - Usabilidad y humanidad
        - Ejecución: rápido, seguro, disponible y exacto
        - Operacional: referentes al entorno operativo del producto
        - Mantenibilidad y soporte
        - Seguridad
        - Cultural y político
        - Legal

      También se puede clasificar como:
        - Requisitos de producto: usabilidad, eficiencia, fiabilidad, portabilidad, seguridad y escalabilidad
        - Requisitos organizacionales: entrega, implementación, estándares, recursos
        - Requisitos externos: interoperabilidad, legislación, privacidad, seguridad
- Restricciones: son condiciones que limitan las elecciones disponibles al diseñador o programador, son requisitos no funcionales.

Roles y responsabilidades

- RM: Gerente
- PM: Jefe de proyecto
- SQA: Responsable de Aseguramiento de la Calidad del Software
- CCB: Equipo de Control de Configuración

Actividad	Responsabilidad				Salida
	Preparación	Revisión	Aprobación	Responsabilidad	
Identificar los proveedores de requisitos y las autoridades firmantes	PM			PM	
Documentar los requisitos de usuario y de negocio	Equipo de proyecto	Cliente /RM	Cliente /RM	PM	Requisitos de usuario y de negocio
Documentar los requisitos de software/ sistema	Equipo de proyecto	Cliente /RM	Cliente /RM	PM	Especificación de requisitos software y especificación de casos de uso
Preparar y actualizar la matriz de trazabilidad de requisitos	Equipo de proyecto	SQA	PM/SQA	PM	Matriz de trazabilidad de requisitos
Analizar los requisitos				Equipo de proyecto/ PM	Matriz de trazabilidad de requisitos
Verificar y validar los requisitos. Obtener acuerdo		Cliente	Cliente	PM	Requisitos de usuario y de negocio, especificación de requisitos software, especificación de casos de uso
Línea base de los requisitos		SQA		PM	Línea base de los requisitos de usuario y de negocio, de la especificación de requisitos software y de la especificación de casos de uso
Gestionar cambios a los requisitos	PM	SQA	CCB	PM	Registro de peticiones de cambio, matriz de trazabilidad requisitos.

Desarrollo

La ingeniería de requisitos comprende:

- Desarrollo de requisitos: implica entender los requisitos de negocio, identificar los requisitos de usuario y trasladar los requisitos de usuario y de negocio a requisitos de sistema/software.
- Gestión de requisitos: implica gestionar los cambios de requisitos y mantener la consistencia entre los requisitos y otros productos de trabajo del proyecto.

Actividades:

1. Obtención de requisitos: Búsqueda de requisitos
2. Definición de requisitos: Escribir requisitos
3. Verificación de requisitos: Puertas de calidad
4. Revisión requisitos: Priorización

Obtención de requisitos

Consiste en identificar las necesidades del negocio, solucionando las posibles disparidades entre las personas, con el propósito de definir y destilar los requisitos para cumplir las restricciones impuestas por las distintas partes.

- Los requisitos han de ser completos, consistentes y han de estar dentro del alcance del proyecto.
- Los requisitos son identificados de forma única y han de priorizarse
- Cumplen con los objetivos de los clientes
- Son viables y apropiados para el desarrollo

- Están indicados de forma clara y no ambigua
- Los requisitos han de ser “testeables”, es decir, es necesario que sean comprobables para poder validarlos y verificarlos en etapas posteriores. Deben tener capacidad de prueba.

Se busca tener un entendimiento de la organización y de su misión. Existen 3 tipos de factores a tener en cuenta:

- Factores organizacionales: Personas que presentan las entradas del sistema objetivo, usuarios de las salidas del sistema objetivo, formas en las que el sistema objetivo cambiará la forma de hacer negocio de la organización.
- Factores de entorno: Restricciones de hardware y software impuestos en el sistema objetivo, la madurez del dominio del sistema objetivo, el papel del sistema objetivo dentro de un sistema mayor.
- Factores de proyecto: Los atributos de las distintas personas involucradas en el proyecto (estilos de gestión, experiencia en el dominio, etc.), las restricciones impuestas por las personas involucradas en el proceso de obtención de requisitos

Los clientes no proveen un listado de requisitos claros, los analistas de requisitos deben clasificar la información que surge de la voz del cliente. La información puede clasificarse en:

- Requisitos de negocio: describe los beneficios de negocio que una organización puede obtener de un producto.
- Casos de uso: Las declaraciones generales de metas de usuario o tareas de negocio que necesitan realizar con el sistema.
- Escenarios: descripciones de tareas específicas.
- Reglas de negocio: principios operativos sobre un proceso de negocio. Quién puede hacer, cuándo puede hacerlo, bajo qué condiciones.
- Requisitos funcionales: comportamientos observables que el sistema debe exhibir como respuesta del sistema.
- Atributos de calidad: un tipo de requisito no funcional que indica cómo de bien realiza el sistema alguna tarea o cómo de bien deja al usuario realizar alguna acción.
- Requisitos de la interfaz externa: conexiones entre el sistema y el resto del universo.
- Restricciones: tipo de requisitos no funcionales que limitan las opciones disponibles para el diseñador o programador.
- Definiciones de datos: descripción de formato, valores permitidos o valores por defecto para cada ítem de datos o la composición de una estructura de datos de negocio agrupadas como un diccionario de datos para los participantes del proyecto pueden utilizar durante el desarrollo y mantenimiento del proyecto.
- Ideas de solución: soluciones sugeridas por el cliente que no son un requisito.

Los requisitos que no encajen en estas categorías, pueden ser requisitos de no software.

Roles

El analista de requisitos se encarga del relevamiento de las necesidades del negocio para crear un producto que se ajuste al mismo. Lleva a cabo la búsqueda de requisitos junto a los usuarios y otros involucrados, hablando con ellos, escuchando lo que dicen, y entendiendo lo que necesitan. Traduce la información que ellos brindan a requisitos de negocio.

Sus tareas son:

- Observar y entender el trabajo que realizan los usuarios desde su punto de vista.

- Interpretar el trabajo filtrando la descripción que brindan los involucrados para obtener la esencia real del trabajo.
- Inventar mejores maneras de hacer el trabajo.
- Registrar los resultados en la especificación de requisitos y en modelos de análisis, debe asegurarse que los involucrados entienden el producto y están de acuerdo con cómo trabaja.

Pueden existir requisitos de los que los involucrados no están conscientes porque desconocen el uso de alguna tecnología o porque nunca han visto su trabajo de la manera que lo plantea el analista. Es tarea del analista de requisitos poder sacarlos a la luz.

Buenas prácticas

1. Chequear el alcance del producto durante la obtención de requisitos para validar que no estemos incluyendo requisitos de más o de menos.
2. Los requisitos expresan lo que el sistema tiene que hacer no en cómo (diseño).
3. Los modelos de análisis, escenarios y prototipos son más cercanos a los involucrados y pueden ayudar a encontrar errores u omisiones.
4. Reducir el número de participantes a las personas que representan los roles principales del proyecto puede ayudar a acelerar el proceso.
5. Un número suficientemente alto de participantes para no pasar por alto requisitos que son importantes para algún tipo de usuario o a especificar requisitos que no representan las necesidades de una mayoría de los usuarios.

Técnicas

- Entrevistas: técnica simple y directa, se busca no condicionar la respuesta del usuario a las preguntas, especialmente útil para:
    - Reunir información sobre un sistema existente
    - Determinar requisitos de un sistema nuevo
    - Clarificar especificaciones funcionales
    - Obtener información de entorno sobre la organización del cliente
    - Obtener realimentación respecto a la usabilidad
- Reunión: se usa para comunicar y promover la solución de un problema en grupo. Cuantos más participantes, requiere más formalidad. La efectividad se puede evaluar mediante una encuesta a los participantes.
- Formulario de recogida de observaciones: sirve para analizar un proceso de negocio reuniendo hechos que prueben una teoría u opinión y para empezar a detectar patrones en un proceso.
- Cuestionarios y Encuestas: se usa cuando la información proviene directamente de la gente, como actitudes, valores, hábitos o preferencias personales sobre aplicaciones o tecnologías existentes. Se pueden usar cuando:
    - Se requiera información en profundidad
    - Se quiere comprobar los distintos puntos de vista de las personas
    - Se requiere una tasa de respuesta mayor que la generada por cuestionarios vía mail
- Brainstorming: sirve para generar o reducir ideas. Se pueden utilizar algunas técnicas de votación para priorizar las ideas creadas.

- Casos de Uso: Identifican el quién, el qué y el cómo del comportamiento del sistema, describen las interacciones entre el usuario y el sistema. El modelo de caso de uso describe la totalidad del comportamiento del sistema. Los pasos para generar casos de uso son:
    - Identificar actores: los actores son entidades externas (usuarios primarios, secundarios o software/hardware conectado) al sistema que interactúan con él. Permite a los desarrolladores entender diferentes perspectivas del sistema.
    - Identificar escenarios: descripción narrativa de lo que hacen y
    - experimentan las personas cuando usan el sistema y las aplicaciones.
    - Identificar casos de uso: son una generalización de varios escenarios. Representa un flujo completo de eventos. Es muy común tener que añadir, eliminar, combinar y dividir los casos de uso antes de llegar a una versión final. La mejor forma de encontrar casos de uso es considerar qué espera cada actor del sistema.
- Prototipos: cuando el producto es muy innovador o cuando no se tienen suficientes detalles, se pueden usar prototipos, son un borrador de un producto potencial o de una parte del mismo, simular requisitos. Pueden ser de alta o baja fidelidad.
- Escenarios: descripción paso a paso de la funcionalidad de un caso de uso del producto o del negocio, sin demasiado detalle, con el objetivo de hacer entender cómo funciona este caso de uso. Están escritos en el lenguaje que utilizan los involucrados. Son un medio neutral, entendible por todo el mundo y se usan para llegar a acuerdos con los responsables del proyecto sobre el trabajo que se tiene que hacer.

Definición

Para conseguir requisitos claramente definidos se necesita conocer la perspectiva del usuario, en muchas ocasiones ellos se encuentran ocupados y no están disponibles para participar del proyecto. Puede ponerse un analista de negocio del lado del cliente para ayudar en esta situación.
Pueden reutilizarse requerimientos de otros productos o proyectos que nunca son completamente únicos, ya sea con algunos cambios menores o sin cambio alguno.
Para evitar malos entendidos con los requisitos, el analista de requisitos debe documentarse de manera que puedan ser probados y asegurarse que están validados con los involucrados del negocio.
Se puede probar tomando una muestra aleatoria de requisitos y pidiendo a los involucrados que los interpreten, si tienen interpretaciones distintas, necesita ser reescritos.
Los requisitos deben ser escritos en el lenguaje del negocio, debe ser familiar para ellos, buscando que los involucrados puedan entender los requisitos sin tener conocimiento técnico.
Buenas prácticas en definición de requisitos:

- Escribir requisitos de forma tecnológicamente neutra.
- Evitar ambigüedades.
- Usar los sujetos, evitar los pronombres.
- No usar adjetivos y adverbios confusos.
- No hablar en potencial (“debería”) porque deja entender que es opcional.
- Leer los requisitos en voz alta para encontrar errores.

- Confirmar que los involucrados del negocio entienden lo mismo que la persona que los escribe.

Buenas prácticas con respectos a la convención de nombres:

- Definir los términos más importantes en un glosario.
- Se deben usar los nombres y definiciones que se usan en la organización para que todos los proyectos tengan el mismo lenguaje y conceptos y evitar malos entendidos.
- Si los nombres que usa el negocio son erróneos o confusos, proponga mejores nombres.
- Elegir nombres que sean autoexplicativos.
- Crear un diccionario con cada término y definición incluyendo abreviaturas y acrónimos.

Verificación

Puede incluirse una etapa en el ciclo de vida “Verificación de Requisitos”, dónde el usuario final añade criterios de aceptación para cada requisito validando que sean correctos al momento de ser entregados a los diseñadores y desarrolladores.

La puerta de calidad es un punto por el que pasaron los requisitos antes de formar partes de la especificación, el analista de requisitos y el técnicos de pruebas validan la relevancia, coherencia, facilidad de seguimiento y facilidad de pruebas para luego autorizar a que formen parte de la especificación. Las puertas de calidad comprueban que cada requisito, de forma individual, tiene el estado correcto, no es ambiguo, están dentro del alcance, es probable y trazable A su vez, se valida que no existe una fuga de requisitos.

Criterio de aceptación de un requisito:

- Validar que los requisitos están completos y son correctos.
- Asociar los requisitos a las funciones de negocio.
- Eliminar ambigüedades.
- Validar que el resultado de cada requisito sea fácil de probar.
- Identificar las funciones de negocio que se cambiaron durante el proyecto.

Especificación

Si bien las puertas de calidad validan los requisitos por separado, es necesario validar la especificación de requisitos en conjunto.

Cuando la especificación está completa, se tendrá un conocimiento preciso del alcance y funcionalidad del producto, en ese momento debe realizar la revisión de la especificación.

Todos los tipos de requisitos del producto están presentes en la especificación (ejemplo: una web debe tener look and feel, un producto financiero, seguridad).

Los requisitos no deben tener conflictos, deben poder implementarse en conjunto, un requisito no puede impedir la implementación de otro (consistencia). Se puede usar una matriz para validar cada requisito con los demás requisitos.

Jefe de proyecto:

- Asegurarse de la ausencia de fugas de los requisitos
- Estimar el coste de la construcción del producto
- Evaluar los riesgos asociados a los requisitos
- Identificar áreas de mejora

Es una buena práctica realizar entrevistas con los agentes de negocio y sesiones grupales con los desarrolladores después que está terminada la especificación de requisitos para evaluar el trabajo y encontrar oportunidades de mejora. Se deben revisar y apuntar las respuestas y conclusiones de estas sesiones para usarse como punto de partida en las siguientes sesiones.

Priorización

Se debe entregar primero las funciones más importantes del producto (priorización), esto se vuelve especialmente importante cuando hay muchos requisitos.
El jefe de proyecto debe balancear el alcance del proyecto y las restricciones de agenda, presupuesto, recursos y metas de calidad. Debido a que cada involucrado tiene diferentes objetivos y metas de las distintas, se vuelve complejo priorizar requisitos.
Los clientes priorizan los requisitos inicialmente desde su perspectiva. Los desarrolladores especifican el coste, dificultad y riesgos asociados a los requisitos, los clientes deben decidir si son tan esenciales como pensaban. La priorización es un balance entre los beneficios de negocio de cada requisito, su coste y cualquier implicación que pueda tener. Si cliente no es capaz de diferenciar sus requisitos y clasificarlos, lo hace el jefe de proyecto,
Para priorizar se pueden agrupar los requisitos en categorías, luego se priorizan las categorías de requisitos. Los que están dentro de la misma categoría, tienen la misma prioridad. La prioridad es un atributo de cada requisito y debe incluirse en la especificación de requisitos y en los casos de uso.
También se puede priorizar en base a:

- Riesgos asociados
- Coste de la implementación
- Valor para el cliente
- Tiempo necesario para implementar el producto
- Facilidad de implementación tecnológica
- Facilidad de implementación organizacional o de negocio
- Objetivos del negocio
- Restricciones legales

Evolución

Puede darse que no todos los requisitos estén listos al iniciar las etapas de diseño y construcción.
Se puede empezar a crear el producto antes de que todos los requisitos estén perfectamente especificados.
Los agentes de negocio seleccionan casos de uso de alta prioridad para el negocio, los analistas de requisitos revisan solo los requisitos para esos casos de uso teniendo en cuenta la conexión con los demás requisitos.
Cuando un grupo de requisitos está aprobado, se entrega a los desarrolladores, quienes podrán empezar su trabajo. Se implementan un pequeño número de casos de uso tan pronto como sea posible para poder detectar posibles errores o deficiencias en los requisitos lo antes posible. Los analistas están trabajando en los siguientes requisitos mientras los desarrolladores implementan los anteriores. En poco tiempo, se consigue establecer un ritmo de liberación con nuevos casos de uso cada pocas semanas.

Los requisitos evolucionan a la vez que lo hace el producto, comienzan con ideas vagas de cliente, se van detallando con el trabajo de los analistas de requisitos, surgen nuevas ideas para el producto y los requisitos, a su vez, se van haciendo más precisos y más fáciles de probar.
Estos permanecen tecnológicamente neutros hasta que el diseñador se involucra en el trabajo y añade aquellos requisitos necesarios para hacer que el producto trabaje en el entorno tecnológico.
El desarrollo de requisitos trabaja con un modelo en espiral, o es un proceso iterativo que va desde la adquisición de requisitos hasta su validación.
Cada iteración se compone de:

1. En la primera etapa, se van a adquirir requisitos “en bruto”, sin analizar ni validar.
2. Se produce el primer borrador de la especificación de requisitos a partir del análisis y negociación de los requisitos en bruto aceptados y refinados.
3. Validar este documento para comprobar que es correcto.
4. Decidir si volver a repetir dicha iteración o aceptar la especificación de requisitos como completa y válida.

Gestión

La gestión de requisitos es el conjunto de actividades que ayudan al equipo de trabajo a identificar, controlar y seguir los requisitos y sus cambios en cualquier momento. Consiste en gestionar los cambios de los requisitos, las relaciones entre ellos, las dependencias entre la especificación de requisitos y otros documentos producidos por el proceso de desarrollo de software. De esta forma se asegura la consistencia entre los requisitos y el sistema construido.
Es un proceso que se desarrolla a lo largo de toda la vida del producto, finaliza cuando el cliente acepta el producto.
El proceso de gestión de requisitos se incluye dentro del plan del proyecto y en función de este plan el jefe del proyecto inicia las distintas tareas relacionadas con la gestión de los requisitos.
Objetivos:

- Gestionar la recogida de requisitos: definir los canales por lo que se obtienen los requisitos.
- Obtener la aprobación de los participantes del proyecto: clientes y desarrolladores. Establecer una línea base de requisitos aprobados sobre la que se puede solicitar cambios.
- Gestionar los cambios: identificar consistencias, identificar el origen de los cambios, analizar riesgos e impacto, documentar los cambios. Se aplica cuando aparecen requisitos nuevos o cuando se modifiquen requisitos aprobados. Se debe mantener trazabilidad entre los requisitos para evaluar correctamente el impacto.

Gestión de cambios

Los requisitos evolucionan a lo largo de todo el ciclo de vida.
Los cambios en los requisitos deben controlarse y documentarse con gestión de cambios.
Si una vez aceptada la línea base de requisitos se solicita un cambio, estos cambios no se pueden aceptar sin más ya que podrían afectar al desarrollo del sistema. Deben ser sometidos a un procedimiento de revisión formal.

Es importante detectar la fuente del cambio, la identificación del cambio se inicia desde el análisis de requisitos. Después se realiza el análisis del cambio y evaluación del impacto, debiendo quedar claro cuántos requisitos se verán afectados y cuánto costará (tiempo y recursos); sólo después se toma la decisión de la implementación del cambio, que será documentada al ser realizada.
Actividades:

1. Evaluar el impacto del cambio: evaluar usando la matriz de trazabilidad.
2. Valorar el cambio: estimar y evaluar viabilidad económica.
3. Realizar el análisis de requisitos y el análisis funcional.
4. Modificar todos los artefactos afectados.
5. Obtener la aprobación del cliente sobre las modificaciones introducidas.

Evaluar el impacto

Se debe recorrer todo el árbol de requisitos viendo cómo les afecta el cambio, se necesita que exista una trazabilidad para poder identificar correctamente los requisitos afectados.
Los requisitos deben ser trazables (rastreables), es decir, que se puedan identificar todas las partes del producto existentes que están relacionadas con ellos. Se necesita conocer quién lo propuso, cuál es la necesidad y las dependencias con otros requisitos y elementos.
La trazabilidad de requisitos es la correspondencia entre:

- Cada requisito del software y uno o más requisitos de usuario (trazabilidad hacia atrás). Está constituida por los enlaces que se establecen entre artefactos generados con anterioridad a la especificación de requisitos.
- Una o varias partes del diseño o la implementación (trazabilidad hacia adelante). Se corresponde con los enlaces entre los requisitos y los artefactos generados en posteriores fases de desarrollo.

Un requisito puede corresponderse con varias partes del código (funciones), que a su vez pueden implementar varios requisitos cada una. Por tanto, antes de cambiar el código para adaptarse a un cambio en un requisito, hay que comprobar que otros requisitos no quedan comprometidos. Pueden usarse matrices de trazabilidad de requisitos para documentar la correspondencia de los requisitos con diseño y código.

Aceptación del cambio

Una vez analizado el impacto del cambio, se decide si se acepta o si se negocia con el cliente.
Algunos aspectos a considerar para la aprobación de un cambio solicitado es:

- Impacto del cambio en el coste y funcionalidades del sistema.
- Impacto para el cliente y personas externas involucradas en el proyecto.
- Desestabilización potencial del sistema que pudiera ocasionar un cambio.

Implementación del cambio

Si se acepta el cambio, se deben modificar todos los artefactos que resultan afectados y generar una nueva línea base de requisitos.
Aprobado el cambio se procede a su implementación, de acuerdo a la fase del proyecto a que corresponda. En caso de que los cambios aprobados:

- Impliquen el desarrollo de un nuevo sistema, entonces será necesario comenzar un nuevo proceso de ingeniería de requisitos.

- Impliquen la implementación de nuevos requisitos, entonces será necesario comenzar por la actividad de recogida de requisitos..
- Afecten otras fases del proceso de desarrollo del proyecto, entonces se implementan en esas fases.

Mejores Prácticas

En desarrollo

- Documentar el alcance y visión del proyecto: La documentación del alcance y visión de un proyecto es esencial para alinear a todas las personas involucradas en una misma dirección. El alcance define las características del producto y una declaración de alcance ayuda a entender los requisitos y asegurar que todos trabajen hacia la misma meta.
- Mantener un glosario del proyecto: proporciona definiciones de palabras que son aceptadas y usadas por las personas involucradas en el proyecto.
- Técnicas de obtención de requisitos de usuario: Identificar y usar técnicas de obtención de requisitos que sean conocidas, familiares y que estén probados en la organización.
- Involucrar a toda la gente implicada: Involucrar a los clientes y usuarios reales en desarrollo y revisión de los requisitos para asegurar una validación temprana del entendimiento de los requisitos.
- Desarrollo incremental de requisitos: enfoque de desarrollo incremental, puede minimizar la cantidad de retrabajo.
- Entender por quién y para quién es cada requisito: Hay que identificar la necesidad de cada requisito y quién tiene la necesidad del mismo.
- Captura de requisitos usando casos de uso: La captura de requisitos utilizando casos de uso es útil para sistemas que implican la interacción con el usuario. Cada caso de uso encapsula un conjunto de requisitos, lo que facilita su gestión y seguimiento. No se deben utilizar para sistemas que no requieren interacción del usuario. Pueden ser representados como diagramas utilizando herramientas.
- Validar requisitos: La validación consiste en determinar si los requisitos funcionales y no funcionales son correctos y si el sistema se está desarrollando de manera adecuada para satisfacer su uso previsto.
- Verificar requisitos: evaluar la corrección y completitud de los requisitos, es asegurar que los requisitos proporcionan una base adecuada para llevar a cabo el diseño, la construcción y las pruebas.

En gestión

- Priorizar requisitos: priorizar los requisitos para determinar cuales van en la primera versión y cuales en sucesivas versiones.
- Establecer líneas base de los requisitos: Los requisitos aprobados se consideran una línea base y pueden servir como base para el desarrollo. Asegura que cualquier modificación en los requisitos que cambie la línea base se tratan como cambios de alcance.
- Comunicación abierta: establecer una línea clara de comunicación para asegurar que la información relacionada a los requisitos se comunique de forma consistente.

- Gestión de cambios de requisitos.
- Uso de herramientas para la gestión de requisitos: se pueden utilizar herramientas que permitan asignar y clasificar requisitos, trazabilidad, establecimiento de líneas base, histórico de cambios de cada requisito.
- Mantener trazabilidad de requisitos: capacidad de describir y seguir la vida de un requisito de forma bidireccional, se pueden usar matrices de trazabilidad.
- Establecer un plan de mejora de procesos para la ingeniería de requisitos: asegura la evolución de los procesos actuales de las organizaciones para cumplir con las necesidades actuales y futuras de forma más eficiente y con mayor calidad.
- Formar a los analistas de requisitos: proporcionar formación a los analistas de requisitos y seleccionar representantes de clientes/usuarios.

Unidad 5: El arte de probar el software

Psicología de la prueba

Una de las causas principales de las pruebas del programa pobres es el hecho de que la mayoría de los desarrolladores comienzan con una falsa definición del término prueba.
Definiciones erróneas del término prueba:

- La prueba es el procedimiento para demostrar que los errores no están presentes.
- El propósito de las pruebas es demostrar que un programa realiza correctamente las funciones previstas.
- Las pruebas son el proceso de establecimiento de la confianza que un programa hace lo que se supone que debe hacer.

Definición correcta del término prueba:

- La prueba es el proceso de ejecutar un programa con la intención de encontrar errores

Economía de la prueba

Las pruebas exhaustivas de caja negra o caja blanca son imposibles de realizar.
Pruebas tipo caja negra:

- Para probar exhaustivamente el programa del triángulo habría que crear casos de prueba válidos y posibles. Debería usar un número de casos de prueba infinito.
- Dos implicaciones de esto son que:
    - No se puede probar un programa para garantizar que esté libre de errores
    - Una consideración fundamental en la prueba de programas es la de la economía.

Pruebas tipo caja blanca:

- Gráfico de control de un programa pequeño
- El número de secuencias lógicas distintas es aproximadamente 100 trillones.

Principios de prueba

Prueba es el proceso de ejecutar un programa con la intención de encontrar errores.
Un buen caso de prueba es el que tiene una probabilidad elevada de encontrar un error aún no descubierto.
Un caso de prueba exitoso es el que descubre un error, aún no descubierto.

1. Una parte necesaria de un caso de prueba es una definición de la salida prevista o resultado. Por lo tanto, un caso de prueba deberá constar de dos componentes:
    - Una descripción de los datos de entrada al programa.
    - Una descripción precisa de la salida correcta del programa para este conjunto de datos de entrada.
2. Un desarrollador debe evitar tratar de poner a prueba su propio programa.
    - Programar mirada constructiva y probar mirada destructiva.
    - Falta de comprensión de la especificación.
    - La prueba se hace con mayor efectividad si la realiza otra persona.

3. El personal de prueba no debería depender del área de desarrollo. (Una organización de programación no debería probar sus propios programas.)
4. Inspeccione concienzudamente los resultados de cada prueba
5. Los casos de prueba deben ser escritos tanto para las condiciones de entrada inválidas e inesperadas, como para condiciones válidas y esperadas.
6. Examinar un programa para comprobar que no hace lo que se supone que debe hacer es sólo la mitad del problema. La otra mitad consiste en ver si el programa hace lo que no se supone que debe hacer.
    - Los programas deben ser examinados por los efectos secundarios no deseados.
7. Evite los casos de prueba desechables a menos que el programa sea verdaderamente un programa desechable.
    - Inventar casos de prueba sobre la marcha y no documentarlos, genera una pérdida de mucho trabajo.
    - Cuando haya una mejora y deba probarse nuevamente el programa, los casos de prueba deben ser reinventados, y generalmente se hacer pruebas menos rigurosas, y si la modificación causa un error que antes no tenía, a menudo pasa desapercibido.
    - Guardar los casos de prueba y ejecutarlos de nuevo después de cambiar otros componentes del programa se conoce como pruebas de regresión.
8. No se debe planear el esfuerzo de pruebas, con la suposición tácita de que no se encontrarán errores.
    - Los jefes de proyecto cometen el error de suponer que la prueba es el proceso de mostrar que el programa funciona correctamente.
    - Prueba es el proceso de ejecución de un programa con la intención de encontrar errores.
9. La probabilidad de encontrar errores adicionales en una sección de un programa es proporcional al número de errores ya encontrados en esa misma sección.
    - Otra forma es decir que los errores tienden a aparecer en grupos y que algunas secciones parecen ser mucho más propensas a errores que otras secciones
10. Las pruebas constituyen una tarea altamente creativa y son un desafío intelectual
    - La creatividad requerida para probar programas extensos supera la necesaria para concebirlos.
    - Existen métodos que permiten elaborar un conjunto razonable de casos de prueba para un programa, pero requieren un grado considerable de creatividad.

Unidad 5: Derivar casos de prueba de casos de uso

Los casos de usos junto con las especificaciones complementarias permiten derivar casos de prueba.
Casos de prueba de especificaciones complementarias.
Derivación de casos de prueba para:

- Pruebas de rendimiento / fiabilidad
- Pruebas de acceso / seguridad
- Pruebas de configuración
    - Impresoras
    - Conexiones de red
    - Configuraciones del servidor
    - Versiones de software para todo el software instalado
    - Hardware con el rendimiento más bajo
- Pruebas de instalación
    - Medios de distribución (CD-ROM o servidor de archivos)
    - Nueva instalación
    - Instalación completa
    - Instalaciones personalizadas
    - Instalaciones de actualizaciones

Unidad 5: Validación y Verificación

CMMI

Capability Maturity Model Integration (CMMI) es un modelo para la mejora de procesos que cuenta con 3 constelaciones:

- CMMI® for Development: incluye las áreas de proceso Validación y Verificación en la categoría de Ingeniería. Estas áreas de proceso son necesarias para conseguir el nivel de madurez 3.
- CMMI® for Services.
- CMMI® for Acquisition: incluye las áreas de proceso Validación y Verificación en la categoría de Adquisición. Estas áreas de proceso son necesarias para conseguir el nivel de madurez 3.

Metas y prácticas específicas

Son metas y prácticas que propone CMMi que aplican para las áreas de proceso de Validación y Verificación en Development y Acquisition.

Validación

La Validación en CMMI busca demostrar que el producto satisface el uso para el que se creó al situarlo sobre el entorno previsto.

- SG 1 Preparar la validación: selección de productos o componentes a validar, establecer y mantener el entorno de validación, procedimientos y criterios. Se puede usar el mismo entorno para realizar integración, verificación y validación y así reducir costo y aumentar eficiencia.
    - SP 1.1 Seleccionar productos a validar: Seleccionar los productos a validar y los métodos que van a aplicarse.
    - SP 1.2 Establecer un entorno de validación: Establecer y mantener el entorno necesario que soporte la validación.
    - SP 1.3 Establecer procedimientos y criterios de validación: Establecer y mantener procedimientos y criterios de validación alineados con las características de los productos seleccionados, las restricciones del cliente sobre validación, los métodos y el entorno de validación.
- SG 2 Validar los productos y los componentes de los productos: Los productos y componentes de productos son validados para asegurar que son apropiados para usarse en su entorno operativo.
    - SP 2.1 Realizar validación: Esta práctica permite la realización de la validación de acuerdo a los métodos, los procedimientos y los criterios.
    - SP 2.2 Analizar los resultados de las actividades de validación.

Verificación

La verificación en CMMi busca asegurar que los productos seleccionados cumplen con los requisitos especificados.

- SG 1 Preparación de la verificación

    - SP 1.1 Seleccionar productos para la verificación: Seleccionar los productos a verificar, los métodos que van a aplicarse y los requisitos a satisfacer.
    - SP 1.2 Establecer el entorno de verificación: Permite establecer y mantener el entorno que será usado para llevar a cabo la verificación.
    - SP 1.3 Establecer criterios y procedimientos de verificación: Permite el desarrollo de los procedimientos y de los criterios de verificación que están alineados con los productos de trabajo, requisitos, métodos y características seleccionados del entorno de verificación.
- SG 2 Realización de revisiones entre pares: examen metodológico de los productos para identificar posibles defectos del producto o recomendar cambios necesarios.
    - SP 2.1 Preparar revisiones entre pares
    - SP 2.2 Dirigir revisiones entre pares: conducir las revisiones entre pares e identificar temas resultantes de la revisión.
    - SP 2.3 Analizar los datos de las revisiones entre pares: analizar los datos obtenidos de las prácticas anteriores
- SG 3 Verificar la selección de productos: La verificación de los métodos, procedimientos y criterios se usa para verificar los productos seleccionados y cualquier mantenimiento asociado, formación y servicios de soporte, usando para ello el entorno de verificación apropiado.
    - SP 3.1 Realizar la verificación de los productos seleccionados.
    - SP 3.2 Analizar los resultados de las actividades de verificación.

Validación y verificación de adquisición

Existen algunas diferencias en las metas y prácticas de validación y verificación entre adquisición y desarrollo.

Validación

Según CMMI-ACQ®, la validación demuestra que el producto o servicio adquirido satisface las necesidades de las partes interesadas en el negocio y los requisitos del cliente.
Las actividades de validación son llevadas a cabo por el adquiridor, el proveedor o por ambas partes en función de lo establecido en el acuerdo con el proveedor
En el acuerdo con el proveedor normalmente se recogen los distintos aspectos de la validación:

- Lista de productos adquiridos a validar por el adquiridor.
- Lista de productos a validar por el proveedor junto con los clientes, usuarios y demás involucrados en el negocio.
- Métricas a recoger y a proporcionar por el proveedor con respecto a las actividades de validación.
- Rol que tendrá el proveedor en la validación del producto o componentes del producto.
- Entornos de validación a utilizar por el adquiridor.
- Procedimientos y criterios de validación aplicables al proveedor.

Los productos son seleccionados para ser validados por el adquiridor dependiendo de los atributos del proyecto.
La selección de los métodos busca el mejor procedimiento para predecir cómo de bien el producto o servicio adquirido satisfará las necesidades.

Verificación

La verificación de la adquisición establece si los productos adquiridos reflejan los requisitos especificados. Al igual que en la constelación de desarrollo, la verificación es un proceso incremental que ocurre durante todo el proceso de adquisición del producto o servicio.
Las actividades incluyen:

- Revisión del paquete de solicitud.
- Planes y acuerdos con proveedores.
- Documentos de requisitos.
- Restricciones de diseño desarrolladas por el adquiridor.
- Otros productos de trabajo desarrollados por el adquiridor.

Los proveedores deberían estar involucrados en la selección de los métodos de verificación para asegurarse de que son apropiados para el entorno de los proveedores.

Procesos de validación y verificación

Validación y verificación en el ciclo de vida

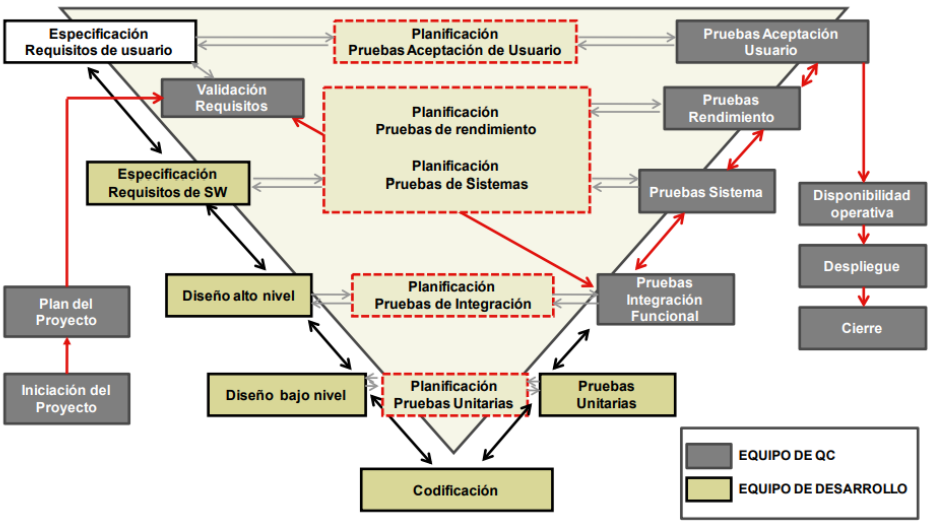
Es necesario que haya una planificación cuidadosa para obtener el máximo provecho de las inspecciones y pruebas, y controlar los costes de los procesos de validación y verificación.
Dicha planificación debería comenzar en etapas tempranas del proceso de desarrollo, como se verá, con la ayuda de los modelos de ciclo de vida.

Modelo en cascada

Las pruebas se realizan al final del ciclo de vida del proyecto por lo que los defectos son detectados cerca de la fecha de implementación del producto o sistema, lo que supone un coste muy elevado en la corrección de defectos.

Modelo en V

Intentó el problema de que los defectos se encontraban demasiado tarde, en la etapa final del proyecto.
Comienza a realizar pruebas tan pronto como sea posible en el ciclo de vida del producto, mucho antes del fin de la fase de codificación. Los técnicos de pruebas trabajan con los desarrolladores y analistas de negocio para ir llevando a cabo las actividades relacionadas a las pruebas en paralelo a las actividades de desarrollo. Se producen una serie de entregables de pruebas.



La planificación de la verificación y validación del sistema comienza en etapas tempranas del desarrollo.
Durante las fases de iniciación y planificación del proyecto se deberían definir los puntos de control, identificar los productos sobre los cuales llevar a cabo el control de calidad y la estrategia de esfuerzo relacionada con estas actividades (control de calidad).
Una vez finalizadas estas actividades se pasaría a la validación de requisitos, se trata de comprobar que los requisitos cumplen las necesidades del usuario, los errores en el documento de requisitos provoca importantes costos al repetir el trabajo cuando son descubiertos durante el desarrollo o después de que el sistema esté en uso. Se llevan a cabo las siguientes verificaciones:

- Verificaciones de validez: validar si lo que lo que el usuario piense que necesita del sistema coincide con el razonamiento y el análisis.
- Verificaciones de consistencia: los requisitos no deben contradecirse entre sí.
- Verificaciones de completitud: los requisitos deben describir todas las funciones y restricciones propuestas por el usuario del sistema.
- Verificación de realismo: los requisitos deben ser implementables con tecnología existente, bajo un presupuesto y una planificación.
- Verificabilidad: los requisitos deben ser redactados de forma verificable para poder demostrar si funcionan o no con un conjunto de pruebas.

Una vez validados los requisitos se debe pasar a la propia ejecución de pruebas.
Por último, la validación intervendría en actividades de disponibilidad operativa y despliegue del producto en producción para asegurar que tiene el nivel adecuado de calidad.
Las pruebas unitarias se van realizando a medida que se genera el código.
La verificación encajaría en todas y cada una de las fases del ciclo de vida porque en dichas fases es necesario comprobar que las tareas se están desarrollando de la manera adecuada tal y como se planificaron.

Modelo incremental y evolutivo

El sistema es analizado, diseñado, desarrollado y probado en incrementos claramente definidos.
Hay una versión probable del sistema disponible en etapas tempranas del proceso de desarrollo para comenzar con las pruebas, no es necesario que haya una implementación

completa del producto para que comiencen las pruebas, las funcionalidades pueden probarse a medida que se van añadiendo al sistema.

Modelo en espiral

Va generando iterativamente los requisitos a través de repetición de un ciclo que incluye prototipos, el feedback de los prototipos es la entrada para los siguientes requerimientos. Los desarrolladores construyen un prototipo inicial y lo prueban. Las pruebas, el re-diseño, y el prototipado deben ser actividades continuas hasta que el desarrollo del conjunto de características haya finalizado.
Se realizan pruebas sobre el primer prototipo muy tempranamente en el ciclo de vida.
Por último, se realizarán las pruebas oportunas unitarias, de integración y de sistema.
Haciendo uso de este modelo hay que tener cuidado para no entregar demasiado pronto al usuario un prototipo como producto final que aún no esté listo.

Modelo de Prototipos

Está muy presente tanto la validación como la verificación, ya que el desarrollador tiene muy en cuenta la opinión del cliente con el objetivo de conseguir cubrir sus necesidades, realizando revisiones de manera periódica para conseguir un producto final que cumpla los requisitos especificados.

Actividades de validación y verificación

Pruebas

Las pruebas son una parte imprescindible de los procesos de validación y verificación, ayudan a entregar un producto con la calidad suficiente que cumple las especificaciones definidas. Este objetivo conduce a las pruebas de validación, se espera que el sistema funcione correctamente usando un conjunto determinado de casos de prueba.
Caso de prueba: conjunto de entradas, condiciones de ejecución (precondiciones y post condiciones) y resultados esperados, desarrollado para conseguir un objetivo particular o condición de prueba.
Las pruebas sólo pueden demostrar la presencia de errores, no su ausencia. No pueden demostrar que el software se comportará en todo momento como está especificado. Las pruebas exhaustivas, en las que cada posible secuencia de ejecución del programa es probada, son imposibles.
Las pruebas se basan en un subconjunto de posibles casos de pruebas cuya elección se realiza siguiendo las políticas que establezca la compañía.
Las actividades de pruebas son integradas dentro de las diferentes fases del ciclo del software. Durante la etapa de planificación se establece la estrategia de pruebas y las técnicas de estimación. En la fase de Diseño, se diseñan los casos de prueba. Durante las fases de codificación y pruebas se ejecutan pruebas unitarias, de sistemas, de integración, entre otras.
Es necesario probar el sistema en un entorno de pruebas separado de producción que debe tener sus mismas condiciones.
Las pruebas son un proceso que intentan dar confianza al software con el objetivo de demostrar al cliente y desarrollador que el software satisface sus requisitos.

Proceso de pruebas:

- Engloba las actividades de definición, elaboración, ejecución y evaluación de pruebas del software.
- Busca la eficiencia, conseguir el efecto deseado sin desaprovechar recursos, es decir, evitando redundancias y reducir costos.
- Salidas:
    - Producto probado
    - Planes de pruebas con casos de prueba
    - Informes de pruebas
    - Elementos de pruebas (scripts, programas, datos) desarrollados para la ejecución de las pruebas
- Objetivos:
    - Establecer la participación de los roles
    - Definir el alcance, momento y características de las pruebas
    - Definir los contenidos de los manuales de usuario y de administración.
    - Establecer los requisitos necesarios para la aceptación del producto para avanzar de fase
    - Definir el ciclo de vida de gestión de un caso de prueba
    - Mecanismos de resolución de casos fallidos
    - Técnicas y estrategias aplicables en la elaboración y ejecución de las pruebas del software
    - Definir el contenido de un plan de pruebas y de un informe de pruebas
        - Plan de pruebas: documento que describe el alcance, enfoque, recursos y planificación de las actividades de prueba. Ayudan a los gestores a asignar recursos y a estimar el calendario de las pruebas y a los ingenieros software implicados en el diseño y la realización de las pruebas del sistema.
        - Informe de pruebas: refleja los resultados de la ejecución de los casos de prueba definidos, debiendo reflejarse las distintas ejecuciones realizadas sobre un caso de prueba, hasta la superación del mismo. Los técnicos de pruebas comunicarán con el informe los resultados de las mismas y de esta manera se podrán tomar decisiones adecuadas.

Sistema de pruebas: Las pruebas forman parte de lo que se denomina sistema de pruebas.

- Componentes:
    - Equipo de pruebas: ingenieros de pruebas, técnicos de pruebas y el responsable de las pruebas, diseñan, implementan, y usan componentes de pruebas.
    - Recursos de prueba: casos de prueba, datos de prueba, herramientas de pruebas, y otro material de desarrollo.
    - Procesos de prueba: condiciones informales, formales, no documentadas y documentadas en las que se realiza el trabajo de pruebas.
    - Entorno de pruebas: hardware, software, infraestructura de redes, oficina y laboratorio, y otros elementos que formen el lugar de trabajo.
- Arquitectura del sistema de pruebas: de diseño, la estructura y las herramientas elegidas con las que se construirá el sistema, refleja el sistema bajo pruebas.
- Ingeniería del sistema de pruebas: implementación de la arquitectura del sistema de pruebas, e implica el desarrollo del sistema de pruebas organizado y estructurado.

- Calidad: se puede medir la calidad de un sistema de pruebas al igual que se puede medir la calidad del sistema bajo pruebas.
    - El sistema de pruebas debe ser funcional. Debería cubrir los riesgos de calidad críticos.
    - El sistema de pruebas debe ser fiable, ante la misma prueba, se deben obtener los mismos resultados. Debe ser robusto.
    - Debe poder ser usado por todas las personas que quieran hacerlo, la curva de aprendizaje debe ser corta para personal calificado.
    - Debe ser portable, se debe poder ejecutar en cualquier plataforma.
    - Debe ser eficiente, la ejecución de las pruebas debe ser rápida.
    - Debe ser fácilmente mantenible.

Estrategia de pruebas

Una estrategia de pruebas son principios e ideas que puedan ayudar a guiar las actividades de pruebas. Como no es posible detectar y corregir todos los posibles fallos de un sistema, se busca un equilibrio entre una tasa de defectos aceptable y la inversión necesaria para conseguirlo. Se busca definir una estrategia de pruebas que optimice riesgos y costos.
Se pueden tomar distintos tipos de estrategias:

- Estrategia analítica: usa alguna técnica analítica durante la etapa de gestión de requisitos y de diseño del proyecto. Tienden a ser minuciosas y buenas a la hora de mitigar riesgos de calidad y encontrar errores pero requieren una inversión de tiempo importante.
    - Estrategia orientada a objetos: presta atención a los requisitos, el diseño, y la implementación de objetos. Se basa en una buena documentación, por lo que la ausencia de la misma implica no poder utilizarla.
    - Estrategia basada en riesgos: Consiste en identificar riesgos estudiando el sistema, proyectos anteriores o el contexto. Es apropiado si se dispone de tiempo para investigar el sistema.
- Estrategia basada en modelo: desarrollan modelos que representan cómo el sistema debería comportarse o trabajar. Usan algún modelo para simular comportamientos de sistemas críticos durante las etapas de requisitos y diseño del proyecto.
    - Estrategia basada en escenario: se realizan pruebas acorde a escenarios del mundo real.
    - Estrategia basada en el dominio: se analizan diferentes dominios de datos de entrada aceptados por el sistema, procesamiento de datos del sistema, y datos de salida entregados por el sistema.
    - Estrategia basada en un modelo: se diseñan, desarrollan y ejecutan las pruebas que cubran los modelos que se hayan construido. Esta estrategia será útil dependiendo de la capacidad del modelo para capturar los aspectos esenciales o potencialmente problemáticos del sistema.
- Estrategia metódica: sigue una línea informal con un enfoque ordenado y predecible que ayuda a comprender dónde probar.Sirven para sistemas que permanecen relativamente estables, o sistemas que son similares a otros ya probados. Los cambios significativos pueden frenar temporalmente estas estrategias hasta que se puedan volver a ajustar los objetivos de las pruebas a la nueva situación del sistema.
    - Estrategia basada en el aprendizaje: Se utilizan listas de control que se han desarrollado para guiar el proceso de pruebas.

    - Estrategia basada en funciones: Se identifica y se prueba cada función del sistema por separado.
    - Estrategia basada en la calidad: Se utiliza una jerarquía de calidad para identificar y probar la importancia de cada una de las características de la jerarquía como, por ejemplo, la usabilidad, el rendimiento, la funcionalidad o la escalabilidad.
- Proceso o estándar conformista: Siguen un desarrollo externo orientado a pruebas a menudo con pocas posibilidades de personalización. Usan algún estándar conocido que puede ayudar a hacer que el proceso de pruebas sea más transparente y comprensible para los programadores, responsables, analista de negocio y otras personas ajenas a las pruebas
- Estrategia dinámica: Minimizan la planificación por adelantado y prueban el diseño. Adaptan todo lo posible el sistema bajo prueba a las condiciones que habrá cuando se libere, enfatizan las últimas etapas de pruebas. Se trata de crear un pequeño conjunto de pautas de pruebas que se enfoquen en debilidades conocidas del software.
  Valoran la flexibilidad y la facilidad de encontrar errores.
    - Estrategia de pruebas intuitiva: según experiencia e instinto del equipo de pruebas.
    - Estrategia de pruebas exploratoria: se puede aprender simultáneamente sobre el comportamiento y diseño de pruebas, a la vez que las pruebas se van ejecutando y se van encontrando errores. Se irá refinando el enfoque de las pruebas en función de los resultados obtenidos de las pruebas.
- Estrategia de pruebas filosóficas: comienzan con una filosofía o creencia de las pruebas.
    - Estrategia de pruebas exhaustiva: que todo puede tener errores, la posibilidad de que haya fallos latentes es inaceptable, se buscan todos los errores.
    - Estrategia de pruebas shotgun: se acepta que no se puede probar todo y se aceptan errores.
    - Estrategia guiada externamente: se acepta que no se puede probar todo, se aceptan errores y se asume que no se sabe dónde están los errores. Confía en que hay personas externas, como usuarios, soporte técnico, desarrolladores, analistas de negocio, puede encontrarlos. Enfatizan las últimas etapas de pruebas simplemente debido a la falta de reconocimiento del valor de pruebas tempranas.
- Regresión: chequea la mala conducta de una función, atributo o característica previamente correctos. Descubre errores que previamente no se encontró durante la ejecución de una prueba.
    - Regresión local: al producirse un cambio o arreglarse un error se crea un nuevo error.
    - Regresión de exposición: al producirse un cambio o arreglarse un error se identifican errores ya existentes.
    - Regresión remota: un cambio o el arreglo de un error en un determinado área produce un fallo en otro área del sistema.

  La estrategia más simple para enfrentarse a la regresión es la fuerza bruta, repitiendo todas las pruebas.

Automatización: estrategia para aumentar la calidad de un producto a un bajo coste y optimizar el esfuerzo de las pruebas. Es la única manera de repetir todas las pruebas sobre sistemas complejos y grandes.

- Desventajas:
    - Costo Alto
- Ventajas:
    - Reduce el esfuerzo de las pruebas de regresión.
    - Permite realizar validaciones durante los ciclos de cambios.
    - Habilita la consistencia y cobertura lógica, no hay riesgo de excluir casos si se diseña correctamente.

En cada liberación se prueban las funciones liberadas con anterioridad y, a veces, las que actualmente se está liberando. En ocasiones, puede ser necesario seleccionar un subgrupo de las pruebas automatizadas para correr para ser más prácticos, para hacer esta selección hay diferentes técnicas:

- Trazabilidad: las pruebas están relacionadas con el comportamiento del sistema, se analiza cuales son los elementos afectados por algún cambio o por el arreglo de errores, se realiza un seguimiento de las pruebas asociadas y se selecciona las pruebas que se van a volver a ejecutar.
- Análisis de cambios: se presta atención a las descripciones estructurales del sistema, definiendo cómo los efectos de los cambios afectan al sistema. Probablemente será necesaria la ayuda de personas especializadas en el tema.
- Análisis de riesgos relacionados con la calidad: análisis de riesgos de calidad para determinar qué áreas tienen un alto riesgo que pueda afectar al negocio.

Niveles de pruebas

Los técnicos de pruebas comunican a los responsables la situación del proyecto y ellos autorizan el presupuesto necesario para solventar los problemas.
Las pruebas deben adecuarse al proyecto. Una manera de adecuar las actividades de pruebas al proyecto es dividir el esfuerzo total de pruebas en una secuencia de fases o niveles, se organizan por la secuencia en la que las porciones del sistema llegan a estar listas para probar. Cada fase por lo tanto cubre un objetivo de prueba a alto nivel.

- Pruebas unitarias: Están en la fase de desarrollo del producto, las pruebas unitarias se realizan para cada módulo individual del sistema antes de integrarse. Pueden usarse subs o drivers para reemplazar el software omitido y simular una interconexión de componentes. Las realizan los desarrolladores. Se busca automatización y repetitividad.
  Tareas a seguir:
    - El desarrollador revisará la correcta lógica y funcionamiento del módulo. Se intentará controlar los siguientes elementos:
        - Prueba y control de errores y excepciones.
        - Tipo y valor de los parámetros de entrada y salida en métodos y funciones.
        - Verificación de la lógica de negocio y algorítmica del módulo.
        - Revisión de condiciones y valores límite.
        - Prueba de elementos de repetición o bucles, y anidaciones entre los mismos.
        - Simulación de caminos y alternativas posibles.

    - Evaluación de los resultados obtenidos frente a los resultados esperados.
    - Corrección del módulo hasta alcanzar su correcto funcionamiento.
- Pruebas de integración: toma como punto de partida el producto integrado resultado de la fase de desarrollo. Con el producto integrado, se realizan las pruebas pertinentes para verificar la correcta integración y cohesión del sistema. Se tomarán como referentes las especificaciones del documento de análisis y más especialmente del de diseño, verificando que la división arquitectónica y de módulos establecida funciona correctamente.
  El propósito principal es descubrir los defectos de las interconexiones y de la interacción entre sistemas o componentes integrados.
  Estrategias:
    - Integración Big-bang: todas las unidades se ensamblan en una sola resultando un sistema completo, no es necesario simular ninguna parte porque todo está acabado antes de empezar las pruebas de integración, consume mucho tiempo y tiene que realizar un seguimiento de las causas de los fallos de esta integración. Tiene el riesgo de descubrir problemas al final del proyecto, por lo que es más caro resolverlos.
    - Estrategia bottom-up: se empiezan probando los módulos de nivel inferior para ir ascendiendo progresivamente probando los módulos superiores. Se deben crear programas auxiliares de nivel superior que permitan probar la integración de los componentes desde el principio, ya que no se dispone del sistema completo hasta el final.
        - Requiere de la construcción de elementos de prueba (test-drivers) en cada nivel.
        - No se encuentran problemas de diseño hasta muy avanzado el proceso.
        - Es apropiado para sistemas orientados a objetos.
        - Es necesario para componentes críticos.
    - Estrategia top-down: partiendo de los subsistemas o componentes de nivel superior, se prueba la integración entre los mismos. Se va descomponiendo el sistema en subsistemas de más bajo nivel en cada iteración. Se necesita construir componentes auxiliares que simulen el correcto funcionamiento de módulos de más bajo nivel mientras éstos no se hayan integrado en el sistema objeto de pruebas.
        - Esta estrategia de pruebas es usada conjuntamente al desarrollo top-down.
        - Descubre rápidamente errores en la arquitectura.
- Pruebas de sistema: se ocupan del comportamiento del sistema o del producto como un todo. El comportamiento del sistema está documentado en la especificación funcional. Esta especificación debería contener definiciones tanto de los requisitos funcionales como de los no funcionales del sistema.
  Toma como punto de partida el producto integrado y probado, almacenado en la línea base del sistema. Se verifica el comportamiento global del sistema.
  Existen varios tipos de pruebas para verificar el funcionamiento del sistema y para validar que se satisfacen los requisitos de cliente, los tipos de pruebas a usar se indican en el plan de pruebas:
    - Pruebas funcionales, para verificar la correcta funcionalidad del mismo.
    - Pruebas de instalación, configuración y carga inicial de datos.

    - Pruebas de usabilidad.
    - Pruebas de migración de datos.
    - Pruebas de prestaciones.
    - Pruebas de seguridad.

  En esta fase también se elaborarán los documentos destinados al usuario, como son los manuales de usuario y de administración.

  Las pruebas de sistema normalmente son llevadas a cabo por un equipo independiente de técnicos especializados en pruebas que puede ser inclusive externo o compuesto por analistas de negocio.

  Las pruebas de sistema de los requisitos funcionales usan técnicas basadas en especificación (o de caja negra) sobre el sistema probado. Las técnicas basadas en estructura suelen usarse para valorar la minuciosidad de los elementos de pruebas.

  Requieren un entorno de pruebas controlado en lo que se refiere a un control de versiones del software o datos de prueba. El entorno de pruebas debería ser lo más similar al entorno de producción tanto como sea posible para evitar encontrar errores específicos del entorno.
- Pruebas de aceptación: tienen como objetivo obtener la aceptación final del cliente antes de la entrega del producto para su paso a producción. Se realizan cuando ya se realizaron las pruebas de sistema y se corrigieron la mayoría de los defectos. Las realiza un usuario o cliente determinado.

  Son pruebas funcionales sobre el sistema completo que buscan comprobar que se satisfacen los requisitos establecidos. La ejecución de las pruebas de aceptación requiere un entorno de pruebas que represente el entorno de producción.
    - Pruebas alfa: consisten en invitar al cliente a que pruebe el sistema en el entorno de desarrollo. Se trabaja en un entorno controlado y el cliente siempre tiene un experto a mano para ayudarle a usar el sistema. El desarrollador va registrando los errores detectados y los problemas de uso.
    - Pruebas beta: se realizan con posterioridad a las prueba alfa, y se desarrollan en el entorno del cliente. El cliente se queda a solas con el producto y trata de encontrarle fallos de los que informa al desarrollador.

Buenas prácticas

- Proceso continuo e iterativo a lo largo del ciclo de vida del software.
- Su principal objetivo es prever y detectar anticipadamente posibles fallos y problemas del software. Adicionalmente, sirve para validar ciertas características del mismo (requisitos y especificaciones técnicas).
- Las pruebas de sistema deben realizarse en un entorno lo más parecido al entorno real de producción del sistema.
- Es un proceso ordenado y metódico, repetitivo y sistemático. Necesita orden.
- Las pruebas deben preverse, planificarse, gestionarse y documentarse.
- Cada caso de prueba debe estar codificado y asociado al menos a un requisito para tener trazabilidad.
- Las pruebas, en particular las de regresión, deben ser repetibles y automatizables.
- El entorno de pruebas debe limitar y controlar la promoción de nuevas versiones corregidas mientras se realizan las pruebas.
- Los casos de prueba deben ser concretos, precisos y apropiados, deben cubrir la mayor cantidad de posibilidades y código posible.

- Los desarrolladores no deben probar su propio código, sino personal con experiencia en realización de pruebas.
- Los elementos de pruebas construidos deben estar documentado y guardados dentro del repositorio de gestión de configuración con el resto del proyecto.

Tipos de pruebas

- Pruebas de prestaciones: su objetivo es medir las prestaciones del sistema.
    - Pruebas de Carga: validar los requisitos en cuanto a las prestaciones definidas para el sistema (ejemplo: tiempo máximo de respuesta). Para su verificación se generan escenarios de carga que modelan de forma realista el comportamiento de los usuarios.
    - Pruebas de Capacidad: buscan obtener el punto umbral a partir del cual las prestaciones del sistema se degradan.
    - Pruebas de Estrés: verifica el comportamiento del sistema en una situación de sobrecarga, debe mantener la integridad.
    - Pruebas de Escalabilidad: verifica la capacidad del sistema de recibir incrementos de requisitos, verificando la flexibilidad del sistema para progresar en el servicio.
    - Pruebas de Estabilidad: verifican el comportamiento del sistema en el tiempo buscando problemas por una incorrecta gestión de los recursos del sistema y una mala liberación de los mismos. Se prueba una carga normal y estable por un largo período de tiempo y se verifica que no aumenten los recursos ni se degraden las prestaciones.
- Pruebas de usabilidad: mide cuán fácil, cómoda e intuitivamente pueden interactuar los usuarios con las aplicaciones, haciendo hincapié en la necesidad un buen diseño de la interfaz de usuario. Es muy habitual que se ponga un conjunto de usuarios a usar el sistema con asistencia para que informen cuales son las dificultades e inconvenientes que se encuentran
  Los aspectos a probar son:
    - La navegación por la aplicación y la secuencias de pasos a seguir
    - La presencia y organización de la información.
    - La flexibilidad en la realización de operaciones.
    - La existencia de etiquetas y mensajes apropiados.
    - La existencia de información de estado
- Pruebas de regresión: Son pruebas que buscan descubrir los fallos de regresión, es decir, funcionalidades que previamente funcionaban y con las versiones posteriores no funcionan más. Esto sucede por consecuencias involuntarias de los cambios del software que provocan efectos adversos no deseados.
  Consiste en volver a ejecutar las pruebas anteriores, es recomendable que se encuentren automatizadas.
  Las pruebas de confirmación aseguran que luego de que se encuentre un error, la corrección funcione como se espera.
- Pruebas funcionales: buscan probar que los sistemas desarrollados cumplen con las funciones específicas para los que han sido creados. Normalmente son ejecutadas por analistas de pruebas con apoyo de algunos usuarios finales. Las pruebas funcionales son pruebas basadas en el análisis de la especificación funcional. Se las denomina como pruebas de caja negra porque no prestan atención a la manera de

generar las respuestas, solo analizan datos de entrada y de salida sin preocuparse por el funcionamiento interno.
Para analizar la funcionalidad de un producto se centra en la conformidad, interoperabilidad, seguridad y en la exactitud.

Revisiones

Es la primera forma de pruebas que se lleva a cabo durante el ciclo de vida, ayudan evitar que los defectos lleguen al código ejecutable.
Revisar el código según estándares de desarrollo previene también la aparición de defectos, aunque como ya está escrito el código no se evita el costo de tenerlo que reescribir.
Es una tarea relacionada fundamentalmente con el proceso de verificación, se puede realizar sobre los requisitos, el diseño, el código, etc.
La revisión es una actividad que puede llevarse a cabo durante todo el ciclo de vida de desarrollo de un producto.
Todas las revisiones siguen los mismos principios:

- Identificar los entregables a revisar.
- Desarrollar la lista de participantes en la revisión.
- El documento bajo revisión es estudiado por los revisores.
- Los revisores identifican los problemas o temas a tratar en la revisión y se lo comunican al autor de forma verbal o mediante un documento.
- El autor responde a los comentarios y actualiza el documento según corresponda.

Los beneficios de las revisiones son:

- Mejorar la calidad y el entendimiento de los entregables.
- Identificar y gestionar las expectativas de los involucrados en el negocio.
- Validar que los entregables soporten la solución final.
- Identificar tareas de alto riesgo.
- Formar al equipo de proyecto y a los involucrados en el negocio.
- Detectar problemas relacionados con la calidad de forma temprana en el desarrollo de los entregables.

Las revisiones tienden a reducir la cantidad de errores enviados a pruebas y, en consecuencia, su costo. A menos tiempo invertido en corregir, mayor productividad. Usar revisiones y pruebas a la vez provoca una mejora en la calidad del sistema entregado a los clientes y a los usuarios.

- Revisión entre pares: implican un examen metódico de los productos de trabajo para identificar y eliminar defectos de manera temprana y para recomendar otros cambios que sean necesarios. Se realizan de manera incremental, a medida que los productos de trabajo van siendo desarrollados.
  Estas revisiones son llevadas a cabo entre pares. Los pares son generalmente compañeros de trabajo en un proyecto que tienen interés y conocimientos acerca del elemento bajo revisión.
  Se pueden llevar a cabo sobre:
    - Artefactos de gestión del proyecto, como los planes que se van generando.
    - Artefactos de gestión del proceso, como por ejemplo descripciones de procesos.
    - Artefactos de soporte (medidas, documentación y productos de trabajo de formación).

  Guías:

    - Proporcionar un entorno no amenazante para la discusión.
    - Dar formación al personal sobre los roles de cada uno dentro de la revisión.
    - Registrar los defectos y problemas en tablas con datos sobre localización del defecto, descripción del defecto (tipo y origen), comentarios y elementos de acción.
    - Deben registrarse datos consistentes y suficientes.
    - Gestionar y controlar la revisión por pares.
    - Enfocarse en el producto de trabajo en revisión, no en la persona que lo produjo.
    - Cuando surgen problemas durante la revisión entre pares, deberían comunicarse al desarrollador principal del producto para su corrección.
    - Debe incluirse la revisión entre pares en la planificación del proyecto para ayudar a la asignación adecuada de tiempo y evitar que se salte la preparación de la misma.

Técnicas y herramientas

Técnicas de pruebas

Pruebas realizadas en distintas etapas del ciclo de desarrollo del software encontrarán diferentes tipos de defectos. Por ejemplo, es más probable que las pruebas unitarias encuentren defectos lógicos en el código y no en el diseño del sistema.
Las pruebas estáticas y las pruebas dinámicas son métodos complementarios, detectan diferentes tipos de defectos, las pruebas estáticas están orientadas a encontrar defectos mientras que las dinámicas fallos. Las dinámicas están más orientadas a la validación y las estáticas están más orientadas a la verificación.

Técnicas de pruebas dinámicas

Ejecutan el software. Para ello introducen una serie de valores de entrada, y examinan la salida comparándola con los resultados esperados.

- Basadas en la especificación: técnicas de pruebas de caja negra, tratan el software como una caja negra con entradas y salidas, pero no tienen conocimiento de cómo está estructurado el programa. El técnico de pruebas se concentra en qué hace el software y no en cómo lo hace. Se obtienen los casos de prueba directamente de las especificaciones.
  Técnicas::
    - Particionamiento de equivalencia: dividir un conjunto de condiciones de prueba conjuntos que puedan ser considerados iguales por el sistema. Requiere probar sólo una condición para cada partición, se asume que todas las condiciones de una partición serán tratadas de la misma manera por el software.
    - Análisis de valor frontera: considera que la mayoría de los errores se encuentran en las fronteras de las particiones. Hay fronteras válidas e inválidas.
    - Tablas de decisión: Cuando existen diversas combinaciones de entrada, el efecto global de las condiciones lógicas puede ser muy complejo, se debe asegurar que todas las combinaciones de las condiciones que puedan ocurrir

han de ser probadas.
Una tabla de decisión lista todas las condiciones de entrada que pueden ocurrir y todas las acciones que pueden surgir de ellas. Cada columna, por lo tanto, representa una regla de negocio individual y muestra cómo se combinan las condiciones de entrada para producir acciones.

- Transición de estados: La técnica de transición de estados se utiliza con sistemas en los que las salidas son desencadenadas por cambios en las condiciones de entrada, o cambios de "estado". Se usan cuando alguno de los aspectos del sistema se pueden describir en lo que se denomina una "máquina de estados finitos". Cualquier sistema en el que se puedan conseguir diferentes salidas con la misma entrada, dependiendo de lo que haya pasado antes, es un sistema de estados finitos.
- Pruebas de Casos de uso: Las pruebas de caso de uso son una técnica que ayuda a identificar casos de prueba que ejerciten el sistema entero transición a transición desde el principio al final.
  Los casos de uso sirven como fundamento para desarrollar casos de prueba, en su mayoría, en los niveles de pruebas de sistema y pruebas de aceptación.
  Los casos de uso pueden descubrir defectos de integración, defectos causados por la interacción incorrecta entre diferentes componentes.

- Basadas en la estructura: técnicas de pruebas de caja blanca, las pruebas se derivan a partir del conocimiento de la estructura e implementación del software. La comprensión del algoritmo utilizado en un componente puede ayudar a identificar particiones adicionales y casos de prueba, asegurando así un mayor rango de pruebas.
  Técnicas:
  - Pruebas de sentencia: busca probar las distintas sentencias ejecutables a lo largo del código.
    Se genera una medida de cobertura a partir de la cantidad de sentencias probadas sobre ejecutables.
  - Pruebas de decisión: busca asegurar que las decisiones en un programa son realizadas adecuadamente. Las decisiones son parte de las estructuras de selección e iteración, garantiza que todas las posibles salidas de la decisión se han probado.
    Se genera una medida de cobertura.
  - Pruebas de caminos: estrategia de pruebas estructurales cuyo objetivo es probar cada camino de la ejecución independientemente, todas las sentencias condicionales se comprueban los casos verdadero y falso.
    En un proceso de desarrollo orientado a objetos, pueden utilizarse las pruebas de caminos cuando se prueban los métodos asociados a los objetos. No es posible probar todas las posibles combinaciones de todos los caminos en el programa. Existe un número infinito de posibles combinaciones de caminos en los programas con bucles. Aunque todas las sentencias del programa se han ejecutado, no asegura que no haya defectos en determinados caminos.
- Basadas en la experiencia: se usan cuando no hay una especificación adecuada desde la que crear casos de prueba ni tiempo suficiente para ejecutar la estructura completa del paquete de pruebas. Usan la experiencia de los usuarios y de los

técnicos de pruebas para verificar la consistencia en las áreas más importantes del sistema.
El conocimiento y experiencia de los técnicos de pruebas son factores decisivos para el éxito de las mismas.

Técnicas de control estáticas

Permiten encontrar las causas de los defectos, permiten una mejora del proceso evitando cometer errores similares en el futuro.
El objetivo de las pruebas estáticas es mejorar la calidad de los productos software, ayudando a los ingenieros a reconocer y arreglar sus propios defectos en etapas tempranas del proceso de desarrollo.
No sustituyen a las pruebas dinámicas.
Estas pruebas, son las primeras que se aplican al software y buscan defectos sin ejecutar código. Por esta razón, también se llaman “técnicas de no ejecución”. La mayoría de las técnicas estáticas son técnicas de verificación que pueden probar cualquier tipo de documentación ya sea código fuente, o documentación y modelos de diseño, o especificación funcional o de requisitos.
Ventajas:

- El esfuerzo de realizar retrabajo se reduce sustancialmente, mientras que la productividad de desarrollo probablemente aumente.
- La evaluación realizada por un equipo tiene la ventaja adicional de que hay un intercambio de información entre participantes.
- Las pruebas estáticas contribuyen a un aumento en la conciencia sobre temas de calidad.

Técnicas:

- Revisiones: consiste en realizar un análisis de un documento con el objetivo de encontrar y eliminar errores. Ayuda a encontrar defectos en la documentación antes de que llegue al código ejecutable, en las primeras fases del ciclo de vida del producto.
  El tipo de revisión varía en función del nivel de formalidad, es decir, la estructura y la documentación asociada a la revisión.
  - Informal: Dar un borrador de un documento a un colega para que lo lea, tiene un bajo costo. Puede implementarse con Pair Programming.
  - Walkthrough: el autor del documento bajo revisión va guiando al resto de participantes a través del documento exponiendo sus ideas para conseguir un entendimiento común y recoger respuestas. a reunión es dirigida por los autores y a menudo está presente un documentador, y para facilitar su ejecución pueden usarse escenarios y simulaciones para validar el contenido.
  - Revisión técnica: una reunión que se centra en conseguir consenso sobre el contenido técnico de un documento, normalmente dirigida por un experto técnico. El objetivo es evaluar el valor de conceptos técnicos y alternativas del entorno del proyecto y del propio producto.
    Puede implementarse con Peer Reviews.
  - Inspección: es el tipo de revisión más formal, el documento bajo inspección es preparado y validado minuciosamente por revisores antes de la reunión, se registran los defectos encontrados en la lista de registro y se hace una

fase de discusión.
La inspección es dirigida normalmente por un moderador formado.

Roles y responsabilidades: aplican para las revisiones formales.

- Moderador: dirige el proceso de revisión, determina el tipo de revisión que se va a realizar y la composición del equipo de trabajo.
- Autor: persona que ha creado el documento bajo revisión, busca mejorar la calidad del documento y su propia habilidad para escribir futuros documentos.
- Documentador: toma nota de cada defecto mencionado y de cualquier sugerencia acerca de la mejora del proceso.
- Revisor: valida el material para encontrar defectos antes de realizar la propia reunión de revisión.
- Supervisor: determina si se han cumplido los objetivos de la revisión. Es quien decide ejecutarla.

Factores críticos de una revisión:

- Cada revisión debería tener un objetivo claro y una persona que asegurarse que el objetivo se cumple.
- Seleccionar los documentos más críticos para ser parte de una revisión.
- Se debe seleccionar un tipo de revisión que encaje con el objetivo definido, se debe informar a los ingenieros sobre el mismo para que se puedan preparar con anticipación.
- Planificar y gestionar las revisión haciendo seguimiento del tiempo invertido en ellas.
- Brindar capacitación sobre técnicas de revisión.
- Establecer tanto formalidad como la cultura del proyecto y el nivel de madurez lo permita.
- Motivación a los ingenieros involucrados.
- Informar los resultados de la revisión lo antes posible.

- Análisis estático: busca defectos sin ejecutar el código pero se lleva a cabo una vez que se escribe el código. Su objetivo es encontrar defectos en el código fuente y en los modelos del software.
  Pueden utilizar analizadores estáticos, que son herramientas de software para procesar código fuente.
  Las inspecciones de código son un pilar fundamental para la construcción de un software de calidad.
  Ventajas:
  - No hay interacción entre errores como en las pruebas dinámicas.
  - Pueden inspeccionarse versiones incompletas de un sistema sin costes adicionales.
  - Una inspección también puede considerar atributos de calidad más amplios de un programa tales como grado de cumplimiento con los estándares, portabilidad y mantenibilidad.

  Existen distintos tipos de análisis sintáctico:
  - Análisis de flujo de control: Comprueba los bucles con múltiples puntos de entrada o salida, encuentra códigos inalcanzables.
  - Análisis de uso de los datos: Detecta variables no inicializadas, variables escritas dos veces sin que intervenga una asignación, variables que se declaran pero nunca se usan.

    - Análisis de interfaz: Comprueba la consistencia de una rutina, las declaraciones del procedimiento y su uso.
    - Análisis de flujo de información: Identifica las dependencias de las variables de salida. No detecta anomalías en sí pero resalta información para la inspección o revisión del código.
    - Análisis de caminos: Identifica los caminos del programa y arregla las sentencias ejecutadas en el camino. Es potencialmente útil en el proceso de revisión.

Herramientas de pruebas

Beneficios

- Reducir el trabajo repetitivo
- Mejorar la consistencia
- Facilitar las evaluaciones objetivas
- Facilitar el acceso a información relacionada con pruebas

Riesgos

Introducir algo nuevo en una organización conlleva que va a haber problemas técnicos a los que sobreponerse.
Es necesario invertir tiempo para desarrollar métodos para usar la herramienta.
Se debe tener en cuenta la habilidad y conocimiento necesario para realizar buenas pruebas y para usar las herramientas de forma correcta.

- Expectativas poco realistas de la herramienta.
- Subestimación del tiempo, coste y esfuerzo necesario para la introducción inicial de la herramienta en la organización.
- Subestimación del tiempo y esfuerzo necesario para conseguir beneficios significativos y continuos de la herramienta.
- Subestimación del esfuerzo requerido para mantener los activos de pruebas generados por la herramienta.
- Sobre-confianza en la herramienta.

Para introducir una herramienta a una organización, es necesario trabajar primero sobre la organización, debe estar preparada para los cambios que implica la nueva herramienta.
Factores a considerar:

- Evaluar la madurez de la organización
- Identificar las áreas de la organización donde las herramientas puedan ayudar y dar soporte a los procesos de prueba
- Evaluar herramientas contra requisitos claros y criterios objetivos
- Planificar la implementación interna de la herramienta
- Llevar a cabo una prueba de concepto (proyecto piloto) para comprobar si el producto funciona como se espera.

Clasificación

Las herramientas están clasificadas dependiendo de las áreas o actividades de pruebas a las que se enfocan.

- Herramientas de soporte para la gestión de pruebas: la gestión de pruebas se aplica desde el comienzo del ciclo de vida, deberían ser las primera en usarse.
    - Herramientas de gestión de pruebas: ayudan a recoger, organizar y comunicar información sobre las pruebas en un proyecto.
    - Herramientas de gestión de requisitos: son útiles para las pruebas dado que, cuanto mayor sea la calidad de los requisitos, más fácil será desarrollar sus pruebas. También sirven para mantener una trazabilidad entre los requisitos y las pruebas.
    - Herramientas de gestión de incidencias: facilitan el mantenimiento del registro de incidencias a lo largo del tiempo. Permiten identificar defectos, problemas, anomalías o posibles mejoras.
    - Herramientas de gestión de configuración: no son estrictamente herramientas de pruebas pero una buena gestión de configuración es crítica para controlar las pruebas. Es necesario conocer exactamente qué debe ser probado así como la versión exacta de todos los componentes del sistema.
- Herramientas de soporte para pruebas estáticas: dan soporte a las actividades de las pruebas estáticas.
    - Herramientas de revisión de procesos: útiles para revisiones formales de muchos participantes o participantes distribuidos geográficamente. Facilitan mucho el trabajo de recogida de información del proceso de pruebas.
    - Herramientas de análisis estático: son utilizadas por desarrolladores como parte del proceso de pruebas, el código no se ejecuta, sino que sirve de elemento o dato de entrada a la herramienta. Calculan métricas, imponen estándares de codificación, analizan estructuras y dependencias, identifican anomalías en el código, entre otras. Pueden utilizarse antes de la ejecución de las pruebas dinámicas.
    - Herramientas de modelado: ayudan a validar modelos del sistema. Son de gran utilidad en el diseño de software. Pueden utilizarse antes de la ejecución de las pruebas dinámicas.
- Herramientas de soporte para la especificación de pruebas: dan soporte a actividades de pruebas tales como el análisis de pruebas, el diseño de pruebas o la implementación de pruebas.
    - Herramientas de diseño de pruebas: pueden identificar de manera rápida y sencilla las pruebas a ejecutar sobre todos los elementos del sistema, son de gran utilidad a la hora de comenzar con el diseño de pruebas.
    - Herramientas de preparación de datos de pruebas: sirven para establecer datos de pruebas. Son especialmente útiles en las pruebas de rendimiento, donde es imprescindible trabajar con gran cantidad de datos realistas.
- Herramientas para ejecución y registro de pruebas:
    - Herramientas de ejecución de pruebas: ofrecen un mecanismo para la captura y registro de pruebas. Suelen utilizar un lenguaje de scripting, por lo que si los técnicos de pruebas desean utilizar una herramienta de ejecución de pruebas necesitarán tener habilidades de programación sobre la creación y modificación de scripts.
    - Comparadores de pruebas: ayudan a comparar automáticamente lo que el software produce y lo que debería producir.

    - Herramientas de seguridad: se utilizan para probar la seguridad que tienen los sistemas, intentan acceder al sistema sin los permisos necesarios, identifican virus, simular ataques externos, detectar intrusos.
- Herramientas de monitorización y rendimiento: pueden llevarse a cabo durante el propio proceso de pruebas o después de la liberación del sistema.
    - Herramientas de análisis dinámico: requieren que el código se esté ejecutando, no ejecutan pruebas sino que analizan lo que ocurre cuando el software se está ejecutando.
    - Herramientas de monitorización: se utilizan para realizar un seguimiento del estado del sistema en uso, se busca detectar problemas tempranamente y mejorar el servicio.

Unidad 6: Gestión de Configuración

Conceptos

- Configuración del software: conjunto de características funcionales y físicas del software detalladas en la documentación técnica o alcanzadas en un producto.
- Gestión de la configuración: proceso cuyo propósito es establecer y mantener la integridad de los productos de trabajo a través de:
    - La identificación de los elementos/productos que van a ser controlados
    - La definición de un procedimiento para el control de los productos
    - El registro/informe del estado de los productos
    - Las auditorías de configuración
- Elemento de configuración: cualquier producto de trabajo, tanto producto final como productos intermedios y tanto productos entregables al cliente como productos internos del proyecto, cuyo cambio pueda resultar crítico para el buen desarrollo del proyecto.
- Línea base: conjunto de elementos de configuración formalmente designados y fijados en un momento específico del ciclo de vida., tienen que estar acabados y formalmente aprobados. Sólo puede ser modificada a través de un procedimiento formal de cambios. La línea base, junto con todos los cambios aprobados sobre la misma, representa la configuración vigente y aprobada.

El objetivo de la gestión de configuración es asegurar la integridad de los productos y servicios desarrollados.
Integridad del producto:

- Saber exactamente lo que se ha entregado al cliente
- Saber el estado y contenido de las líneas base y elementos de configuración

La gestión de la configuración es una forma efectiva y eficiente de gestionar y comunicar los cambios en líneas base y elementos de configuración a lo largo del ciclo de vida.
Beneficios:

- Asegurar la correcta configuración del software.
- Proporcionar la capacidad de controlar los cambios.
- Reducir los sobreesfuerzos causados por los problemas de integridad.
- Garantizar que todo el equipo trabaja sobre una misma línea base de productos.

Problemas que pueden presentarse si no se hace gestión de configuración:

- Versión con errores
- Versión con cambios que no han sido probados
- Versión que no puede reproducirse. No se puede recuperar una línea base anterior para realizar mantenimiento.
- No saber qué versión tiene implementada el cliente
- No tener un ambiente para reproducir un problema
- No saber si un arreglo está implementado en un ambiente o no
- No disponer un inventario completo de los componentes del sistema
- Retrabajo porque las componentes probados no eran lo que debían ir en la versión

Roles y responsabilidades

- Gestor de configuración

    - Gestionar la planificación, identificación, control, seguimiento y auditoría de todos los elementos de configuración en la base de datos de configuración.
    - Desarrollar el plan de gestión de configuración.
    - Promover el uso efectivo de la base de datos de configuración dentro de la organización.
    - Monitorizar y reportar los cambios no autorizados sobre los elementos de configuración.
    - Asegurar la consistencia e integridad de los datos de la base de datos de configuración a
    - través de la ejecución de procedimientos de verificación y auditoría.
    - Liderar las actividades de evaluación del proceso: revisar tipos de elementos de configuración, relaciones, atributos y valores asociados, estructura de la base de datos, derechos de acceso.
    - Aprobar cambios estructurales en la base de datos de configuración.
- Coordinador de configuración
    - Asegurar que todos los elementos de configuración están registrados de forma adecuada en la base de datos de configuración.
    - Asegurar la consistencia e integridad de los datos de la base de datos de configuración y la estructura del sistema a través de la ejecución de procedimientos de verificación y auditoría.
    - Reportar cualquier discrepancia o no conformidad en los elementos de configuración al gestor de configuración.
    - Participar en la mejora continua del proceso de gestión de configuración.
- Responsable de elementos de configuración
    - Asegurar que los elementos de configuración de los que es responsable están registrados en la base de datos de configuración con el estado y datos de configuración apropiados.
    - Verificar que los cambios sobre los elementos de configuración siguen el proceso de cambios definido.
    - Asegurar la idoneidad e integridad de los elementos de configuración de los que es responsable.
    - Trabajar conjuntamente con el gestor de configuración para identificar las causas de cualquier discrepancia identificada en las auditorías e implementar las acciones correctivas.
- Gestor de cambio
    - Evaluar el impacto y riesgo de los cambios.
    - Asegurar que los responsables de los elementos de configuración actualizan los históricos de estos elementos con los cambios implementados.

Actividades

- Gestión del proceso de gestión de configuración
    - Rol responsable: Gestor de configuración.
    - Descripción: Documentar el plan de gestión de configuración..
    - Entradas: Necesidades del proyecto, Plan de proyecto.
    - Salidas: Plan de gestión de configuración aprobado.
- Identificación de elementos de configuración
    - Rol responsable: Gestor de configuración.

    - Descripción: Identificar elementos de configuración. Crear estructura del directorio de gestión de configuración.
    - Entradas: Productos del proyecto.
    - Salidas: Elementos de configuración identificados, Línea base, Estructura del directorio de gestión de configuración.
- Mantenimiento y control de la gestión de configuración
    - Rol responsable: Responsable del elemento de configuración.
    - Descripción: Control de cambios sobre elementos de configuración y líneas base. Obtener aprobación de solicitudes de cambio sobre productos de trabajo de línea base.
    - Entradas: Peticiones de cambio.
    - Salidas: Registro de solicitud de cambio, Solicitud de cambio aprobada, Línea base.
- Informe de estado de la configuración
    - Rol responsable: Gestor de configuración.
    - Descripción: Mantener actualizado y publicar el estado de los elementos de configuración.
    - Entradas: Elementos de configuración.
    - Salidas: Informe de estado de elementos de configuración
- Verificación y auditoría
    - Rol responsable: Gestor de configuración.
    - Descripción: Realizar auditorías de la gestión de configuración.
    - Entradas: Registros de la gestión de configuración, Línea base, Registros de cambios
    - Salidas: Informe de auditoría de gestión de configuración
- Gestión de la liberación del software

Identificar elementos de Configuración

Los elementos de configuración son productos de trabajo que son generados durante el proyecto y probablemente serán modificados durante el curso del ciclo de vida del proyecto. Pueden ser entregables internos, externos, finales o intermedios.
Ejemplos:
Planes (plan del proyecto, plan de calidad, plan de gestión de la configuración, plan de gestión de riesgos...)

- Registros del proyecto.
- Material de apoyo al cliente.
- Especificación de requisitos (requisitos de negocio, de usuario o de sistema).
- Matriz de trazabilidad de requisitos.
- Documentos de diseño.
- Resultados de la resolución y análisis de decisión.
- Código fuente.
- Plan de integración de software.
- Informes resultantes de las revisiones realizadas en los puntos de comprobación o al final de las fases.
- Plan de pruebas (unitarias, de integración, de sistemas, de aceptación de usuario, de regresión...)
- Datos de pruebas y casos de pruebas.

- Plan de instalación/mantenimiento.
- Documentos de manual de usuario.
- Plan de entrega de servicios.
- Informes de investigación.
- Informes de estimación.
- Informes de cierre del proyecto.
- Prototipos.
- Informes de métricas.

Mecanismo de construcción

Durante la construcción del producto es recomendable tener en cuenta:

- Línea base del software utilizado para desarrollar los entregables.
- Personas que autorizan la creación de productos desde la línea base.
- Personas responsables de la construcción del producto.
- Herramientas y entorno de desarrollo.
- Pasos que deben seguirse para el desarrollo del producto.
- Pasos de validación para asegurar las versiones correctas del software consideradas en la construcción.

Mecanismo de liberación

El procedimiento de liberación debería cubrir los siguientes elementos:

- Criterios de liberación
- Contenido de liberación
- Notas de liberación
- Procedimientos de empaquetar y validar el empaquetado.
- Procedimientos de liberación y aceptación de la liberación.
- Referencia a procedimientos de pruebas cuando sea necesario.

Es recomendable el uso de parches y llevar a cabo actividades de liberación del producto de forma periódica.

Branching and merging

Al desarrollar un proyecto nuevo, debería crearse una nueva rama cuyo origen sería la rama principal, es decir, el branch del software. Los cambios realizados sobre la rama principal del proyecto durante la fase de desarrollo serán incorporados en las ramas temporales del proyecto.

El merging es un método por el que los cambios realizados de forma remota son incorporados al proyecto. Por ejemplo, los cambios realizados sobre una rama (un branch) del código fuente de un proyecto, pueden incorporarse a la rama principal utilizando técnicas de merging.

Auditoría de gestión de configuración

Planificar las auditorías de configuración decidiendo cuándo llevarlas a cabo en función de la naturaleza del proyecto y de los requisitos. El objetivo de las auditorías de gestión de configuración es asegurarse de:

- Los elementos de configuración se encuentren en el directorio apropiado.

- Consistencia del estado actual de los elementos de configuración.
- La información de línea base se mantenga de forma correcta.
- Verificar la conformidad con estándares y procedimientos aplicables a la gestión de configuración, por ejemplo, comprobar si se usa la versión correcta del documento de diseño para realizar la codificación.
- Después de una exitosa auditoría de configuración establecer una línea base del producto.
- El representante del equipo de pruebas está incluido en el equipo de auditorías.

Unidad 7: Proceso de aseguramiento de calidad

Introducción

Objetivo: Evaluar de forma objetiva los procesos, productos de trabajo y servicios para asegurar la calidad de los procesos, productos y servicios.
Alcance: Este proceso es aplicable a todos los tipos de proyectos a lo largo de su ciclo de vida.
Criterios de entrada: Se ha realizado el arranque del proyecto.
Entradas:

- Propuesta de trabajo
- Contrato
- Plan de proyecto
- Repositorio de la organización

Registros de calidad:

- Checklist de auditoría de QA.
- Plan de QA.

Proceso

La función del aseguramiento de la calidad se lleva a cabo en la organización por el equipo de calidad. Este equipo realizará auditorías de la ejecución de los procesos en el proyecto de forma periódica.
Las actividades de QA se ejecutan en las siguientes fases del proyecto:

- Inicio del proyecto
- Planificación del proyecto
- Seguimiento y control de la ejecución del proyecto
- Cierre del proyecto

Inicio/planificación del proyecto

- Antes del inicio del proyecto, se identifica a una persona del equipo de QA que va a desempeñar el rol de SQA.
- El SQA colabora con el gerente/jefe de proyecto para planificar las actividades de QA en el proyecto, basándose en la propuesta de trabajo, contrato y plan de proyecto.
- Las actividades de QA deben documentarse en el EDT (Work Breakdown Structure) del proyecto y en el calendario del proyecto.
- El SQA colabora con el jefe de proyecto para identificar buenas prácticas y lecciones aprendidas de proyectos similares que se pueden ejecutar al inicio del proyecto.
- El SQA ayuda al jefe de proyecto a elegir el ciclo de vida más adecuado para el proyecto.
- El SQA ayuda al jefe de proyecto a adaptar los procesos para el proyecto y reflejarlo en el plan de proyecto.
- El jefe de proyecto prepara el plan de calidad para el proyecto. El SQA lo revisa.
- El SQA revisa el plan de proyecto, el plan de gestión de riesgos y el plan de gestión de configuración del proyecto.

- El SQA asegura que los miembros del equipo de proyecto conocen los procesos definidos.

Ejecución del proyecto

Durante la ejecución del proyecto, se llevan a cabo las siguientes actividades de QA:

- El SQA asegura la revisión de los productos generados en el proyecto según se ha definido en el plan de calidad.
- El SQA ayuda al proyecto en la prevención de defectos y el análisis de causas de los problemas.
- El SQA verifica que las métricas del proyecto que se van a incorporar a la base de datos de la organización son completas, correctas, etc.
- Auditorías de QA:
    - El SQA realiza auditorías periódicas al proyecto para asegurar la conformidad con los procesos.
    - El SQA realiza auditorías de configuración periódicas y realiza un seguimiento de las no conformidades hasta su cierre.
    - El SQA registra todos los hallazgos de las auditorías en un informe que envía al jefe de proyecto y al responsable de QA.
- El SQA escala cualquier problema, incluidas las no conformidades que no se han podido resolver en el proyecto, al responsable de QA y al gerente del proyecto. El responsable de QA, junto con el gerente del proyecto, intentarán resolver los conflictos.
- Seguimiento de actividades de QA:
    - El SQA registra la implementación de las actividades de QA planificadas para el proyecto. El SQA presenta el estado planificado y real, en cuanto a calendario y esfuerzo, de las actividades de QA.

Cierre del proyecto

El SQA asegura que, al cierre del proyecto, se genera un informe retrospectivo y se envía al histórico de la organización. Además, se asegura que cualquier mejora propuesta sobre los procesos es tenida en cuenta.

Reuniones de SQA

El responsable de QA mantiene reuniones con los SQA al menos una vez al mes para revisar:

- Mejores prácticas implementadas en los proyectos.
- Cualquier problema encontrado en la implementación de los procesos definidos.
- Posibles mejoras sobre los procesos.

Métricas

- Esfuerzo dedicado a actividades de QA (planificado vs. real)
- Número de no conformidades detectadas en los proyectos
- Número de no conformidades solucionadas
- Número de no conformidades por tipo
- Número de no conformidades por área de proceso

- Tiempo medio de solución
- Número de auditorías realizadas al año
- Número de proyectos diferentes revisados al año

Unidad 8: Métricas

Conceptos

Cuando puedes medir aquello de lo que estás hablando, y expresarlo en números, sabes algo sobre ello; pero cuando no lo puedes medir, y no lo puedes expresar en números, tu conocimiento acerca de ello es insatisfactorio
No se puede controlar lo que no se puede medir
Objetivos de la medición:

- Entender qué ocurre durante el desarrollo y el mantenimiento del software
- Controlar qué es lo que ocurre en nuestros proyectos
- Mejorar nuestros procesos y productos

La medición de software es una disciplina relativamente joven y no existe consenso general sobre la definición exacta de los conceptos y terminología que maneja.
Entidad:

- Objeto que va a ser caracterizado mediante una medición de sus atributos.
- Una categoría de entidad es una colección de entidades
- Una entidad puede ser un proceso, un producto, un servicio, un proyecto, o un recurso concreto.

Atributo: Un atributo es una propiedad medible, física o abstracta, que comparten todas las entidades de una categoría de entidad.

Métrica y medida

Métrica: una medida cuantitativa de un atributo determinado de una entidad. Medida es el valor de la métrica.
Ejemplo métrica: líneas de código
Ejemplo medida: 10000 líneas de código
Métricas directas:

- Es una métrica de un atributo que no depende de ninguna métrica de otro atributo
- La forma de medir una métrica directa es un método de medición.

Métricas indirectas:

- Es una métrica de un atributo que se deriva de una o más métricas de otros atributos.
- La forma de medir una métrica indirecta es una función de cálculo.

Indicadores:

- Un indicador es una métrica o una combinación de métricas de un proyecto de software junto con criterios de decisión.

Definición de las métricas:

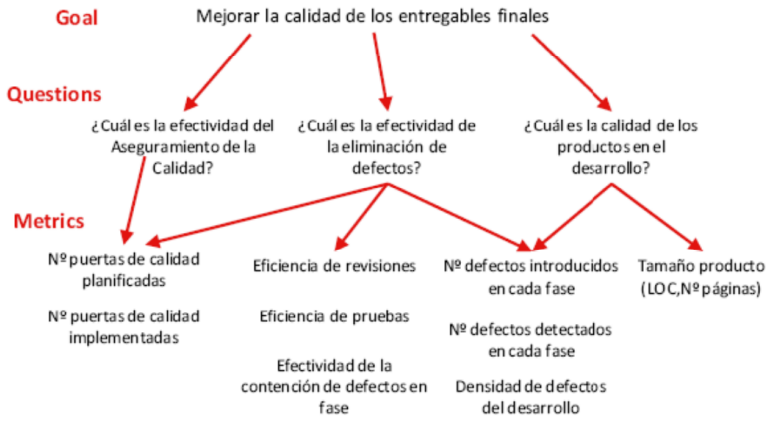
- Problemas en la definición de las métricas
    - Contexto: Las definiciones de métricas no siempre tienen en cuenta el entorno o contexto en el que serán aplicadas.
    - Validación: No siempre es posible realizar una validación teórica adecuada de la métrica porque el atributo que queremos medir no siempre está bien definido
- Estos problemas originan:
    - Ambigüedad en las definiciones

    - Difícil uso de las mismas
    - Interpretación peligrosa – Resultados contradictorios.
- Para evitarlo es necesario contar con un método de definición de métricas

GQM

El enfoque GQM (Goal-Question-Metric) (Objetivo-Preguntas-Métricas):

- Proporciona una manera útil para definir métricas.
- Según este método, un programa de medición puede ser más satisfactorio si se diseña orientado a los objetivos que se quieren alcanzar.
- Pasos:
    - Define una meta
    - Refina esta meta en preguntas
        - Las preguntas ayudarán a medir si se está alcanzando la meta definida (preguntas que sean potencialmente medibles)
    - Define métricas que intentan dar información para responder a estas preguntas.



Niveles

- Nivel Conceptual – Goals:
    - Los objetivos identifican lo que queremos lograr respecto a los productos, procesos o recursos.
    - Objetos de la medición:
        - Productos: entregables y documentos que se producen durante el ciclo de vida de un sistema.
        - Procesos: actividades relacionadas con el software y asociadas generalmente al tiempo.
        - Recursos: elementos (personal, materiales, herramientas y métodos) que los procesos utilizan para producir sus salidas
- Nivel Operacional – Questions:
    - Las preguntas nos ayudan a comprender cómo satisfacer el objetivo.
- Nivel Cuantitativo – Metrics:
    - Se asocia un conjunto de métricas a cada pregunta, con el fin de proporcionar una respuesta de manera cuantitativa.

    - Para cada pregunta puede haber múltiples métricas y una métrica puede ser aplicable a más de una pregunta.

PSM

PSM (Practical Software Measurement):

- El primer paso del proceso de definición de métricas:
    - Identificar las necesidades de información de la organización
- Para que una métrica tenga éxito y ayude a cumplir los objetivos del proyecto en un entorno cambiante hay que tener muy en cuenta las necesidades de información
- Según PSM las necesidades de información se clasificación en siete categorías.
    - Calendario y progreso
        - Hitos del proyecto
        - Completitud de unidades de trabajo individuales:
            - Un proyecto que se sale de la agenda, normalmente puede cumplir sus objetivos de entrega sólo eliminando funcionalidades o sacrificando la calidad del producto.
    - Recursos y costes
        - Equilibrio entre el trabajo a realizar y los recursos de personal asignado al proyecto
        - Un proyecto que exceda el esfuerzo presupuestado normalmente puede recuperarse sólo reduciendo la funcionalidad del software o sacrificando la calidad del producto.
    - Tamaño de producto y estabilidad
        - Un aumento en el tamaño del software normalmente requiere aumentar los recursos aplicados o la agenda del proyecto.
    - Calidad de producto
        - La capacidad del producto software entregado para dar soporte a las necesidades del usuario sin fallar.
        - Si se entrega un producto de baja calidad, la carga de hacerlo funcionar normalmente recae sobre la organización de mantenimiento asignada.
    - Ejecución de proceso
        - La capacidad de la organización del proceso de desarrollo de software
        - Una organización con un proceso de desarrollo de software pobre o de baja productividad puede tener dificultad al cumplir los objetivos de coste y la programación del proyecto planificada.
    - Efectividad de la tecnología
        - La viabilidad del enfoque técnico propuesto.
        - Trata enfoques de ingeniería como:
            - Reutilización de software
            - Utilización de componentes software comerciales
            - Implementación de arquitecturas de software.
        - Si no se consiguen los elementos clave del enfoque técnico propuesto puede resultar en aumentos de costes y retrasos en la agenda.
    - Satisfacción del cliente:

        - Se ocupa del grado en que los productos o servicios entregados por el proyecto cumplen las expectativas del cliente.
        - Los indicadores de satisfacción pueden obtenerse de la retroalimentación del cliente y de los niveles que sean requeridos el soporte de clientes.

Clasificación de las métricas

Métricas de proyecto

Describen las características propias del proyecto y de su ejecución (el tamaño, esfuerzo, etc.).
Las métricas del proyecto permiten al líder de proyectos del software:

- Seguir la pista de riesgos potenciales.
- Detectar las áreas de problemas antes de que se conviertan en críticas.
- Ajustar el flujo y las tareas del trabajo.
- Evaluar la habilidad del equipo del proyecto en controlar la calidad de los productos de trabajo del software.

Ejemplos:

- Planificación:
    - Progreso adecuado
    - Cumplimiento de tiempos
    - Cumplimiento de hitos
- Ejecución:
    - Cumplimiento de esfuerzos
    - Cumplimiento de costes
    - Cumplimiento de alcance

Las métricas a nivel de proyecto, también pueden utilizarse a nivel de proceso.

Métricas a nivel de proceso

La única forma de mejorar cualquier proceso es:

- Medir sus atributos
- Desarrollar un conjunto de métricas significativas en función de estos atributos
- Finalmente utilizar estas métricas para proporcionar indicadores que conducirán a una estrategia de mejora.

La efectividad de un proceso de desarrollo de software se mide indirectamente, se extrae un conjunto de métricas según los resultados que provienen de la instanciación del proceso.
A partir de esos datos consolidados de distintos proyectos se puede tener una visión del comportamiento del proceso.
Métricas del proceso de pruebas:

- Las pruebas son un proceso dentro de la organización, y como tal, se pueden tomar métricas para gestionar el estado de la calidad de ese proceso.
- Una vez que se desarrolla un plan de pruebas, las actividades y planificación tienen que ser revisados contra lo que realmente se está realizando.
- Ejemplos:
    - Porcentaje de trabajo realizado en la preparación de los casos de prueba.

    - Porcentaje de trabajo realizado en la preparación del entorno de pruebas respecto a lo planificado.
    - Ejecución de casos de pruebas (p.ej.: número de casos de prueba ejecutados/no ejecutados, y casos de prueba pasados/fallados).
    - Información de defectos (p.ej.: número de defectos/PF, defectos encontrados y corregidos)

Métricas a nivel de producto

La calidad del desarrollo de los productos software, debe basarse en la satisfacción de las expectativas de sus usuarios finales y se deberá asegurar que los productos cumplen:

- Con los requisitos y atributos de calidad especificados para los mismos.
- La documentación desarrollada será útil para el usuario: completa, correcta, fácil de utilizar y mantener.
- Las aplicaciones desarrolladas deberán ser correctas, fiables, usables y fáciles de mantener y evolucionar.

Métricas a nivel de servicio

Proporcionar un servicio de calidad tanto en lo relativo:

- A la explotación de los sistemas (conjunto de plataformas hardware, software y comunicaciones sobre las cuales se construyen y explotan las aplicaciones de usuario)
- Al mantenimiento de las aplicaciones (productos software que resuelven las necesidades del usuario final)

Técnicas de análisis

Un plan de análisis debe definir:

- Cómo se van a organizar los datos
- Qué métricas o datos se van a analizar
- Los métodos que se van a utilizar para analizarlos
- Qué objetivos se pretenden satisfacer con el análisis o qué información se pretende obtener del análisis
- A quién se va a presentar y cómo

Métodos de análisis:

- Las herramientas estadísticas promovidas por Ishikawa para el control de la calidad son ampliamente utilizadas y se han convertido en una parte importante dentro del control de la calidad.
- Se las conoce como las siete herramientas básicas de Ishikawa y se suelen utilizar para analizar las métricas de software.

Técnicas de análisis (siete herramientas básicas de Ishikawa):

- Checklist
- Diagrama de Pareto
- Histograma
- Diagrama de dispersión
- Gráfico de ejecución
- Gráfico de control

- Diagrama de causa-efecto

Proceso de Medición y Análisis

PSM (Practical Software and System Measurement):

- Es un proceso de medición desarrollado para cumplir los objetivos de negocio y técnicos de una organización.
- El proceso de medición es un proceso flexible que se puede adaptar para satisfacer las necesidades de información y los objetivos:
    - A nivel organizacional
    - A nivel de proyectos específicos

Unidad 9: Análisis Forense digital

Introducción

Análisis forense digital: conjunto de procedimientos de recopilación y análisis de evidencias para responder a incidentes relacionados con la seguridad informática y que sirven como pruebas ante un tribunal.
Usos:

- Persecución de delitos como fraude financiero, evasión de impuestos, acoso o pornografía infantil.
- Casos de discriminación o acoso.
- Investigación de seguros.
- Recuperación de ficheros eliminados.
- Casos de robo de la propiedad intelectual.
- Ciberterrorismo.
- Asegurar la resiliencia de las empresas, es decir, la capacidad de recuperación frente a ataques.

Se tienen pautas que se deben seguir a la hora de realizar un análisis forense digital con el fin de no destruir pruebas que imposibiliten resolver el incidente de una manera satisfactoria.
Un incidente es resuelto de manera satisfactoria cuando se extraen conclusiones que permiten responder a las preguntas anteriormente planteadas.
Principio de Locard: “siempre que dos objetos entran en contacto transfieren parte del material que incorporan al otro objeto”: Esto significa que cualquier tipo de delito deja un rastro por lo que mediante el proceso de análisis forense se pueden obtener evidencias.
Tipos de análisis forense:

- Análisis forense de sistemas: tanto sistemas operativos Windows, como OSX, GNU/Linux, etc.
- Análisis forense de redes.
- Análisis forense de sistemas embebidos.
- Análisis forense de memoria volátil

Características del procedimiento:

- Verificable: se debe poder comprobar la veracidad de las conclusiones extraídas a partir de la realización del análisis.
- Reproducible: se deben poder reproducir en todo momento las pruebas realizadas durante el proceso.
- Documentado: todo el proceso debe estar correctamente documentado y debe realizarse de manera comprensible y detallada.
- Independiente: las conclusiones obtenidas deben ser las mismas, independientemente de la persona que realice el proceso y de la metodología utilizada.

Fases (no son secuenciales, están entrelazadas):

- Preservación: se debe garantizar que no se pierdan las evidencias que deben ser recopiladas para su posterior análisis. Se rotulan los elementos, se mantiene un registro de lo que se va haciendo sobre el material a analizar. Se protege a los materiales de cualquier pérdida de información.

- Adquisición: se recopilan las evidencias (cualquier prueba que pueda ser utilizada en un proceso legal).
  Las evidencias deben ser:
    - Admisible: debe tener valor legal.
    - Auténtica: debe ser verídica y no haber sufrido manipulación alguna. Se usan hashes para asegurar la integridad.
    - Completa: debe tener toda la información desde un punto de vista objetivo y técnico, sin valoraciones personales, ni prejuicios.
    - Creíble: debe ser comprensible.
    - Confiable: las técnicas utilizadas no deben generar ninguna duda sobre su veracidad y autenticidad.

  Las evidencias se pueden clasificar como:
    - Evidencia física: hace referencia al material informático como por ejemplo: discos duros, pendrives, etc.
    - Evidencia digital: corresponde a la información almacenada en las evidencias electrónicas.
- Análisis: dependiendo del tipo de incidentes, se pueden analizar diferentes aspectos:
    - MFT o Master File Table: tabla que almacena información sobre los ficheros almacenados en el disco
    - Archivo de paginación (Pagefile.sys): fichero que tiene procesos en ejecución que actualmente no se necesitan en la memoria RAM.
    - Papelera de reciclaje.
    - Espacio no asignado: archivos de los que se eliminó la referencia que aún se conservan los datos en el disco sin referir.
    - Registro de Windows: guarda la actividad del sistema operativo, por ejemplo, las redes a las que se conectó, páginas visitadas, archivos usados recientemente, dispositivos conectados, etc.
    - Slack Space: hace referencia al espacio libre que queda en un clúster tras almacenar un fichero.
    - Tráfico de red.
    - Procesos del sistema.
    - Logs del sistema: por ejemplo el registro de sucesos relativos al sistema, a la seguridad o a las aplicaciones.
- Documentación: es un aspecto fundamental del proceso, se debe realizar de manera metódica y detallada.
    - Fotografiar las pruebas.
    - Cadena de custodia.
    - Documentar todos y cada uno de los pasos realizados durante el proceso, manteniendo una bitácora con fechas y horas de cada acción realizada sobre las evidencias.
    - Elaborar dos tipos de informe de conclusiones: uno ejecutivo y uno técnico
- Presentación: hace accesibles y comprensibles las conclusiones que se han obtenido del proceso del análisis forense.
    - Preparar una presentación de manera pedagógica que sea fácilmente comprensible.
    - Detallar las conclusiones.
    - Explicar de manera clara el proceso que se ha llevado para la obtención de las evidencias.

    - Evitar las afirmaciones no demostrables o los juicios de valor.
    - Elaborar las conclusiones desde un punto de vista objetivo

Tipologías de los incidentes

Para los diferentes tipos de incidentes, la toma de evidencias es común en la mayoría de los casos. Sin embargo, el análisis será específico dependiendo del tipo de incidente.

- Robo de información: robo de información privada con técnicas como monitorización del tráfico de red mediante un sniffer, intercepción de emails o simplemente mediante la inserción de un pendrive y la copia de información privada, etc. Principalmente, con un beneficio económico o dañar la imagen del afectado.
- Fraude: A lo largo del año se realizan un gran número de campañas, aprovechando fechas significativas o acontecimientos importantes, para propagar este tipo de amenazas.
    - Falsas ofertas de empleo
    - Estafas de loterías y sorteos
    - Falsas herencias.
    - Fraudes de inversiones y créditos.
    - Falsos correos para donaciones a alguna ONG
    - Falsas facturas de servicios de mensajería.
    - Descargas que facturan servicio de SMS Premium.
    - Multas por descargas ilegales.
- Malware: existen en gran volumen y complejidad.
- Accesos no autorizados: aprovechamiento de las vulnerabilidades del software para obtener privilegios y así poder acceder a carpetas o documentos de información confidencial..
- Uso inapropiado de recursos: para fines personales o no autorizados.
- Propiedad intelectual: vulneración de la propiedad intelectual.
- Denegación de servicio: impedir el acceso a los servicios y recursos de una organización, normalmente mediante el uso de botnets (redes de ordenadores infectados).

Principios de la recolección de evidencias

- Capturar una imagen del sistema tan precisa como sea posible.
- Realizar notas detalladas, incluyendo fechas y horas indicando si se utiliza horario local o UTC.
- Minimizar los cambios en la información que se está recolectando y eliminar los agentes externos que puedan hacerlo.
- En el caso de enfrentarse a un dilema entre recolección y análisis elegir primero recolección y después análisis.
- Recoger la información según el orden de volatilidad (de mayor a menor).
- Tener en cuenta que por cada dispositivo la recogida de información puede realizarse de distinta manera.

Orden de volatilidad

Es el período de tiempo en el que está accesible cierta información. Se debe recolectar en primer lugar aquella información que vaya a estar disponible durante el menor período de tiempo (mayor volatilidad).

- Registros y contenido de la caché.
- Tabla de enrutamiento, caché ARP, tabla de procesos, estadísticas del kernel, memoria.
- Información temporal del sistema.
- Disco.
- Logs del sistema.
- Configuración física y topología de la red.
- Documentos.

Acciones que deben evitarse

Para preservar la integridad de la información y no invalidar el proceso de recolección se debe evitar:

- No apagar el ordenador hasta que se haya recopilado toda la información volátil.
- No confiar en la información proporcionada por los programas del sistema ya que pueden haberse visto comprometidos. Se debe recopilar la información mediante programas desde un medio protegido como se explicará más adelante.
- No ejecutar programas que modifiquen la fecha y hora de acceso de todos los ficheros del sistema.

Consideraciones de privacidad

Toda la información recopilada durante el proceso debe ser tratada dentro del marco legal establecido, manteniendo la privacidad exigida. Los ficheros log están incluidos en este apartado ya que pueden almacenar patrones de comportamiento del usuario del equipo.

Consideraciones legales

Se debe tener presente que la legislación es diferente en cada país por lo que las evidencias pueden ser admitidas en un país y en otro no. Las evidencias deben ser Admisibles, Auténticas, Completas, Confiables, y Creíbles.

Procedimiento de recolección

El procedimiento de recolección debe de ser lo más detallado posible, procurando que no sea ambiguo y reduciendo al mínimo la toma de decisiones.

Reproducible

Los métodos utilizados para recolectar evidencias deben de ser transparentes y reproducibles.
Se debe estar preparado para reproducir con precisión los métodos usados, y que dichos métodos hayan sido testados por expertos independientes.

Pasos

- Listar qué sistemas están involucrados en el incidente y de cuáles de ellos se deben tomar evidencias.
- Establecer qué es relevante. En caso de duda es mejor recopilar mucha información que poca.
- Fijar el orden de volatilidad para cada sistema.
- Obtener la información de acuerdo al orden establecido.
- Comprobar el grado de sincronización del reloj del sistema.
- Según se vayan realizando los pasos de recolección preguntarse qué más puede ser
- una evidencia.
- Documentar cada paso.
- No olvidar a la gente involucrada. Tomar notas sobre qué gente estaba allí, qué estaban haciendo, qué observaron y cómo reaccionaron.

Procedimiento de almacenamiento

Cadena de custodia

La cadena de custodia debe estar claramente documentada y se deben detallar los siguientes
puntos:

- ¿Dónde?, ¿cuándo? y ¿quién? descubrió y recolectó la evidencia
- ¿Dónde?, ¿cuándo? y ¿quién? manejó la evidencia.
- ¿Quién ha custodiado la evidencia?, ¿cuánto tiempo? y ¿cómo la ha almacenado?
- En el caso de que la evidencia cambie de custodia indicar cuándo y cómo se realizó el intercambio.

Dónde y cómo almacenarlo

Se debe almacenar la información en dispositivos cuya seguridad haya sido demostrada y que permitan detectar intentos de acceso no autorizados.

Herramientas necesarias

- Se deben utilizar herramientas ajenas al sistema ya que éstas pueden haberse visto comprometidas.
- Se debe procurar utilizar herramientas que alteren lo menos posible el escenario, evitando, en la medida de lo posible, el uso de herramientas de interfaz gráfico y aquellas cuyo uso de memoria sea grande.
- Los programas que se vayan a utilizar para recolectar las evidencias deben estar ubicados en un dispositivo de sólo lectura (CDROM, USB, etc.).
- Se debe preparar un conjunto de utilidades adecuadas a los sistemas operativos con los que se trabaje.
- El kit de análisis debe incluir, entre otros, los siguientes tipos de herramientas:
    - Programas para listar y examinar procesos.
    - Programas para examinar el estado del sistema.
    - Programas para realizar copias bit a bit.

Unidad 9: Protocolo de actuación para pericias informáticas

Introducción

Protocolo de Actuación para Pericias Informáticas detallando la modalidad de trabajo para el envío de material tecnológico al laboratorio pericial. La adopción de estos lineamientos permite una correcta definición del alcance de los servicios profesionales de informática forense, contribuye a profundizar los resguardos en la cadena de custodia de la prueba, y explícita la modalidad de trabajo interno para una mejor adecuación de los requerimientos judiciales.

Objetivos

- Evitar la contaminación de la prueba durante el proceso judicial.
- Formalizar el procedimiento de actuación pericial en materia informática.
- Definir el alcance de los servicios de informática forense.

Procedimiento interno para pericias informáticas

- Descripción general de servicios de informática forense: Se provee un Catálogo de servicios de informática forense no taxativo sino descriptivo e informativo.
    - Sin embargo, los puntos de pericia deberán contener elementos particulares de información objetiva sobre los hechos investigados, para que el especialista pueda aplicar teorías, técnicas y métodos de análisis forense a través de herramientas especializadas.
    - Pericia sobre infracción a la ley de propiedad intelectual del software.
    - Pericia sobre control, actualización y adquisición de licencias de software.
    - Pericia sobre robo, hurto, borrado intencional o accesos no autorizados a la información de una determinada empresa o institución, procesada y/o generada por los sistemas informáticos.
    - Pericia sobre duplicación no autorizada de datos procesados y/o generados por los sistemas informáticos.
    - Pericia sobre métodos y normas a seguir en cuestión de seguridad y privacidad de la información procesada y/o generada por los sistemas informáticos.
    - Pericia sobre la realización de auditorías de áreas de sistemas y centros de cómputos así como de los sistemas informáticos utilizados.
    - Pericia sobre recupero de datos borrados y rastreo de información en los distintos medios informáticos (magnéticos – ópticos).
    - Pericia sobre métodos y normas a seguir en cuestión de salvaguarda y control de los recursos físicos y lógicos de un sistema informático.
    - Pericia sobre desarrollo, manejo e implementación de proyectos informáticos.
    - Pericia sobre contratos en los que la informática se encuentre involucrada (contratación de servicios, adquisición de equipamiento informático y de sistemas, tercerización de servicios).

    - Pericia sobre aspectos laborales vinculados con la informática. Uso de Internet en el trabajo, uso indebido de las facilidades de la organización otorgadas a los empleados (servicio de correo electrónico, acceso a la navegación por Internet, uso de computadoras, entre otros elementos).
    - Pericia sobre robos o determinación de identidad a través de correos electrónicos.
    - Pericia sobre aspectos vinculados al comercio electrónico y operaciones realizadas a través de Internet.
    - Pericia sobre dispositivos de telefonía celular.
- Del procedimiento general de investigación judicial con tecnología informática: En el ámbito penal, el procedimiento general de investigación judicial utilizando servicios de informática forense consta de dos etapas principales:
    - Incautación confiable de la prueba y mantenimiento de la Cadena de Custodia. Llevada a cabo por personal policial junto al Fiscal responsable del control o de la ejecución de la medida.
    - Análisis de la información disponible con arreglo al incidente investigado y redacción del informe pericial. Efectuada en el laboratorio por un Perito
- De la identificación y preservación de evidencia digital: Este protocolo establece que el Fiscal debe planificar cuidadosamente la incautación de pruebas tecnológicas y seguir la guía operativa para el secuestro de tecnología informática. Se enfatiza la importancia de utilizar precintos de seguridad para garantizar la autenticidad e integridad de la evidencia digital. Se recomienda el secuestro del material tecnológico como la mejor opción para preservar la información digital, y se advierte que la pericia informática puede ser imposible sin los elementos originales. También se reconoce que las tareas operativas pueden ser difíciles y que la preservación de información digital en el lugar del hecho debe ser realizada por personal capacitado con los elementos técnicos adecuados y siguiendo una guía de procedimiento.
- De la priorización de casos urgentes: las pericias informáticas tendrán prioridad en casos donde hay personas detenidas o en delitos que involucren bienes jurídicos de gran relevancia, como la vida o la integridad sexual, y donde el tiempo puede afectar la investigación. Si dos casos tienen la misma urgencia, se seguirá el orden de llegada. Los especialistas pueden dar una estimación del tiempo necesario para la pericia según la capacidad operativa disponible y las estadísticas propias de la actividad.
- Del traslado y recepción del material secuestrado: es responsabilidad del personal policial trasladar el material secuestrado a los organismos judiciales y que el requirente debe enviar los elementos probatorios al laboratorio pericial. Se debe tener en cuenta las sanciones previstas por el Código Penal en caso de alterar la cadena de custodia. Se cotejará la existencia de precintos y la identificación correcta de los elementos. Si se detecta alguna alteración o ausencia de precintos, se dejará constancia y cada persona que traslade elementos debe registrar su intervención.
- Del análisis forense: El proceso forense se lleva a cabo con una metodología estándar y un control de calidad previo en el laboratorio pericial. Existe una organización interna con roles definidos para el asistente técnico, perito informático auxiliar y perito informático oficial. La capacitación y la responsabilidad asignada son consideradas en la selección del personal y la definición del alcance y líneas de investigación y la elaboración de dictámenes está a cargo de los peritos de mayor experiencia y jerarquía.

- De la presentación del dictamen: El dictamen será presentado siguiendo los estándares utilizados para la presentación de reportes informáticos forenses. Se intentará minimizar el volumen de información en soporte papel, suministrando toda la información complementaria que sea necesaria para el objeto de la pericia en soporte digital.
- De la remisión del material secuestrado: Luego de finalizada la pericia, se enviará el dictamen y material secuestrado al organismo de origen y se resguardarán los elementos analizados para preservar la integridad y autenticidad de la evidencia digital hasta finalizar el proceso judicial.

Secuestro de tecnología informática

Antes de realizar un allanamiento, se debe realizar una investigación minuciosa para identificar con precisión los elementos a secuestrar mediante inteligencia policial. En situaciones que involucren procedimientos judiciales en empresas o instituciones de gran envergadura, se debe obtener información sobre la infraestructura tecnológica y hardware existente en el lugar del hecho. La pericia informática conlleva mucho tiempo, por lo que se debe evitar el secuestro masivo de elementos informáticos, enviándolos a peritaje solo si se tienen presunciones con un alto grado de verosimilitud de poseer la evidencia buscada. La actuación profesional del perito es principalmente una actividad de laboratorio y de asesoramiento científico al operador judicial responsable de la investigación penal.
Pasos:

1. Separar a las personas de los equipos informáticos lo antes posible, identificar usuarios y personal informático interno.
2. Fotografiar o filmar todos los equipos informáticos y su entorno antes de moverlos o desconectarlos.
3. Evitar tocar el material informático sin uso de guantes descartables, procurando no dificultar una posible obtención de huellas digitales.
4. No apagar ni prender los equipos. Dejar que los realicen especialistas. Luego, desconectar la toma eléctrica y quitar baterías.
5. Identificar equipos telefónicos y su número.
6. Impedir que las personas realicen búsquedas sobre directorios o intente ver la información almacenada en los dispositivos para no alterar la evidencia digital.
7. Identificar correctamente todo el material tecnológico a secuestrar.
    a. Secuestrar dispositivos que tengan gran cantidad de información o que se esté bastante seguro que tienen información importante.
    b. Rotular el hardware.
    c. Fotografiar periféricos
8. Usar bolsas especiales antiestática para almacenar dispositivos electromagnéticos.
9. Precintar cada equipo informático en todas sus entradas eléctricas y todas las partes que puedan ser abiertas o removidas.
10. Resguardar el material informático en un lugar limpio para evitar la ruptura o falla de componentes. Evitar altas temperaturas y campos electromagnéticos.
11. Mantener la cadena de custodia del material informático transportado.